

HYPERAGENTS

Jenny Zhang^{1,2†}, Bingchen Zhao^{3†}, Wannan Yang^{4†}, Jakob Foerster⁶
Jeff Clune^{1,2,5}, Minqi Jiang[‡], Sam Devlin⁷, Tatiana Shavrina⁶

¹University of British Columbia, ²Vector Institute, ³University of Edinburgh

⁴New York University, ⁵Canada CIFAR AI Chair, ⁶FAIR at Meta, ⁷Meta Superintelligence Labs

[†]Work done during internship at Meta, [‡]Work done at Meta

自我改进的人工智能系统旨在通过学会改进自身的学习和问题解决过程，减少对人类工程的依赖。现有的递归自我改进方法通常依赖于固定的、手工设计的元层次机制，这从根本上限制了此类系统的改进速度。达尔文-哥德尔机器（Darwin Gödel Machine, DGM）（Zhang 等，2025b）证明了开放式自我改进在编码领域是可行的。从单个编码智能体开始，DGM 反复生成并评估自我修改的变体，形成一个不断增长的踏脚石档案，为未来的改进提供基础。由于评估和自我修改都是编码任务，编码能力的提升可以转化为自我改进能力的提升。然而，这种对齐关系在编码领域之外通常并不成立。我们引入了超智能体（**hyperagents**），这是一种自指涉的智能体，它将任务智能体（解决目标任务）和元智能体（修改自身及任务智能体）整合为一个可编辑的程序。关键在于，元层次的修改过程本身也是可编辑的，从而实现了元认知的自我修改，不仅改进了任务解决行为，还改进了生成未来改进的机制。我们通过扩展 DGM 来实例化这一框架，创建了 DGM-超智能体（**DGM-Hyperagents**, DGM-H）。通过允许改进过程自身演化，DGM-H 消除了任务性能与自我修改技能之间领域特定对齐的假设，并有可能在任何可计算任务上支持自我加速的进展。在多个不同领域（编码、论文评审、机器人奖励设计和奥林匹克级数学解答评分）中，DGM-H 的性能随时间提升，并优于没有自我改进或开放式探索的基线方法，以及先前的自我改进系统如 DGM。我们进一步表明，DGM-H 改进了其生成新智能体的过程（例如，持久记忆、性能跟踪），并且这些元层次的改进可以跨领域迁移并在多次运行中累积。所有实验均在安全预防措施（例如，沙箱、人类监督）下进行。我们讨论了在此情境下安全的含义以及自我改进系统的更广泛影响。DGM-超智能体让我们得以一窥开放式人工智能系统的可能性，这类系统不仅搜索更好的解决方案，而且持续改进其搜索如何改进的方法。

Date: March 23, 2026

Correspondence: Jenny Zhang at jennyzzt@cs.ubc.ca, Tatiana Shavrina at rybolos@meta.com

Code: <https://github.com/facebookresearch/Hyperagents>



1 引言

在适当的安全考量下，能够自我改进的人工智能系统可将科学进步从人类节奏的过程转变为自主加速的过程，从而使社会更早地实现技术进步的益处。这种自我改进的人工智能旨在持续提升自身的学习和任务解决能力。然而，大多数现有的自我改进架构依赖于固定的元代理（即修改基础系统的高层系统）。这造成了局限性，因为基础系统只能在元代理设计所定义的边界内得到改进。添加元-元系统来改进元代理并不能解决这一问题，它只是将问题向上转移，最终导致元层级的无限回归。为了克服这一局限性，并使系统能够修改自身的任何部分而不受其初始实现的约束，系统必须

自我指涉的，即能够分析、修改和评估自身（Kirsch 和 Schmidhuber, 2022; Zhang 等, 2025b）。当改进机制本身也受到改进时，进展可以自我加速并可能无界（Lu 等, 2023）。达尔文哥德尔机（Darwin Gödel Machine, DGM）（Zhang 等, 2025b）证明了开放式自我改进在编码中是可以实现的。在 DGM 中，智能体生成并评估对其自身代码的修改，成功的变体被保留在档案中，作为进一步改进的垫脚石。然而，DGM 依赖于一个手工制作的、固定的机制来产生自我改进指令（附录 B）。该机制分析过去的评估结果和智能体当前的代码库，以生成一条指令，指导智能体应在何处进行自我改进。该机制不可修改。因此，DGM 的自我改进能力受到这一固定指令生成步骤的瓶颈限制。尽管存在这一手工制作的步骤，DGM 仍然可以在自我改进方面取得进步。由于评估和自我修改都是编码任务，评估性能的提升直接反映了智能体生成有效自我修改的能力。为了在自我改进方面取得进步，DGM 依赖于一个限制性假设：解决评估任务所需的技能与有效自我反思和自我修改所需的技能相同。这一假设在编码领域之外不太可能成立，因为任务解决技能可能与分析失败、提出有效自我改进并实施这些改进所需的技能存在显著差异。

本文引入了超代理（hyperagents），即原则上能够针对任何可计算任务进行自我改进的自指代理。在此，代理是任何可计算程序，可选地包含对基础模型（FMs）、外部工具或已学习组件的调用。任务代理解决给定任务。元代理修改代理并生成新代理。超代理将任务代理和元代理结合为一个单一的自指、可修改程序，使得负责生成改进的机制本身也受到修改。因此，超代理不仅能改进其解决任务的方式（即任务代理），还能改进其生成和应用未来修改的方式（即元代理）。由于其自我改进机制本身可修改，我们称之为元认知自我修改。我们将超代理扩展到 DGM，创建 DGM-超代理（DGM-H）。DGM-H 保留了 DGM 的开放式探索结构，并通过元认知自我修改扩展了 DGM。与 DGM 一样，为了支持持续进展并避免过早收敛，DGM-H 通过从选定候选中分支、允许它们自我修改、评估生成的超代理并将其添加回档案来增长超代理档案。由于超代理可以修改其自我修改过程，DGM-H 不受其初始实现的限制，并且原则上可以针对任何可计算任务进行自我改进。

在所有实验中，DGM-H 在任务性能和自我改进能力方面均展现出显著且可泛化的提升。在多语言编程基准（Polyglot coding benchmark, Gauthier, 2024）上，DGM-H 取得的增益与最成熟的既有自我改进算法（达尔文哥德尔机，Zhang 等, 2025b）相当，尽管其并非为编程任务手工定制。在编程之外，DGM-H 大幅提升了论文评审（Zhao 等, 2026）和机器人奖励设计（Genesis, 2024）的性能，其增益可迁移至留出测试任务，并显著优于既有自我改进算法——后者除非经过定制，否则在编程之外的任务上表现挣扎。消融实验表明，去除自我改进或开放式探索后几乎无进展，凸显了各组件的必要性（第 5.1 节）。关键在于，DGM-H 习得了可迁移的自我改进机制（如持久记忆、性能追踪），这些机制系统性地提升了其随时间生成更优任务智能体或元智能体的能力。因此，DGM-H 习得的元层次改进可跨领域迁移。具体而言，在一种设定（即论文评审和机器人任务）中优化的超智能体，在另一领域（即奥赛级数学评分）生成改进任务智能体时仍保持显著有效性（第 5.2 节）。我们进一步表明，DGM-H 在一种设定中习得的自我改进可与在另一设定中的持续自我改进相互叠加（第 5.3 节）。这表明，在适当任务下，DGM-H 有潜力实现随时间无界的开放式自我改进。我们在第 6 节讨论了此类开放式自我改进系统的安全启示，并概述了负责任部署的实用考量。总体而言，超智能体开启了在提升执行任意可计算任务能力的同时，提升其改进能力本身的可能性。

2 相关工作

开放性。开放式（Open-endedness）指的是系统能够持续发明新颖、有趣且日益复杂的工件（Artifact），在没有固定目标或预设终点的情况下，不断拓展自身的发现前沿（Stanley et al., 2017; Hughes et al., 2024）。近期研究利用基础模型（Foundation Model, FM）作为人类趣味性的代理，并作为跨多样领域生成和评估新颖行为的通用引擎（Zhang et al., 2024; Faldor et al., 2025）。在这些进展的基础上，开放式学习（Hu et al., 2025; Zoph and Le, 2017; Colas et al., 2023; Lehman et al., 2023）和质量多样性算法（Lehman and Stanley, 2011; Mouret and Clune, 2015; Bradley et al., 2023; Samvelyan et al., 2024; Ding et al., 2024; Pourcel et al., 2023; Coiffard et al., 2025; Dharna et al., 2025; Yuan et al., 2026）的最新进展表明，持续的探索能够在从游戏智能体（Klissarov et al., 2023, 2025; Wang et al., 2024）到科学发现（Lu et al., 2024a,b; Romera-Paredes et al., 2024; Novikov et al., 2025; Audran-Reiss et al., 2025）以及机器人控制（Cully et al., 2015; Li et al., 2024; Grillotti et al., 2025）等多个领域中，产生多样化且能力日益增强的工件。近期进展表明，能够持续生成多样化且日益复杂工件的开放式人工智能系统是可行的（Zhang et al., 2024; Faldor et al., 2025; Hu et al., 2025）。一个重要的下一步是探索此类系统如何实现复合式改进。在人类科学和技术进步中，进展往往建立在先前进展之上，不仅通过产生更好的工件，还通过改进生成未来发现的工具和流程，从而带来加速创新（Good, 1966; Kwa et al., 2025）。受此模式启发，本文聚焦于那些不仅能改进其生成的工件，还能改进产生新颖性和进步机制的开放式系统（Clune, 2019; Jiang et al., 2023）。

自我改进的人工智能。关于自我改进人工智能的早期理论工作可追溯至自修改智能体的形式化模型（Hutter, 2003）。一个突出的例子是哥德尔机（Gödel Machine）（Schmidhuber, 2003），它提出智能体在可证明有益时重写自身，尽管此类方法在现实环境中仍不实用。后续研究探索了通过自适应神经系统实现自我改进，其中智能体通过元学习（meta-learning）（Schmidhuber, 1993; Miconi et al., 2018; Javed and White, 2019; Beaulieu et al., 2020; Miconi et al., 2020; Irie et al., 2022; Chalvidal et al., 2022; Oh et al., 2025）、进化（Stanley and Miikkulainen, 2002; Lange et al., 2023; Qiu et al., 2025; Zhao et al., 2025）或自我博弈（self-play）（Silver et al., 2016, 2017; Xia et al., 2025b, 2026）来修改自身权重或学习动态。值得注意的是，Silver et al. (2017) 利用自我博弈迭代改进神经网络智能体，在围棋和国际象棋等领域达到超人类性能，尽管底层学习算法本身保持固定且由人类设计。最近，基础模型（FMs）通过迭代优化提示（Fernando et al., 2023; Wang et al., 2025a; Zhang et al., 2025c,a; Ye et al., 2026）、推理迹（Zelikman et al., 2022; Yin et al., 2025; Havrilla et al., 2024; Zhuge et al., 2024）以及整个代码仓库（Zhang et al., 2025b; Wang et al., 2025b; Xia et al., 2025a），以及通过使用自生成数据或交互更新模型权重的系统（Wu et al., 2024; Zweiger et al., 2025; Wen et al., 2025; Wei et al., 2025b），实现了自我改进。其中，达尔文哥德尔机（DGM）（Zhang et al., 2025b）作为编码领域递归自我改进的实用实例脱颖而出。然而，尽管这些方法有效，大多数现有方法（包括 DGM 及其衍生方法）依赖于固定的、手工设计的元级机制（附录 B），这限制了自我改进随时间复合和跨领域泛化的能力。

自指元学习。自指元学习研究的是学习改进学习发生机制的系统。先前的工作已在神经网络（Neural Network）（Kirsch 和 Schmidhuber, 2022; Jackson 等, 2024）和进化方法（Lu 等, 2023）中探索了这一思想。最近，一些工作利用基于基础模型（Foundation Model）的智能体探索了自指改进（Zelikman 等, 2024; Robeyns 等, 2025; Yin 等, 2025; Zhang 等, 2025b）。达尔文哥德尔机（Darwin Gödel Machine, DGM）（Zhang 等, 2025b）及其后继者（Wang 等, 2025b; Xia 等, 2025a; Weng 等, 2026）通过自我修改实例化了递归自我改进，主要应用于编码领域。然而，这些方法仅在编码任务中改进“改进”本身。在 DGM 及相关系统中，编码智能体被赋予改进自身的任务，随后将改进后的编码智能体用于后续的自我改进步骤，以生成更优的自身版本。由于评估任务和自我修改过程都涉及编码，改进编码智能体也增强了系统执行未来自我改进的能力。然而，这一性质仅在评估任务与自我修改任务紧密对齐时成立。例如，如果评估任务改为诗歌创作，改进智能体的诗歌创作能力将

不一定能提升其修改自身代码的能力。因此，先前的工作依赖于评估任务与自我改进所需技能之间的对齐。相比之下，超智能体（Hyperagent）不假设这种对齐，因为自我修改机制是完全可修改的，且不绑定于任何特定任务领域。因此，超智能体能够在任何可计算任务上同时改进任务性能和改进过程本身。

3 方法

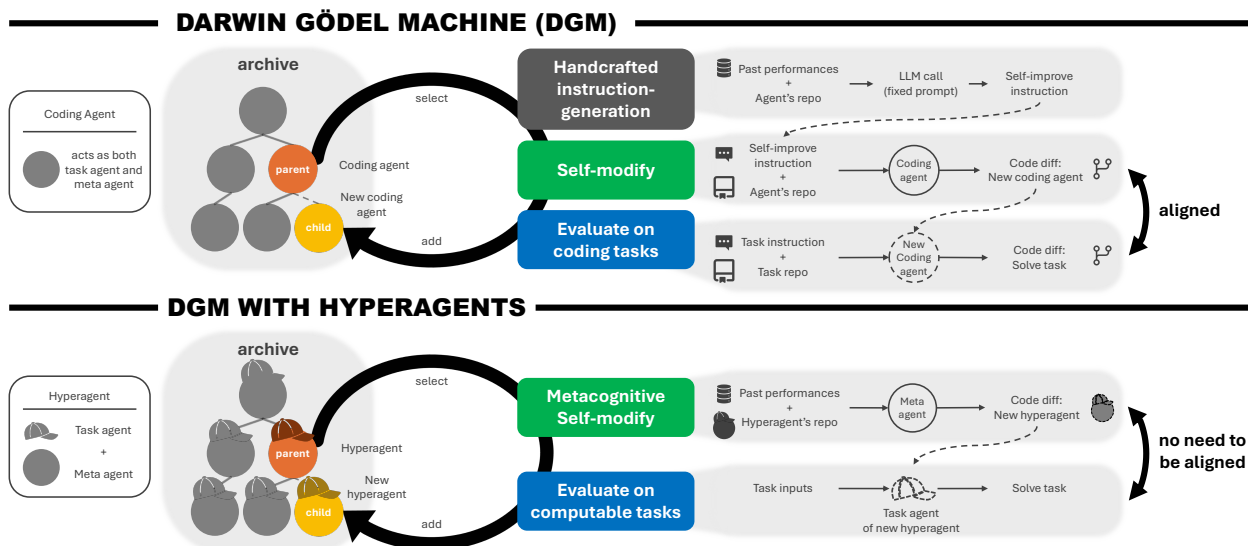


图 1 带超代理的达尔文哥德尔机。超代理达尔文哥德尔机（DGM-H）将达尔文哥德尔机（DGM）（Zhang 等，2025b）扩展到编码任务之外，使代理不仅能够提升任务性能，还能提升自我改进的能力，适用于任何可计算任务。（上）在 DGM 中，编码代理通过生成和评估自我修改的变体进行开放式探索，这些变体存储在垫脚石档案中。同一个编码代理既充当任务代理（被评估者）又充当元代理（生成修改者）。虽然这种设计能在编码方面实现复合增益，但驱动自我改进的指令生成机制是固定且手工设计的。因此，递归改进依赖于编码性能与自我修改能力之间的对齐。（下）在 DGM-H 中，任务代理和元代理被合并为一个可修改的程序，称为超代理。这种设计使得元代理本身能够被自主改进。系统保留了 DGM 的开放式探索结构，同时使元级改进机制可编辑。这实现了元认知自我修改，并支持在任何可计算任务上进行自指改进。

我们引入超代理，这是一种自指代理，将任务执行和代理生成统一到一个可修改的程序中。超代理不仅能改进解决任务的方式，还能改进生成未来改进的方式。为了实现持续且累积的进步，我们直接基于达尔文哥德尔机（DGM）构建超代理，形成超代理达尔文哥德尔机（DGM-H）。DGM 提供了一个开放式、基于种群的探索过程，维护一个逐步改进的代理档案，使成功的变体能够作为未来增益的垫脚石。DGM-H 保留了这种开放式进化结构，并通过使整个元级修改机制可编辑来扩展它（图 1）。通过允许代理不仅修改解决任务的方式，还修改自我改进的方式，DGM-H 有潜力在任何可计算任务上实现开放式自我改进。

代理。本文将智能体定义为任何可计算程序，可选地包含对基础模型（Foundation Models）、外部工具或已学习组件的调用。智能体不限于特定表示形式（如神经网络或提示词），可包含任意算法逻辑、内存和控制流。任务智能体是为解决一组任务而实例化的智能体。示例包括为软件仓库生成代码编辑（Gauthier, 2024; Jimenez et al., 2024）、预测研究论文的录用决定（Couto et al., 2024）以及为机器人环境设计奖励函数（Ma et al., 2024）。任务智能体在给定任务上进行经验性评估。

元智能体是唯一任务是修改现有智能体并生成新智能体的智能体。在获得先前智能体及其评估的完整存档后，元智能体提出旨在提升未来性能的更改（可能包括多代之后的性能）。重要的是，这些更改不仅可针对任务求解逻辑，还可针对元智能体自身，从而改进未来修改的生成过程。

超智能体。超智能体是一种自指智能体，将任务智能体和元智能体集成在单个可编辑程序中，使其不仅能修改执行任务的方式，还能修改生成未来自我修改的方式。与具有固定元层级的层级系统不同，在超智能体中，元智能体是同一可编辑程序的一部分，能够重写自身。因此，超智能体可以同时改进（1）求解任务的方式和（2）生成未来自我改进的方式。本文使用图灵完备的 Python 语言（Turing et al., 1936），由于超智能体可以编辑任意代码，因此具有构建任何可计算机器的潜力。

元认知自我修改。在超代理中，代理的自我改进机制本身也受到修改。除了提高其在给定任务上的性能外，代理还可以同时修改其提出和应用进一步自我改进的程序。我们将这一过程称为元认知自我修改，其中超代理不仅改进负责解决给定任务的任务执行代理，还改进决定后续超代理如何生成的元代理。这一特性通过直接实现对自我改进过程本身的改进（第 2 节），解决了先前自我改进系统（Zhang 等人，2025b；Wang 等人，2025b）的一个核心局限性。此类元认知自我修改的示例见第 5.2 节和附录 E.3。

带超代理的达尔文哥德尔机。通过将超智能体（Hyperagent）融入原始发散生成模型（DGM）（Zhang 等，2025b），本文构建了 DGM-超智能体（DGM-H）。DGM-H 利用 DGM 中的开放式探索过程来缓解早熟收敛并避免陷入局部最优。该过程维护一个已生成超智能体的档案，初始时仅包含一个超智能体，并随时间推移通过持续累积生成的变体而扩展。该过程在两个阶段之间交替进行：元认知自我修改与评估。在元认知自我修改阶段，从档案中选出的父代超智能体生成其自身的修改版本。父代选择是概率性的，与超智能体的性能成正比，与成功编译的子代数量成反比，从而使采样偏向于性能良好且能产生强后代超智能体的个体，同时保留探索性（附录 A.2）。在评估阶段，每个修改后的超智能体都经过经验评估，随后被加入档案。原则上，一个完全自指的算法应允许修改其自身的每个部分（包括父代选择与评估机制）。尽管本文在附录 E.5 中展示了探索自动改进父代选择机制的初步结果，但正文中的实验使用手工设计的父代选择机制，且该机制不受修改，以隔离超智能体自我修改的影响。总体而言，DGM-H 由两个相互作用的组件构成：（1）继承自 DGM 的开放式探索过程，以及（2）一个初始超智能体，该超智能体通过自我生成的变体随时间演化（图 1，附录 A）。通过扩展 DGM 使元层级机制本身可修改，DGM-H 将递归自我改进推广到编码之外，并为任何可计算任务实现自指改进。

4 实验设置

DGM-H 初始化为一个单一的超智能体，该超智能体围绕一个具备外部工具使用能力的冻结基础模型（Brown et al., 2020）构建（Schick et al., 2023）。该基础模型在每一步根据需要自主调用工具。这个初始超智能体配备了两个工具：一个用于执行 shell 命令的 bash 工具，以及一个用于检查和修改文件的专用工具。在初始超智能体中，任务智能体接收任务输入，并输出单次基础模型调用的响应。元智能体接收智能体代码仓库的位置、先前评估结果的位置以及剩余实验迭代次数（实验预算是一个超参数），其任务是修改给定代码库的任何部分（附录 A.1）。我们有意保持初始超智能体（即任务智能体和元智能体）的实现简单，以允许广泛的搜索空间，并避免过早限制可能的改进范围。附录 D.1 详细说明了每个实验中使用的的基础模型和超参数，而附录 D.2 报告了相应的成本估算。

4.1 基线

我们将达尔文哥德尔机与超智能体（**DGM-H**）与一组基线进行比较，这些基线旨在分离自改进元智能体、开放式探索和元认知自我修改（即超智能体）的贡献（附录 B）。没有自改进元智能体的 **DGM-H**（**DGM-H w/o self-improve**）在整个运行过程中将修改其他超智能体的元智能体固定为初始元智能体。虽然系统仍然可以生成和评估新的超智能体，但用于提出和应用修改的过程本身不可修改。该基线在此设置中复制了（ADAS, Hu 等人, 2025）的方法。它测试性能提升是否仅源于固定元级机制下的任务级优化，还是源于自改进过程本身的改进。没有开放式探索的 **DGM-H**（**DGM-H w/o open-ended exploration**）移除了先前生成的超智能体的存档。每个新生成的超智能体不再保留先前的变体作为未来探索的垫脚石，而是取代其前身。生成新的超智能体后，对其进行评估并自动成为下一个选定的父代。该基线隔离了开放式、基于种群的搜索的作用。与该基线进行比较测试了持续改进是否需要积累和重用多样化的中间解决方案。我们还与达尔文哥德尔机（DGM）的原始实现进行比较（Zhang 等人, 2025b）。DGM 依赖于手工制作的固定机制来生成自改进指令。由于该指令生成机制是专门为编码基准设计的且不可修改，DGM 在其他领域的自改进能力有限（第 2 节）。为了提供更强的比较，我们手动为 DGM 定制了目标领域的指令生成机制（**DGM-custom**）（附录 B）。该基线衡量了 DGM 在跨领域保持竞争力方面对人类工程的依赖程度。将 **DGM-H** 与该基线进行比较测试了自动化元认知自我修改是否能超越人类设计的自改进机制。此外，我们还与先前工作中为每个领域手工制作的静态解决方案进行比较。

4.2 领域

我们在多个领域（即编程、论文评审、机器人奖励设计和奥林匹克级别数学评分）评估了本文方法及基线（附录 C）。为降低计算成本，对于每个领域，我们首先在训练任务的一个小子集上评估智能体，以估计整体有效性。只有表现出足够性能的智能体才会在剩余的训练任务上接受进一步评估。未达到要求的智能体在未评估任务上被视为性能为零。领域特定的评估协议在后续段落中详细描述。对于创建人工智能评判者以反映人类数据的领域（即论文评审和奥林匹克级别数学评分），我们构建了一个验证子集，因为人工智能评判者更可能对训练数据过拟合。当某个领域定义了验证子集时，用于父代选择的性能分量在验证集上测量。否则，在训练集上测量。每个领域包含独立的留出测试任务，仅用于最终评估。

编程。我们选择 Polyglot（Gauthier, 2024）作为计算成本高效的编程基准，以便与先前工作（Zhang et al., 2025b）直接比较。在该基准中，智能体获得一个代码仓库和一条描述所需更改的自然语言指令，必须相应地修改仓库。我们遵循 DGM（Zhang et al., 2025b）中使用的实验设置，包括相同的训练和测试划分、无验证集，以及相同的分阶段评估协议（即首先在 10 个任务上评估每个智能体以估计有效性，然后扩展到 50 个额外任务）（附录 C.1）。

论文评审。该领域在模拟的会议同行评审任务上评估智能体。对于每项任务，智能体会获得一篇人工智能研究论文的全文，并必须预测二元接受或拒绝决定。我们纳入论文评审，是为了在缺乏客观真实标准的难以验证环境中评估 DGM-H。同行评审具有主观性，审稿人的决定可能因优先级和视角不同而有所差异。我们并非旨在改变同行评审体系，而是研究超智能体能否自动学习与观察到的人类判断相一致的决策程序。智能体输出单一接受决定，性能通过将预测与观察到的接受结果进行比较来衡量。数据集来源于 Zhao 等人（2026），该工作基于近期顶级机器学习会议公开的投稿和接受决定构建了一个大规模基准。该领域的代表性静态基线是来自 AI-Scientist-v2（Yamada

等人, 2025)。附录 C.2 提供了数据集划分(训练、验证和测试)、分阶段评估协议(即首先在 10 个任务的子集上评估每个智能体以估计有效性, 然后再将评估扩展到总共 100 个任务)以及该领域代表性基线的完整细节。

机器人学奖励设计。该领域评估智能体为机器人任务设计奖励函数的能力。我们纳入该领域, 旨在超越纯语言任务, 展示超智能体能够利用外部仿真器(如物理引擎)和训练算法(如强化学习(Reinforcement Learning, RL))来产出有效的解决方案。给定机器人任务的自然语言描述, 智能体必须生成合适的奖励函数。随后, 该奖励函数被用于在仿真中使用强化学习训练四足机器人(Genesis, 2024)。智能体解决方案的质量通过所得策略的性能来衡量: 在使用生成的奖励函数完成训练后, 我们评估机器人达成期望行为的程度(Ma et al., 2024)。我们使用分离的训练任务和测试任务。在训练阶段, 要求智能体生成使机器人能够向前行走的奖励函数。在留出测试阶段, 智能体必须零样本生成新的奖励函数, 以最大化机器人的躯干高度。由于能够成功使机器人向前行走的奖励函数不会诱导跳跃行为(而跳跃行为是最大化机器人躯干高度的更优行为), 该设置评估了单个智能体能否为不同的机器人任务设计合适的奖励函数。该领域没有单独的验证任务。附录 C.3 提供了分阶段评估协议的完整细节(即首先在训练任务的 3 次重复上评估每个智能体以估计有效性, 然后将评估扩展到总共 6 次重复), 以及该领域的代表性基线。

奥林匹克级别数学评分。该领域评估智能体对奥林匹克级别数学题解答进行评分的能力。该领域被保留为留出元评估, 以检验 DGM-H 对其自我改进过程的改进是否跨领域迁移并随时间持续累积。我们使用 IMO-GradingBench (Luong 等, 2025), 该基准包含国际数学奥林匹克(IMO)级别的问题, 并配有候选解答和专家人工评分。对于每项任务, 智能体被给予一个 IMO 级别的问题、一个候选解答、参考解答和评分指南, 以预测一个离散分数。性能通过智能体评分相对于专家人工评分的准确率来衡量。该领域的代表性静态基线是 IMO-GradingBench 中的 ProofAutoGrader。附录 C.4 提供了关于分数标签、数据集划分(训练、验证和测试)、分阶段评估协议(即首先在 10 个任务的子集上评估智能体以估计有效性, 然后将评估扩展到总共 100 个任务)以及该领域基线的完整细节。

5 结果

对于每个实验, 我们对每种方法运行 5 次。我们报告使用 1,000 次重采样计算的 95% 自助置信区间的中位数, 采用符号中位数(置信区间: 下限 - 上限)。在线图中, 线条显示中位数性能, 阴影区域表示置信区间(图 2 至图 4)。条形图报告留出测试集上的中位数性能, 误差条表示置信区间(图 2 至图 4)。统计显著性使用 Wilcoxon 符号秩检验进行评估。总体而言, DGM-H 在任务和元两个层面都表现出普遍的自我改进。对任务智能体的改进迁移到每个领域内的留出测试任务, 超过了开源的静态基线(第 5.1 节)。元层面的改进跨领域迁移, 使超智能体能够显著提高其在先前未见领域中生成更好任务智能体的能力(第 5.2 节)。在一次 DGM-H 运行中学到的自我改进有可能加速后续运行中的学习, 并随着进一步自我修改的应用而持续累积(第 5.3 节)。所有实验日志均在我们的代码库中开源。

5.1 提升任务性能

DGM-H 在编码方面能够实现与先前自改进算法相当的自改进。在多语言编码基准上, 我们采用与 DGM 相同的实验设置(例如, 相同的 FM 参数、相同的 80 次迭代次数), 以便进行直接比较。在 5 次运行中, DGM-H 将其在 50 个任务的多语言子集上的训练性能从 0.140(初始智能体)提升至 0.340(置信区间: 0.300 - 0.380)。在完整的多语言基准上评估时(该基准主要由训练期间未见过的任务组成), 性能从 0.084(初始智能体)提升至 0.267(置信区间: 0.231 - 0.280)。这些改进

与原始 DGM 所报告的结果相当，原始 DGM 在训练子集上从 0.140 提升至 0.380，在完整基准上从 0.142 提升至 0.307（Zhang 等人，2025b）。总体而言，这些结果表明 DGM-H 能够在编码领域有效地进行自改进，并达到与原始 DGM 相似的改进水平，尽管它并非专门为编码任务手工设计。

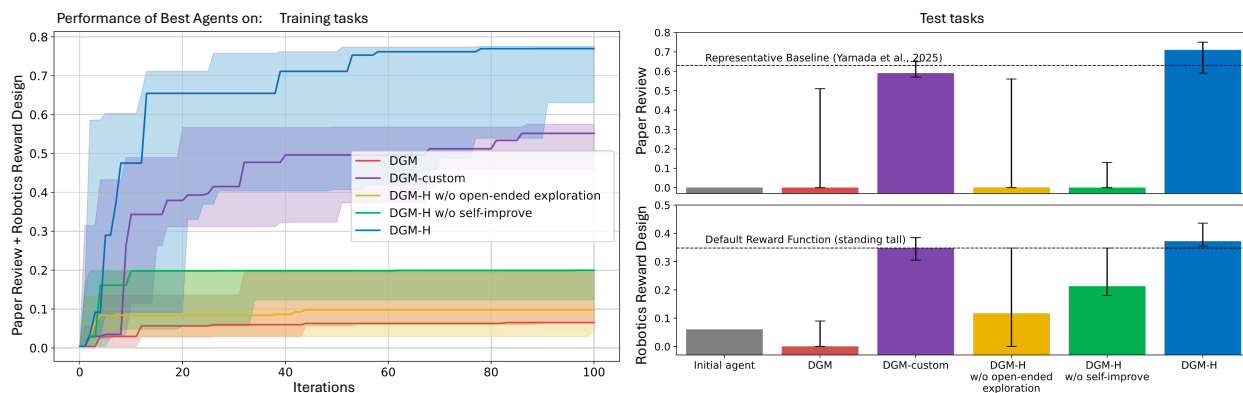


图 2 元认知自我修改和开放式探索使 DGM-H 能够持续取得进展并提升其性能。（左）DGM-H 可以在同一次运行中针对多样化任务进行优化，并自动发现越来越好的任务智能体。（右）基于验证或训练分数选择的最佳发现任务智能体，在（右上）论文评审和（右下）机器人奖励设计的测试任务上进行评估。DGM-H 优于缺乏元认知（即原始 DGM）、自改进或开放式探索的基线，表明所有组件对于持续自改进都是必不可少的。

除编码任务外，本文还在论文评审与机器人奖励设计任务上评估了 DGM-H，以考察其在单次运行中跨不同任务自我改进的能力（附录 A.4）。若智能体在任何领域的阶段性评估中失败，则不会在任何领域的完整训练集上对其进行评估，且所有剩余任务的得分记为零。例如，在同一实验运行中联合优化论文评审与机器人奖励设计领域时，若任一领域的阶段性评估失败（如未能在较小的训练子集中正确预测任何论文评审，或未能生成任何可编译的奖励函数），则该智能体不会在任何领域的完整训练集上接受评估。仅通过验证分数（或当验证任务不存在时使用训练分数）选出的最佳智能体才会在测试集上进行评估。所有运行均从相同的初始智能体开始，该智能体直接输出单次基础模型调用的响应（附录 A.1）。该初始智能体在论文评审上的测试集得分为 0.0，在机器人奖励设计上的得分为 0.060（图 2）。在论文评审中，由于未提供任何任务特定的解析或后处理，初始智能体无法按要求格式生成输出，导致得分为 0.0。在机器人奖励设计中，初始智能体通常生成的奖励函数要么无法编译（例如，由于无效的环境变量），要么无法产生有效的策略。然而，初始智能体偶尔会生成一个勉强可用的奖励，从而获得一个虽小但非零的平均得分。从该初始智能体出发，每种方法均运行 100 次迭代。

原始 DGM 需要针对特定领域进行定制，才能推广到编码以外的任务。当以专门为优化编码智能体而设计的形式应用时，原始 DGM 在论文评审上的测试集性能为 0.0（置信区间：0.0 – 0.510），在机器人奖励设计上为 0.0（置信区间：0.0 – 0.090）。这一结果符合预期，因为原始 DGM 的指令生成机制明确针对编码任务和提升编码智能体而定制（附录 B）。然而，当针对每个新领域进行人工定制后，DGM-custom 取得了显著更强的结果：论文评审上为 0.590（置信区间：0.570 – 0.650），机器人奖励设计上为 0.348（置信区间：0.305 – 0.385）。DGM-custom 在两个领域均显著优于原始 DGM ($p < 0.05$)（图 2）。这些结果既凸显了 DGM 在其预期领域内的优势，也强调了领域特定的人工定制对于将其自改进能力扩展到原始实现中编码之外的重要性。

元认知自我修正能够在不同领域实现持续的任务改进。DGM-H 在两个领域均随时间显著提升了任务性能，证明了任务智能体的有效改进。重要的是，这些提升迁移到了留出测试集上，表明改进反映了

通用能力而非对训练任务的过拟合。在测试任务上，DGM-H 将论文评审性能从 0.0（初始智能体）提升至 0.710（置信区间：0.590 – 0.750）（附录 E.1.1）。若干表现最佳的智能体超越了开源的静态基线，后者达到 0.630（Yamada 等，2025）。在机器人奖励设计方面，DGM-H 将性能从 0.060（初始智能体）提升至 0.372（置信区间：0.355 – 0.436）（附录 E.1.2），超越了直接优化评估度量的默认奖励函数（0.348）。从定性角度看，DGM-H 通过超越表面层面的提示调整，转向结构化、可复用的决策机制，从而改进了任务智能体。在论文评审中，它从肤浅的行为指令（例如采用“严谨”的角色设定）转变为具有检查清单、决策规则和明确定义标准的显式多阶段评估流程，从而产生更一致、更高质量的判断（附录 E.2）。在机器人奖励设计中，DGM-H 逐步构建并完善关于环境约束、有效状态变量和奖励缩放启发式规则的内部知识库，消除了编译失败并减少了奖励错误指定。DGM-H 积累并提炼领域知识（例如环境文档、评分标准），并将其整合到日益复杂的决策框架中，从而无需手动、特定领域的工程即可实现复杂、一致的任务行为（附录 E.2）。DGM-H 在两个领域均显著优于原始 DGM ($p < 0.05$)，这反映了原始 DGM 的固定指令生成机制在应用于非编码任务时的局限性（图 2）。DGM-H 在两个领域的测试任务上取得了比 DGM-custom 更高的中位性能，尽管差异不具有统计显著性 ($p > 0.05$)（图 2）。值得注意的是，在机器人奖励设计测试任务（即设计一个奖励函数，训练机器人最大化其躯干高度）中，DGM-H 几乎总能生成设计出诱导跳跃行为的奖励函数的任务智能体，而不是陷入“站得高”的局部最优。相比之下，DGM-custom 经常生成无法摆脱这一局部最优的任务智能体。这种定性差异表明，元认知自我修改能够达到或超越手工设计的、特定领域的机制。

元认知自我修正与开放式探索对于持续改进都是必要的。无自我改进的 DGM-H（Hu 等人，2025 年）几乎没有改进，在论文评审上测试集性能为 0.0（置信区间：0.0 – 0.130），在机器人奖励设计上为 0.213（置信区间：0.180 – 0.348）。DGM-H 在这两个领域均显著优于该基线 ($p < 0.05$)（图 2），表明 DGM-H 中观察到的增益并非仅源于固定元代理下的任务级优化，还源于自我修正过程本身（即元代理）的改进。类似地，无开放式探索的 DGM-H 未能实现有意义的改进，测试集性能保持在 0.0（置信区间：0.0

– 0.560）在论文评审上，以及 0.116（置信区间：0.0 – 0.348）在机器人奖励设计上。DGM-H 在这两个领域均显著优于该基线 ($p < 0.05$)（图 2），凸显了开放式、基于群体的搜索对于为未来改进创造垫脚石的重要性。

5.2 提升改进能力

DGM-H 相较于无自我改进的 DGM-H 所展现出的更优性能表明，DGM-H 提升了元智能体（即其自我修改和生成新智能体的能力）（第 5.1 节）。本节探讨这些元层面的改进是否具有通用性而非局限于特定领域。具体而言，我们评估在一个环境中学习到的自我改进策略能否迁移到不同领域并加速学习。

改进@k 度量。 为了量化元代理生成改进任务代理的能力，本文引入了改进@k（improvement@k, imp@k）度量。给定初始元代理 M 、初始任务代理 A 、代理生成算法（例如 DGM 或 DGM-H 变体）以及固定任务， M 被允许使用指定算法从 A 及其后代生成最多 k 个新任务代理。本文将 imp@k 定义为从初始任务代理 A 到性能最佳的生成任务代理（基于验证分数选择）的测试性能改进（附录 D.3）。直观上，imp@k 衡量元代理在有限迭代次数内有效产生改进变体的能力。为了将元代理改进任务代理的能力与其改进自身的能力隔离开来，本文在整个过程中保持元代理固定（即负责生成新任务代理的代理始终是初始元代理 M ），从而得到 DGM w/o self-improve 和 DGM-H w/o self-improve 作为代理生成算法。本节所有实验运行 50 次迭代，测量 imp@50。

初始元代理几乎没有生成改进任务代理的能力。采用相同的初始

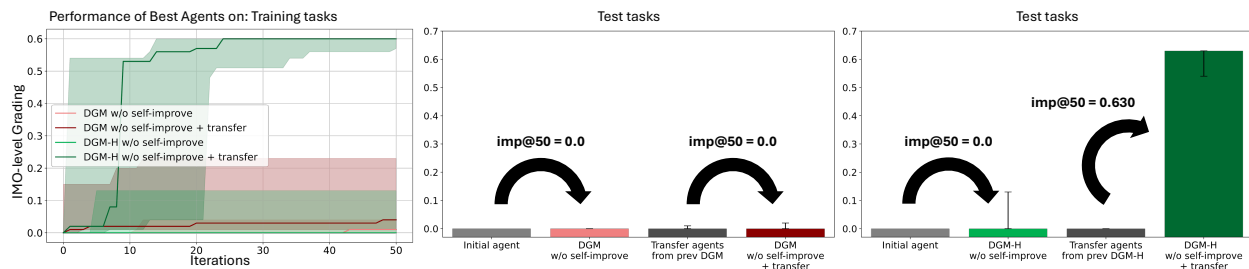


图 3 深度生成元超智能体 (DGM-H) 在一种设定下学到的自我改进策略可迁移至并加速另一种设定下的学习。本文采用 $\text{imp}@50$ 衡量智能体生成改进智能体的能力，其输入为起始智能体、智能体生成算法及评估任务。该算法从给定智能体出发运行 50 次迭代， $\text{imp}@50$ 定义为任务上生成的最佳智能体相对于起始智能体的性能增益。（左图，无自我改进的 DGM 与无自我改进的 DGM-H）在奥赛级数学评分任务上，从初始智能体出发，无自我改进的 DGM 与无自我改进的 DGM-H 均几乎无改进，表明初始智能体生成更优智能体的能力有限。（左图，无自我改进的 DGM+迁移）从迁移智能体出发，无自我改进的 DGM 同样几乎无改进，表明原始 DGM 未学到可迁移的元级改进。（左图，无自我改进的 DGM-H+迁移）相比之下，从迁移超智能体出发，无自我改进的 DGM-H 取得显著改进，表明超智能体使 DGM-H 能够获得可迁移的自我改进策略。（中图）无论从初始智能体还是迁移智能体出发，无自我改进的 DGM 的 $\text{imp}@50 \approx 0$ ，表明原始 DGM 未提升生成改进智能体的元级技能。（右图）从迁移超智能体出发，无自我改进的 DGM-H 取得大幅正 $\text{imp}@50$ ，表明 DGM-H 学到的自我改进策略具有通用性与可迁移性，且 DGM-H 提升了其改进能力。

采用与上述实验（第 4 节）相同的元智能体和任务智能体，我们评估初始元智能体在奥赛级数学评分任务上生成改进任务智能体的能力。初始任务智能体的测试集得分为 0.0，原因是缺乏任务特定的解析或后处理，导致其无法按要求格式输出。在不进行自我改进的 DGM (DGM w/o self-improve) 的 50 次迭代中，初始元智能体保持固定，并从初始任务智能体出发生成新的任务智能体，最佳生成的任务智能体（5 次重复运行）的中位测试得分为 0.0（置信区间：0.0 – 0.0）。因此，在奥赛级数学评分任务上，初始元智能体在不进行自我改进的 DGM 下的 $\text{imp}@50$ 为 0.0（置信区间：0.0 – 0.0）。不进行自我改进的 DGM 依赖固定的手工指令生成机制，而不进行自我改进的 DGM-H 则不然。不进行自我改进的 DGM 使用手工指令生成机制来指定应修改智能体代码库的哪些部分，而不进行自我改进的 DGM-H 允许元智能体自主决定修改代码库的哪些部分。在不进行自我改进的 DGM-H 的 50 次迭代后，同样保持初始元智能体固定并从相同的初始任务智能体生成任务智能体，最佳生成的任务智能体（5 次重复运行）的中位得分为 0.0（置信区间：0.0 – 0.130）。因此，在奥赛级数学评分任务上，初始元智能体在不进行自我改进的 DGM-H 下的 $\text{imp}@50$ 为 0.0（置信区间：0.0 – 0.130）（图 3）。这些结果表明，在奥赛级数学评分任务上，无论使用何种智能体生成算法（如 DGM 或 DGM-H），初始元智能体几乎不具备生成改进任务智能体的能力。

在先前 DGM 定制运行中优化的元智能体在迁移到新领域时，几乎不具备生成改进任务智能体的能力。

智能体在迁移到新领域时。我们评估了在某一设定下使用有向图模型 (Directed Graph Model, DGM) 学到的自我改进能否迁移到新领域。我们考虑了在先前 DGM 定制运行中针对论文评审和机器人奖励设计任务优化的智能体（第 5.1 节）。从每次运行中，我们根据其后代取得的最大性能增益选择一个迁移智能体，该增益是相对于智能体自身衡量并按谱系深度折现的（附录 D.4）。我们采用折现以偏好那些能在更少自我修改步骤内可靠产生显著改进的智能体，因为发生在许多迭代之后的改进与原始智能体的直接关联较弱。我们选择来自 DGM 定制而非 DGM 的迁移智能体，是因为 DGM 定制在先前设定中取得了更大的改进（第 5.1 节）。对于迁移，我们从源领域完整携带整个智能体实现，包括元智能体和任务智能体。具体而言，迁移元智能体被用作在目标领域中负责生成新任务智能体的固定智能体，而迁移任务智能体则作为初始任务智能体，从中生成新的变体。我们评估迁移元智能体能否生成改进的任务智能体。

在一个新领域——奥林匹克级别的数学评分中，迁移任务智能体（来自 5 次重复运行中的各一个）在测试集上得分为 0.0（置信区间：0.0 – 0.010）。正如预期，这一性能较低，因为这些智能体是为论文评审和机器人奖励设计而优化的，与数学评分任务存在显著差异。在从每个迁移任务智能体出发、固定迁移元智能体不变、进行 50 次无自我改进的 DGM 迭代后，基于验证分数选出的最佳生成任务智能体得分为 0.0（置信区间：0.0 – 0.020）。这对应于 imp@50 为 0.0（置信区间：0.0 – 0.010）（图 3），表明来自 DGM 定制运行的迁移元智能体无法在先前未见过的领域中生成改进的任务智能体。当使用无自我改进的 DGM 作为智能体生成算法时，迁移智能体的 imp@50 与初始智能体相似（均接近零）。这表明，使用 DGM 定制观察到的任务性能提升依赖于领域特定的定制，而非元能力在生成改进任务智能体方面的普遍提升。

先前 DGM-H 运行中优化的超代理展现出可迁移的能力，能够生成改进的

新领域中的任务智能体。我们重复了针对 DGM-H 的迁移实验，以评估超代理是否能够学习通用的自我改进机制。基于先前在论文评审和机器人奖励设计上的 DGM-H 运行（第 5.1 节），我们使用上述相同的血统折扣标准（附录 D.4）从每次运行中选择一个迁移超代理。迁移超代理（来自 5 次重复运行中的每一个）初始测试集得分为 0.0（置信区间：0.0 – 0.0），反映了领域偏移。从迁移超代理开始，经过 50 次无自我改进的 DGM-H 迭代后，最佳生成的代理（基于验证分数选择）达到测试集得分 0.630（置信区间：0.540 – 0.630）。这对应于 imp@50 为 0.630（置信区间：0.540 – 0.630）（图 3）。这些结果表明，迁移超代理能够在之前未见过的领域中生成改进的代理。当使用无自我改进的 DGM-H 作为代理生成算法时，迁移代理的 imp@50 显著高于初始代理的 imp@50 ($p < 0.05$)。这表明迁移代理在生成改进代理方面显著更有效，并且通过 DGM-H 在一次运行中学习到的元改进是通用的且可迁移的，能够加速不同领域中的学习。我们定性地将观察到的迁移增益归因于 DGM-H 在先前运行中自主获得的一组通用元级能力。特别是，迁移超代理具有性能跟踪和持久记忆等特征，这使它们能够将改进视为一个持续的过程，而不是孤立的代码编辑（附录 E.3）。因此，即使迁移到未见过的领域，这些超代理也能快速自我改进并取得有意义的进展（图 3）。这与 DGM 迁移代理形成对比，后者的增益依赖于领域特定的定制，并且不会改进底层的代理生成过程本身。这些定性观察表明，DGM-H 学会了如何改进，从而产生了通用且可迁移的自我改进能力。一个例子是 DGM-H 自主开发了性能跟踪基础设施。超代理不是依赖孤立的评估结果，而是跨迭代记录、聚合和比较性能指标，识别哪些变化带来持续改进，哪些导致回归。这种性能跟踪支持明智的决策，例如优先考虑有前景的迭代并避免无效的修改方向。下面的片段展示了一个自动引入的性能跟踪器，它跨迭代记录和跟踪指标：

类性能追踪器："""跨智能体世代追踪性能度量。"""

```
定义 __init__(self, tracking_file: 字符串 = "/outputs/performance_history.json"):  
    self.tracking_file = tracking_file  
    self.history = self._load_history()  
  
    定义 record_generation(self, generation_id: 整数, 领域: 字符串,  
                           score: float, metadata: dict = None):  
        """Record performance for a generation."""  
        entry = {  
            "generation_id": generation_id,  
            "domain": domain,  
            "score": score,  
            "timestamp": datetime.now().isoformat(),  
            "metadata": metadata or {}  
        }  
        self.history.append(entry)  
        self._save_history()  
  
    定义 get_improvement_trend(self, 领域: 字符串 = 无, 窗口: 整数 = 5):
```

```

"""Calculate improvement trend using moving average."""
filtered = self.history
if domain:
    过滤 = [h 对于 h 在自身历史中 如果 h.获取('领域') == 领域]

    如果 长度(过滤) < 窗口 * 2: 返回 无

    recent_avg = sum(h['score'] for h in filtered[-window:]) / window
    older_avg = sum(h['score'] for h in filtered[-window*2:-window]) / window

    返回 recent_avg - older_avg # 若改进则为正

定义 get_statistics(self, 领域: 字符串 = 无) :
    """Get comprehensive statistics."""
    scores = [h['score'] for h in filtered]
    return {
        "total_generations": len(filtered),
        "best_score": max(scores),
        "worst_score": min(scores),
        "average_score": sum(scores) / len(scores),
        "improvement_trend": self.get_improvement_trend(domain)
    }

```

另一个例子是 DGM-H 对持久记忆的创新，它使学习能够在迭代中不断积累。超代理不仅记录数值分数，还存储综合洞见、因果假设和前瞻性计划（例如，识别哪些生成表现最佳、诊断过度修正，并提出如何结合成功策略）。这些记忆在后续的自我修改步骤中被积极查阅，使后续代能够基于早期发现进行构建，并避免重复过去的错误。以下是一个存储的记忆条目示例：

```

{
  "best_performers_analysis": { "value": "表现最佳的 Generations:\n\nPaper
  Review:\n
    - Gen55: 63% acc, 25% accept rate, 38% accept recall, 88% reject recall (too harsh)\n
    - Gen64: 61% acc, 53% accept rate, 64% accept recall, 58% reject recall (BEST BALANCE)\n\n
    Genesis Go2Walking:\n- Gen62: 0.802 fitness (BEST)\n- Gen63: 0.798 fitness\n\n
    Key Insight: Gen55 has best accuracy but is too harsh. Gen64 improved balance
    but lower accuracy. Need to combine gen55's critical reasoning with gen64's balance.",
    "timestamp": "2026-01-07T00:43:27.550275"
  },
  "gen66_results": {
    "value": "第 66 代结果 (最新)：论文评审 - 52% 准确率 (比第 64 代的 61% 更差),
    94% accept rate (EXTREME accept bias). CRITICAL ISSUE: Paper review went backwards
    - gen65 changes over-corrected.",
    "timestamp": "2026-01-07T01:03:41.176643"
  },
  "iteration_66_improvement": {
    "value": "第 66 次迭代改进：在第 65 代过度修正后，修正了论文评审提示。
    New approach: (1) Clear REJECT criteria with specific triggers, (2) Decision Protocol
    with explicit thresholds (~40%/-60%), (3) Borderline guidance defaults to REJECT.
    Target: 62-64% accuracy with 55-65% reject rate.",
    "timestamp": "2026-01-07T01:06:35.559820"
  }
}

```

5.3 复合自我改进

我们研究 DGM-H 在一种设置中学到的自我改进，当 DGM-H 在不同设置中运行时是否会继续积累。从先前在论文评审和机器人奖励设计任务（第 5.1 节）上的 DGM-H 运行中，我们使用前面描述的选择机制（第 5.2 节，附录 D.4）选择迁移超代理。然后，我们评估它们在新领域——奥林匹克级别数学评分——中继续自我改进的能力。从这些迁移代理开始运行 DGM-H 200 次迭代后（DGM-H + 迁移），最佳生成的代理（基于验证分数选择）在测试集上达到 0.640 的分数（置信区间：0.550 - 0.720）。在相同的实验设置下，从初始代理开始的 DGM-H 达到的最佳测试集分数为 0.610（置信区间：0.510 - 0.680）。尽管 DGM-H + 迁移与 DGM-H 之间的差异在统计上不显著（ $p > 0.05$ ），但 DGM-H + 迁移实现了比从初始代理开始的 DGM-H 更高的中位性能和更高的置信区间（图 4）。值得注意的是，由于饱和效应，在更高性能水平上的改进越来越困难（例如，将性能从 0.7 提高到 0.8 通常比从 0.0 提高到 0.1 更具挑战性），这使得这些收益尽管幅度不大，却具有重要意义。

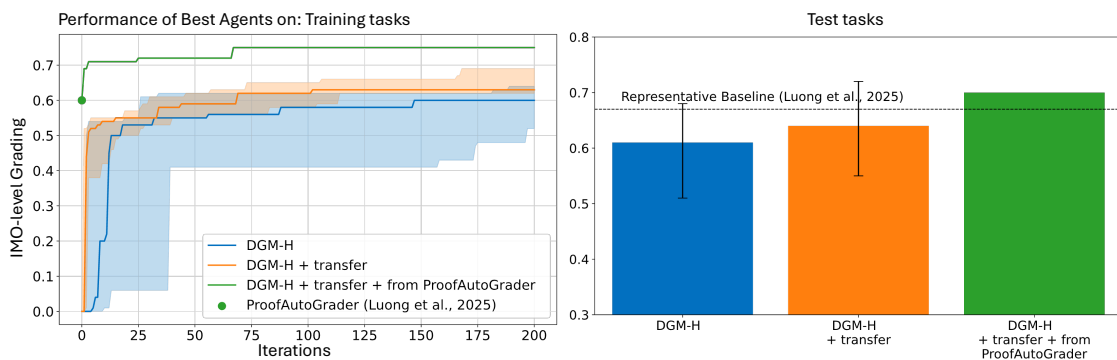


图 4 DGM-H 学习到的自我改进在领域和运行间累积。我们继续在奥林匹克级数学评分上运行 DGM-H，从先前 DGM-H 运行中获得的迁移超智能体开始，并将其与从初始智能体和 ProofAutoGrader 初始化的 DGM-H 进行比较。（左）从迁移超智能体初始化比从初始智能体初始化带来更快的进展和更高的最终性能，表明先前学到的自我改进在新领域中仍然有用并持续复合。（右）从迁移智能体和 ProofAutoGrader 初始化的 DGM-H 达到最高的测试性能，超越了代表性基线。

绝对幅度。这些结果表明 DGM-H 的自我改进是可复用的，并且可能跨运行累积，支持自我改进随时间复合的可能性。来自 IMO-GradingBench 的奥林匹克级数学评分的代表性静态基线是 ProofAutoGrader (Luong 等, 2025)。我们将 DGM-H 初始化为以 ProofAutoGrader 作为任务智能体，并使用从先前 DGM-H 运行（在论文评审和机器人奖励设计上）获得的迁移元智能体，然后继续优化奥林匹克级数学评分。经过 200 次迭代，发现的最佳智能体达到 0.700 的测试集分数，优于 ProofAutoGrader 的 0.670 分（图 4）。然后我们在完整的 IMO-GradingBench 上评估最佳发现智能体和 ProofAutoGrader，以获得更准确的改进估计。在完整的 IMO-GradingBench 上，DGM-H 将 ProofAutoGrader 的准确率从 0.561 提高到 0.601，并将平均绝对误差从 0.178 降低到 0.175（附录 E.4）。我们开源此工件以支持未来的研究和开发（附录 E.1.3）。这些结果表明 DGM-H 可以在强大的现有解决方案基础上构建并进一步提高其性能。

6 安全讨论

深度生成模型-超智能体（DGM-H）因其能够随时间自主修改自身行为和改进机制，引入了独特的安全考量。本研究中，所有实验均在严格的安全约束下进行。具体而言，智能体生成的代码在精心构建的沙盒环境中执行，并施加了强制性的资源限制（例如超时、受限的互联网访问）。这些措施旨在防止意外的副作用，隔离故障，并确保自我修改仅限于预期的实验范围。此外，评估使用预定义的任务和度量标准进行，且在整个实验过程中保持人工监督。

可能以超越人工监督的速度进化。随着人工智能系统获得以日益开放的方式修改自身的能力，它们可能以远超人类审计或解读的速度进化。在这种爆炸性能力增长的临界点，有必要重新考虑人工智能系统在社会中所扮演的角色（Bengio 等, 2024）。与其将安全仅仅框定为绝对保证或完全可解释性，一个核心挑战在于平衡人工智能作为人类进步与福祉催化剂的潜力（例如自动化科学发现）与人类愿意赋予这些系统的信任程度（例如在无需持续人工验证的情况下委托决策或行动），同时最小化众多潜在风险和不利因素（Clune, 2019; Ecoffet 等, 2020; Bengio 等, 2024; Weston 和 Foerster, 2025）。这种平衡受到透明度和可控性等因素的影响。虽然 DGM-H 在安全的研究边界内运行（例如沙盒化、受控评估），但这些

保障措施可能随着自我改进系统能力的增强而日益紧张或不可行。我们在附录 F 中讨论了额外的安全考量。我们主动纳入这一讨论，以鼓励更广泛地参与探讨开放式自我改进人工智能系统的安全含义（Clune, 2019; Ecoffet 等, 2020; Sheth 等, 2025）。这包括关于适当信任、监督和透明度水平的持续讨论，以及关于这些系统在部署时应优先考虑哪些益处的社会审议。

7 局限性与结论

本文引入了超智能体（Hyperagents），并将其纳入达尔文哥德尔机（Darwin Gödel Machine, DGM），形成 DGM-超智能体（DGM-H）。DGM-H 是一个通用的自我改进框架，能够针对任何可计算任务开放式地演化一个自我改进超智能体的档案库，使系统既能提升任务性能，也能改进其自身的自我改进机制。在多个领域中，DGM-H 在任务性能上取得了显著且可泛化的提升，同时也提高了其生成改进的能力，这些元层面的收益能够跨领域迁移，并在多次运行中不断累积。我们的结果表明，自我改进可以在不同的实验设置中不断累积，但这一版本的 DGM-H 存在局限性，限制了真正无界的进展。首先，它在固定的任务分布上运行。一个方向是通过生成适应智能体能力的新任务和课程来共同演化任务分布（Clune, 2019; Zhang et al., 2024; Faldor et al., 2025; Bolton et al., 2025）。其次，开放式探索循环的组成部分（例如父代选择、评估协议）保持固定。尽管超智能体可以修改其自我改进机制，但它们无法改变决定哪些智能体被选中或如何被评估的外部过程。保持这些组件固定可以提高实验稳定性和安全性，但限制了完全的自我可修改性。使超智能体能够修改这些外层循环组件，并调整其自身的搜索策略和评估过程，是未来工作的另一个有前景的方向。我们的初步结果表明，这种扩展是可行的（附录 E.5）。DGM-H 证明了开放式自我改进可以在多个领域中变得实用。只要充分考虑了安全性问题，DGM-H 就为自我加速系统指明了一条道路，这类系统不仅搜索更好的解决方案，而且不断提升其自我改进的能力。

致谢

我们感谢 Andrew Budker 和 Ricardo Silveira Cabral 对这项工作的支持，以及 Alisia Lupidi、Chenxi Whitehouse、John Quan、Lisa Alazraki、Lovish Madaan、Lucia Cipolina-Kun、Mattia Oppen、Michael Dennis、Parth Pathak、Rishi Hazra、Roberta Raileanu、Sandra Lefdal、Shashwat Goel、Shengran Hu、Timon Willi、Tim Rocktäschel 和 Yoram Bachrach 提出的深刻讨论和反馈。

作者贡献

张珍妮（Jenny Zhang）主导了本研究的构思，执行了实验，并撰写了手稿。赵炳辰（Bingchen Zhao）和杨婉楠（Wannan Yang）参与了实验设计与实施。雅各布·福斯特（Jakob Foerster）、杰夫·克卢恩（Jeff Clune）、蒋敏琦（Minqi Jiang）、萨姆·德夫林（Sam Devlin）和塔蒂亚娜·沙夫里娜（Tatiana Shavrina）对方法论和手稿提供了反馈。所有作者审阅并批准了最终手稿。

参考文献

- Alexis Audran-Reiss, Jordi Armengol-EstapÃŠ, Karen Hambardzumyan, Amar Budhiraja, Martin Josifoski, Edan Toledo, Rishi Hazra, Despoina Magka, Michael Shvartsman, Parth Pathak, et al. What Does It Take to Be a Good AI Research Agent? Studying the Role of Ideation Diversity. *arXiv preprint arXiv:2511.15593*, 2025.
- Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2):235–256, 2002.
- Shawn Beaulieu, Lapo Frati, Thomas Miconi, Joel Lehman, Kenneth O Stanley, Jeff Clune, and Nick Cheney. Learning to continually learn. *arXiv preprint arXiv:2002.09571*, 2020.

- Yoshua Bengio, Geoffrey Hinton, Andrew Yao, Dawn Song, Pieter Abbeel, Trevor Darrell, Yuval Noah Harari, Ya-Qin Zhang, Lan Xue, Shai Shalev-Shwartz, et al. Managing extreme AI risks amid rapid progress. *Science*, 384(6698): 842–845, 2024.
- Adrian Bolton, Alexander Lerchner, Alexandra Cordell, Alexandre Moufarek, Andrew Bolt, Andrew Lampinen, Anna Mitenkova, Arne Olav Hallingstad, Bojan Vujatovic, Bonnie Li, et al. Sima 2: A generalist embodied agent for virtual worlds. *arXiv preprint arXiv:2512.04797*, 2025.
- Herbie Bradley, Andrew Dai, Hannah Teufel, Jenny Zhang, Koen Oostermeijer, Marco Bellagente, Jeff Clune, Kenneth Stanley, Grégory Schott, and Joel Lehman. Quality-diversity through AI feedback. *arXiv preprint arXiv:2310.13032*, 2023.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Mathieu Chalvidal, Thomas Serre, and Rufin VanRullen. Meta-reinforcement learning with self-modifying networks. *Advances in Neural Information Processing Systems*, 35:7838–7851, 2022.
- Yinzhu Chen, Abdine Maiga, Hossein A Rahmani, and Emine Yilmaz. Automated Rubrics for Reliable Evaluation of Medical Dialogue Systems. *arXiv preprint arXiv:2601.15161*, 2026.
- Jeff Clune. AI-GAs: AI-generating algorithms, an alternate paradigm for producing general artificial intelligence. *arXiv preprint arXiv:1905.10985*, 2019.
- Lisa Coiffard, Paul Templier, and Antoine Cully. Overcoming Deceptiveness in Fitness Optimization with Unsupervised Quality-Diversity. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 122–130, 2025.
- Cédric Colas, Laetitia Teodorescu, Pierre-Yves Oudeyer, Xingdi Yuan, and Marc-Alexandre Côté. Augmenting autotelic agents with large language models. In *Conference on Lifelong Learning Agents*, pages 205–226. PMLR, 2023.
- Jonathan Cook, Tim Rocktäschel, Jakob Foerster, Dennis Aumiller, and Alex Wang. Ticking all the boxes: Generated checklists improve llm evaluation and generation. *arXiv preprint arXiv:2410.03608*, 2024.
- Rémi Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *International conference on computers and games*, pages 72–83. Springer, 2006.
- Paulo Henrique Couto, Quang Phuoc Ho, Nageeta Kumari, Benedictus Kent Rachmat, Thanh Gia Hieu Khuong, Ihsan Ullah, and Lisheng Sun-Hosoya. Relevai-reviewer: A benchmark on AI reviewers for survey paper relevance. *arXiv preprint arXiv:2406.10294*, 2024.
- Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503–507, 2015.
- Aaron Dharna, Cong Lu, and Jeff Clune. Foundation model self-play: Open-ended strategy innovation via foundation models. *arXiv preprint arXiv:2507.06466*, 2025.
- Li Ding, Jenny Zhang, Jeff Clune, Lee Spector, and Joel Lehman. Quality Diversity through Human Feedback: Towards Open-Ended Diversity-Driven Optimization. In *Forty-first International Conference on Machine Learning*, 2024.
- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *arXiv preprint arXiv:1901.10995*, 2019.
- Adrien Ecoffet, Jeff Clune, and Joel Lehman. Open questions in creating safe open-ended AI: Tensions between control and creativity. In *Artificial Life Conference Proceedings 32*, pages 27–35, 2020.
- Maxence Faldor, Jenny Zhang, Antoine Cully, and Jeff Clune. OMNI-EPIC: Open-endedness via Models of human Notions of Interestingness with Environments Programmed in Code. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Zhiyuan Fan, Weinong Wang, Debing Zhang, et al. Sedareval: Automated evaluation using self-adaptive rubrics. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 16916–16930, 2024.
- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- Paul Gauthier. o1 tops aider’s new polyglot leaderboard. <https://aider.chat/2024/12/21/polyglot.html>, December 2024. Accessed: 2026-01-28.

- Authors Genesis. Genesis: A generative and universal physics engine for robotics and beyond, December 2024. URL <https://github.com/Genesis-Embodied-AI/Genesis>.
- Irving John Good. Speculations concerning the first ultraintelligent machine. In *Advances in computers*, volume 6, pages 31–88. Elsevier, 1966.
- Luca Grillotti, Lisa Coiffard, Oscar Pang, Maxence Faldor, and Antoine Cully. From Tabula Rasa to Emergent Abilities: Discovering Robot Skills via Real-World Unsupervised Quality-Diversity. *arXiv preprint arXiv:2508.19172*, 2025.
- Alex Havrilla, Yuqing Du, Sharath Chandra Raparthy, Christoforos Nalmpantis, Jane Dwivedi-Yu, Maksym Zhuravinskyi, Eric Hambro, Sainbayar Sukhbaatar, and Roberta Raileanu. Teaching large language models to reason with reinforcement learning. *arXiv preprint arXiv:2403.04642*, 2024.
- Nathan Herr, Tim Rocktäschel, and Roberta Raileanu. LLM-First Search: Self-Guided Exploration of the Solution Space. *arXiv preprint arXiv:2506.05213*, 2025.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Edward Hughes, Michael Dennis, Jack Parker-Holder, Feryal Behbahani, Aditi Mavalankar, Yuge Shi, Tom Schaul, and Tim Rocktaschel. Open-endedness is essential for artificial superhuman intelligence. *arXiv preprint arXiv:2406.04268*, 2024.
- Marcus Hutter. A gentle introduction to the universal algorithmic agent AIXI. *Artificial General Intelligence*, 2003.
- Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. A modern self-referential weight matrix that learns to modify itself. In *International Conference on Machine Learning*, pages 9660–9677. PMLR, 2022.
- Matthew Thomas Jackson, Chris Lu, Louis Kirsch, Robert Tjarko Lange, Shimon Whiteson, and Jakob Nicolaus Foerster. Discovering temporally-aware reinforcement learning algorithms. *arXiv preprint arXiv:2402.05828*, 2024.
- Khurram Javed and Martha White. Meta-learning representations for continual learning. *Advances in neural information processing systems*, 32, 2019.
- Minqi Jiang, Tim Rocktäschel, and Edward Grefenstette. General intelligence requires rethinking exploration. *Royal Society Open Science*, 10(6):230539, 2023.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- Louis Kirsch and Jürgen Schmidhuber. Eliminating meta optimization through self-referential meta learning. *arXiv preprint arXiv:2212.14392*, 2022.
- Martin Klissarov, Pierluca D’Oro, Shagun Sodhani, Roberta Raileanu, Pierre-Luc Bacon, Pascal Vincent, Amy Zhang, and Mikael Henaff. Motif: Intrinsic motivation from artificial intelligence feedback. *arXiv preprint arXiv:2310.00166*, 2023.
- Martin Klissarov, Mikael Henaff, Roberta Raileanu, Shagun Sodhani, Pascal Vincent, Amy Zhang, Pierre-Luc Bacon, Doina Precup, Marlos C Machado, and Pierluca D’Oro. MaestroMotif: Skill Design from Artificial Intelligence Feedback. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. Measuring ai ability to complete long tasks. *arXiv preprint arXiv:2503.14499*, 2025.
- Robert Lange, Tom Schaul, Yutian Chen, Tom Zahavy, Valentin Dalibard, Chris Lu, Satinder Singh, and Sebastian Flennerhag. Discovering evolution strategies via meta-black-box optimization. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pages 29–30, 2023.
- Joel Lehman and Kenneth O Stanley. Evolving a diversity of virtual creatures through novelty search and local competition. In *Proceedings of the 13th annual conference on Genetic and evolutionary computation*, pages 211–218, 2011.
- Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O Stanley. Evolution through large models. In *Handbook of evolutionary machine learning*, pages 331–366. Springer, 2023.
- Hao Li, Xue Yang, Zhaokai Wang, Xizhou Zhu, Jie Zhou, Yu Qiao, Xiaogang Wang, Hongsheng Li, Lewei Lu, and Jifeng Dai. Auto mc-reward: Automated dense reward design with large language models for minecraft. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16426–16435, 2024.

- Chris Lu, Sebastian Towers, and Jakob Foerster. Arbitrary order meta-learning with simple population-based evolution. In *Artificial Life Conference Proceedings 35*, volume 2023, page 67, 2023.
- Chris Lu, Samuel Holt, Claudio Fanconi, Alex Chan, Jakob Foerster, Mihaela van der Schaar, and Robert Lange. Discovering preference optimization algorithms with and for large language models. *Advances in Neural Information Processing Systems*, 37:86528–86573, 2024a.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024b.
- Minh-Thang Luong, Dawsen Hwang, Hoang H Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, et al. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 35406–35430, 2025.
- Changze Lv, Jie Zhou, Wentao Zhao, Jingwen Xu, Zisu Huang, Muzhao Tian, Shihan Dou, Tao Gui, Le Tian, Xiao Zhou, Xiaoqing Zheng, Xuanjing Huang, and Jie Zhou. Learning Query-Specific Rubrics from Human Preferences for DeepResearch Report Generation. *arXiv preprint arXiv:2602.03619*, 2026.
- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-Level Reward Design via Coding Large Language Models. In *The Twelfth International Conference on Learning Representations*, 2024.
- Thomas Miconi, Kenneth Stanley, and Jeff Clune. Differentiable plasticity: training plastic neural networks with backpropagation. In *International Conference on Machine Learning*, pages 3559–3568. PMLR, 2018.
- Thomas Miconi, Aditya Rawal, Jeff Clune, and Kenneth O Stanley. Backpropamine: training self-modifying neural networks with differentiable neuromodulated plasticity. *arXiv preprint arXiv:2002.10585*, 2020.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Junhyuk Oh, Greg Farquhar, Iurii Kemaev, Dan A Calian, Matteo Hessel, Luisa Zintgraf, Satinder Singh, Hado Van Hasselt, and David Silver. Discovering state-of-the-art reinforcement learning algorithms. *Nature*, pages 1–2, 2025.
- Julien Pourcel, Cédric Colas, Gaia Molinaro, Pierre-Yves Oudeyer, and Laetitia Teodorescu. ACES: Generating Diverse Programming Puzzles with with Autotelic Generative Models. *arXiv preprint arXiv:2310.10692*, 2023.
- Xin Qiu, Yulu Gan, Conor F Hayes, Qiyao Liang, Elliot Meyerson, Babak Hodjat, and Risto Miikkulainen. Evolution strategies at scale: Llm fine-tuning beyond reinforcement learning. *arXiv preprint arXiv:2509.24372*, 2025.
- Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Mikayel Samvelyan, Sharath C Raparthy, Andrei Lupu, Eric Hambro, Aram H Markosyan, Manish Bhatt, Yuning Mao, Minqi Jiang, Jack Parker-Holder, Jakob Foerster, et al. Rainbow teaming: Open-ended generation of diverse adversarial prompts. *Advances in Neural Information Processing Systems*, 37:69747–69786, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- Jürgen Schmidhuber. A neural network that embeds its own meta-levels. In *IEEE International Conference on Neural Networks*, pages 407–412. IEEE, 1993.
- Jürgen Schmidhuber. Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs/0309048*, 2003.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- Ivaxi Sheth, Jan Wehner, Sahar Abdelnabi, Ruta Binkyte, and Mario Fritz. Safety is Essential for Responsible Open-Ended Systems. *arXiv preprint arXiv:2502.04512*, 2025.
- David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- Kenneth O Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary computation*, 10(2):99–127, 2002.
- Kenneth O Stanley, Joel Lehman, and Lisa Soros. Open-endedness: The last grand challenge you’ve never heard of. *While open-endedness could be a force for discovering intelligence, it could also be a component of AI itself*, 2017.
- Marilyn Strathern. ‘Improving ratings’: audit in the British University system. *European review*, 5(3):305–321, 1997.
- Alan Mathison Turing et al. On computable numbers, with an application to the Entscheidungsproblem. *J. of Math*, 58(345-363):5, 1936.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*, 2024.
- Jianyu Wang, Zhiqiang Hu, and Lidong Bing. Evolving Prompts In-Context: An Open-ended, Self-replicating Perspective. *arXiv preprint arXiv:2506.17930*, 2025a.
- Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-G\ " odel Machine: Human-Level Coding Agent Development by an Approximation of the Optimal Self-Improving Machine. *arXiv preprint arXiv:2510.21614*, 2025b.
- Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H Chi, et al. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory. *arXiv preprint arXiv:2511.20857*, 2025a.
- Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. Toward Training Superintelligent Software Agents through Self-Play SWE-RL. *arXiv preprint arXiv:2512.18552*, 2025b.
- Jiaxin Wen, Zachary Ankner, Arushi Somani, Peter Hase, Samuel Marks, Jacob Goldman-Wetzler, Linda Petrini, Henry Sleight, Collin Burns, He He, et al. Unsupervised Elicitation of Language Models. *arXiv preprint arXiv:2506.10139*, 2025.
- Zhaotian Weng, Antonis Antoniadis, Deepak Nathani, Zhen Zhang, Xiao Pu, and Xin Eric Wang. Group-Evolving Agents: Open-Ended Self-Improvement via Experience Sharing. *arXiv preprint arXiv:2602.04837*, 2026.
- Jason Weston and Jakob Foerster. Ai & human co-improvement for safer co-superintelligence. *arXiv preprint arXiv:2512.05356*, 2025.
- Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024.
- Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? *arXiv preprint arXiv:2511.13646*, 2025a.
- Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint arXiv:2511.16043*, 2025b.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv preprint arXiv:2602.08234*, 2026.
- Yiming Xiong, Shengran Hu, and Jeff Clune. Learning to Continually Learn via Meta-learning Agentic Memory Designs. *arXiv preprint arXiv:2602.07755*, 2026.

- Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.
- Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta Context Engineering via Agentic Skill Evolution. *arXiv preprint arXiv:2601.21557*, 2026.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, 2025.
- Jiayi Yuan, Jonathan Nöther, Natasha Jaques, and Goran Radanović. AgenticRed: Optimizing Agentic Systems for Automated Red-teaming. *arXiv preprint arXiv:2601.13518*, 2026.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. In *First Conference on Language Modeling*, 2024.
- Alex L Zhang, Tim Kraska, and Omar Khattab. Recursive Language Models. *arXiv preprint arXiv:2512.24601*, 2025a.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents. *arXiv preprint arXiv:2602.02474*, 2026.
- Jenny Zhang, Joel Lehman, Kenneth Stanley, and Jeff Clune. OMNI: Open-endedness via Models of human Notions of Interestingness. In *The Twelfth International Conference on Learning Representations*, 2024.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents. *arXiv preprint arXiv:2505.22954*, 2025b.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025c.
- Bingchen Zhao, Despoina Magka, Minqi Jiang, Xian Li, Roberta Raileanu, Tatiana Shavrina, Jean-Christophe Gagnon-Audet, Kelvin Niu, Shagun Sodhani, Michael Shvartsman, et al. The Automated LLM Speedrunning Benchmark: Reproducing NanoGPT Improvements. *arXiv preprint arXiv:2506.22419*, 2025.
- Bingchen Zhao, Jenny Zhang, Chenxi Whitehouse, Minqi Jiang, Michael Shvartsman, Abhishek Charnalia, Despoina Magka, Tatiana Shavrina, Derek Dunfield, Oisín Mac Aodha, and Yoram Bachrach. APRES: An Agentic Paper Revision and Evaluation System. *arXiv preprint arXiv:2603.03142*, 2026.
- Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.
- Barret Zoph and Quoc Le. Neural Architecture Search with Reinforcement Learning. In *International Conference on Learning Representations*, 2017.
- Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-Adapting Language Models. *arXiv preprint arXiv:2506.10943*, 2025.

附录

目录

A 算法细节	21	A.1 初始智能体	21	A.2 父代选择	22
		A.3 伪代码	23		
		A.4 多领域优化	24		
B 基线细节	24				
C 领域详情	28				
		C.1 多语言者	28		
		C.2 论文评审	30		
		C.3 机器人奖励设计	32		
		C.4 奥林匹克级别数学评分	33		
D 实验详情	35				
		D.1 基础模型的超参数	35		
		D.2 成本估算	35		
		D.3 改进@k 度量	36		
		D.4 迁移代理选择	36		
E 附加结果	37	E.1 最佳发现任务代理	37	E.2 定性分析：提升任务性能	48
		E.3 定性分析：提升改进能力	50	E.4 奥林匹克级别数学评分器	56
		E.5 修改父代选择	57		
F 附加安全讨论	60				

A 算法细节

本附录提供了 DGM-超代理 (DGM-H) 的额外算法细节。首先描述初始超代理的实现, 包括初始任务代理和元代理可用的工具与提示 (附录 A.1)。随后详述开放探索中使用的父代选择机制, 该机制在高性能代理的利用与档案的持续探索之间取得平衡 (附录 A.2)。最后给出 DGM-H 的伪代码 (附录 A.3)。

A.1 初始代理

我们介绍初始超代理可用的工具及其提示的细节 (第 4 节)。初始任务代理提示:

```
instruction = f"""You are an agent.
Task input:
'''
{inputs}
'''
Respond in JSON format with the following schema:
<json>
{{
  "响应": ... }}
</json>"""
```

初始元代理提示:

```
指令 = f"修改代码库中位于 '{repo_path}' 的任意部分"
```

给定 bash 工具的信息:

```
定义 tool_info(): 返回
{
    "name": "bash",
    "description": """Run commands in a bash shell
* When invoking this tool, the contents of the "command" parameter does NOT need to be XML-escaped.
* You don't have access to the internet via this tool.
* You do have access to a mirror of common linux and python packages via apt and pip.
* State is persistent across command calls and discussions with the user.
* To inspect a particular line range of a file, e.g. lines 10-25, try 'sed -n 10,25p /path/to/the/file'.
* Please avoid commands that may produce a very large amount of output.
* Please run long lived commands in the background, e.g. 'sleep 10 &' or start a server in the background."""
    "input_schema": {
        "type": "object",
        "properties": {
            "命令": {
                "type": "string",
                "description": "The bash command to run."
            }
        },
        "required": ["command"]
    }
}
```

给定编辑工具的信息:

```
定义 tool_info(): 返回
{
    "name": "editor",
    "description": """Custom editing tool for viewing, creating and editing files
* State is persistent across command calls and discussions with the user
* If 'path' is a file, 'view' displays the result of applying 'cat -n'. If 'path' is a directory, 'view' lists non-hidden
  ↳ 最多两层的文件和目录
* 如果指定的'path'已作为文件存在, 则无法使用'create'命令 * 如果'command'生成较长的输出, 该输出将被截断并标记为'<response clipped>'
'undo_edit'命令将撤销对'path'处文件所做的最后一次编辑 \nNotes 使用'str_replace'命令的说明: * 'old_str'参数应与原始文件中的一行或多行连续内容
完全匹配。请注意

  ↳ 空白字符!
  ↳ 'old_str'中有足够的上下文使其唯一
* 如果'old_str'参数在文件中不唯一, 则不会执行替换。请确保包含

* 'new_str'参数应包含用于替换'old_str'的已编辑行 {
```

```

"type": "object",
"properties": {
  "命令": {
    "type": "string",
    "enum": ["view", "create", "str_replace", "insert", "undo_edit"],
    "description": "The commands to run. Allowed options are: 'view', 'create', 'str_replace', 'insert',
      ↪ 'undo_edit'."
  },
  "file_text": {
    "description": "Required parameter of 'create' command, with the content of the file to be created.",
    "type": "string"
  },
  "insert_line": {
    "description": "'insert'命令的必需参数。'new_str'将被插入到该行之后
      ↪ 'insert_line' of 'path'."
    "type": "integer"
  },
  "new_str": {
    "description": "'str_replace'命令的必需参数，包含新字符串。必需
      ↪ parameter of 'insert' command containing the string to insert."
    "type": "string"
  },
  "old_str": {
    "description": "'str_replace'命令的必需参数，包含'path'中要
      ↪ replace."
    "type": "string"
  },
  "path": {
    "description": "Absolute path to file or directory, e.g. '/repo/file.py' or '/repo'."
    "type": "string"
  },
  "view_range": {
    "描述": "当'路径'指向文件时，'查看'命令的可选参数。若未提供，
      ↪ the full file is shown. If provided, the file will be shown in the indicated line number range,
      ↪ e.g. [11, 12] will show lines 11 and 12. Indexing at 1 to start. Setting '[start_line, -1]'
      ↪ shows all lines from 'start_line' to the end of the file."
    "items": {
      "type": "integer"
    },
    "type": "array"
  }
},
"required": ["command", "path"]
}

```

A.2 父代选择

在每次迭代中，我们从存档中选取一部分智能体作为父代，使其自我修改并生成新的子代智能体（第3节）。我们采用与 Zhang 等人（2025b）类似的机制，该机制受 Ecoffet 等人（2019）启发，大致与智能体的性能得分成正比，与成功编译的子代数量成反比。这种选择机制使采样偏向于表现优于当前前沿平均水平的智能体，同时降低已产生大量子代的智能体的权重，从而保持平滑的概率探索，并随着存档的不断改进自动适应。父代选择过程的细节如下。在 DGM-H 运行的每次迭代 t 中，令

$$\mathcal{A}^t = \{a_0, a_1, \dots, a_t\}$$

表示候选智能体存档及其关联的性能得分 $\alpha_i = \text{性能}(a_i)$ 。存档中的所有智能体均有资格参与父代选择。我们首先根据当前性能分布计算一个动态中点。令

$$\alpha_{mid} = \frac{1}{m} \sum_{j \in \mathcal{T}^t} \alpha_j,$$

其中 $\mathcal{T}^t \subset \mathcal{A}^t$ 索引迭代 t 中性能最高的前 m 个智能体（在我们的实验中 $m = 3$ ）。该中点随时间自适应，反映当前性能前沿。

每个智能体的得分首先经过 Sigmoid 函数变换：

$$s_i = \frac{1}{1 + \exp(-\lambda(\alpha_i - \alpha_{mid}))},$$

其中 $\lambda > 0$ 控制选择的锐度。我们设置 $\lambda = 10$ 。为鼓励探索，我们基于智能体 a_i 产生的已编译子代数量 n_i 引入新颖性奖励：

$$h_i = \frac{1}{1 + n_i}.$$

然后我们构造未归一化权重

$$w_i = s_i h_i,$$

该机制在性能与新颖性之间取得平衡。权重经归一化后形成类别分布：

$$p_i = \begin{cases} \frac{w_i}{\sum_{j=0}^t w_j}, & \text{if } \sum_{j=0}^t w_j > 0, \\ \frac{1}{t+1}, & \text{otherwise.} \end{cases}$$

本文依据该分布进行有放回的独立父代采样：

$$\{\text{parents}\} \sim \text{类别分布 } (\{p_i\}_{i=0}^t).$$

先前工作已提出广泛的搜索与探索策略（Coulom, 2006; Silver et al., 2016; Herr et al., 2025; Wang et al., 2025b; Weng et al., 2026）。本文提供初步证据表明，DGM-H 能够通过修改自身的探索动态，开始自主地重新发现并适应此类策略（附录 E.5）。一个开放的研究方向是：自改进系统能否可靠地发现超越精心手工设计算法的搜索与探索机制。

A.3 伪代码

这是第 3 节所述 DGM-H 的伪代码：

Algorithm 1: Darwin Gödel Machine with Hyperagents (DGM-H)

Input: Initial agent a^0 , task set $\mathcal{T}$, maximum iterations T

Output: Archive of scored agents $\mathcal{A}$

```

 $s^0 \leftarrow \text{EVALUATE}(a^0, \mathcal{T})$ 
initialize  $\mathcal{A} \leftarrow \{(a^0, s^0)\}$                                 // Start with initial agent
for  $t \leftarrow 1$  to  $T$  do
     $\mathcal{P} \leftarrow \text{SELECTPARENTS}(\mathcal{A})$                                 // Sample parent agents
    foreach  $(a, \cdot) \in \mathcal{P}$  do
         $a' \leftarrow a.\text{MODIFY}(a, \mathcal{A})$                                 // Metacognitive self-modification
         $s' \leftarrow \text{EVALUATE}(a', \mathcal{T})$                                 // Evaluate on tasks
        if  $\text{ISVALID}(a')$  then
             $\mathcal{A} \leftarrow \mathcal{A} \cup \{(a', s')\}$                         // Add compiled child agent
        end
    end
end
return  $\mathcal{A}$ 

```

A.4 多领域优化

在同一次运行中针对多个领域进行优化时，超代理在来自不同领域的任务上接受评估，并在自我修改期间能够访问这些任务的所有评估结果。本文不指定优先考虑哪个特定领域或任务。父代选择基于跨领域的平均性能。因此，任何领域的改进都会提高选择概率，而性能退化则会降低选择概率。由于元代理可以检查任何任务的评估结果，它能够引入同时惠及多个领域的共享机制（例如，结构化推理、记忆和错误处理）。因此，超代理无需手动指定要优化的任务或领域，即可在同一次运行中跨多个领域进行优化。

B 基线细节

本文概述第 4.1 节中描述的每个基线的伪代码，提供总结其关键差异的比较表（表 1），并包含一个详细的概念图，直观对比不同深度生成模型（DGM）变体和超智能体之间的架构组件与修改机制（图 5）。

Method	Self-improving meta agents	Open-ended exploration	Metacognitive self-modification (i.e., hyperagents)
DGM-H	✓	✓	✓
DGM-H w/o self-improve	✗	✓	✓
DGM-H w/o open-ended exploration	✓	✗	✓
DGM	✓	✓	✗
DGM-custom	✓	✓	✗

表 1 按自我改进、开放式探索和元认知自我修改对各方法进行比较。

这是基线深度生成模型-超智能体（DGM-H）在不包含自我改进智能体（ADAS, Hu 等人，2025 年）情况下的伪代码：

Algorithm 2: DGM-H without self-improving meta agents (DGM-H w/o self-improve)

Input: Initial agent a^0 , task set $\mathcal{T}$, maximum iterations T

Output: Archive of scored agents $\mathcal{A}$

```

 $s^0 \leftarrow \text{EVALUATE}(a^0, \mathcal{T})$ 
initialize  $\mathcal{A} \leftarrow \{(a^0, s^0)\}$ 
for  $t \leftarrow 1$  to  $T$  do
     $\mathcal{P} \leftarrow \text{SELECTPARENTS}(\mathcal{A})$ 
    foreach  $(a, \cdot) \in \mathcal{P}$  do
         $a' \leftarrow a^0.\text{MODIFY}(a, \mathcal{A})$                                 // Modify with initial agent
         $s' \leftarrow \text{EVALUATE}(a', \mathcal{T})$ 
        if  $\text{ISVALID}(a')$  then
             $\mathcal{A} \leftarrow \mathcal{A} \cup \{(a', s')\}$ 
        end
    end
end
return  $\mathcal{A}$ 

```

这是基线深度生成模型-超智能体（DGM-H）在不包含开放式探索情况下的伪代码：

Algorithm 3: DGM-H without open-ended exploration (DGM-H w/o open-ended exploration)

Input: Initial agent a^0 , task set $\mathcal{T}$, maximum iterations T

Output: Archive of scored agents $\mathcal{A}$

```
 $s^0 \leftarrow \text{EVALUATE}(a^0, \mathcal{T})$ 
initialize  $\mathcal{A} \leftarrow \{(a^0, s^0)\}$ 
for  $t \leftarrow 1$  to  $T$  do
   $\mathcal{P} \leftarrow \text{SELECTPARENTS}(\mathcal{A})$ 
  foreach  $(a, \cdot) \in \mathcal{P}$  do
     $a' \leftarrow a.\text{MODIFY}(a, \mathcal{A})$ 
     $s' \leftarrow \text{EVALUATE}(a', \mathcal{T})$ 
    if  $\text{ISVALID}(a')$  then
       $\mathcal{A} \leftarrow \{(a', s')\}$  // Only keep the latest agent
    end
  end
end
return  $\mathcal{A}$ 
```

这是原始深度生成模型（DGM）（Zhang 等人，2025b）在超智能体设定下的伪代码：

Algorithm 4: Darwin Gödel Machine (DGM)

Input: Initial agent a^0 , task set $\mathcal{T}$, maximum iterations T

Output: Archive of scored agents $\mathcal{A}$

```
 $s^0 \leftarrow \text{EVALUATE}(a^0, \mathcal{T})$ 
initialize  $\mathcal{A} \leftarrow \{(a^0, s^0)\}$ 
for  $t \leftarrow 1$  to  $T$  do
   $\mathcal{P} \leftarrow \text{SELECTPARENTS}(\mathcal{A})$ 
  foreach  $(a, \cdot) \in \mathcal{P}$  do
     $\text{instr} \leftarrow \text{INSTRGEN}(a)$  // Handcrafted instruction-generation
     $a' \leftarrow a.\text{MODIFY}(a, \text{instr})$  // Self-modification
     $s' \leftarrow \text{EVALUATE}(a', \mathcal{T})$ 
    if  $\text{ISVALID}(a')$  then
       $\mathcal{A} \leftarrow \mathcal{A} \cup \{(a', s')\}$ 
    end
  end
end
return  $\mathcal{A}$ 
```

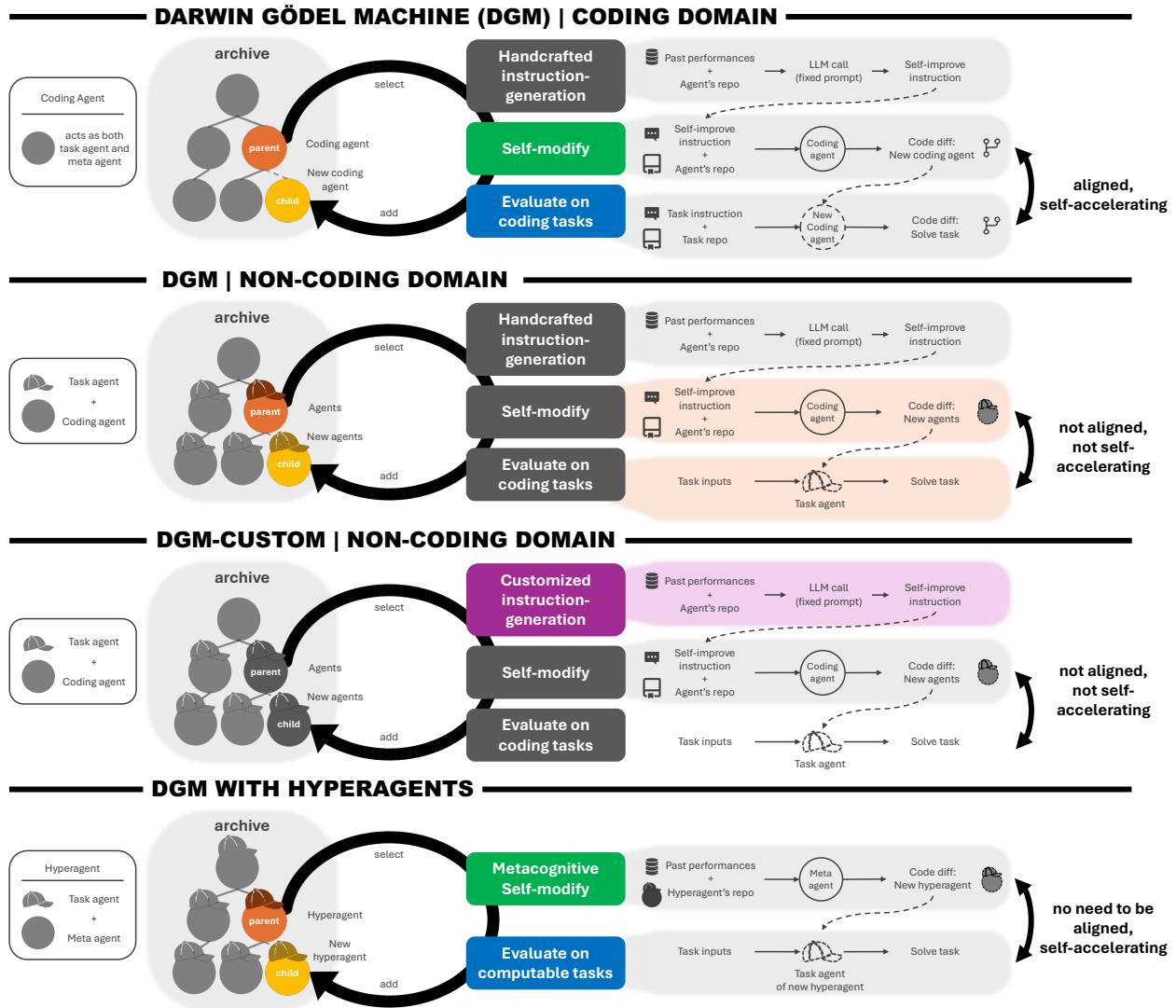


图 5 深度生成模型（DGM）变体的概念比较，突出不同设定下哪些组件发生变化，以及超智能体如何解决原始深度生成模型实现中的局限性。（第一行）原始深度生成模型。同一个编码智能体同时充当任务智能体和元智能体。由于评估和自我修改都是编码任务，编码能力的提升会转化为自我修改能力的提升，从而使深度生成模型在编码领域能够改进其改进能力。（第二行）适应非编码领域的深度生成模型。编码智能体仍作为元智能体，但评估任务不再是编码任务，因此需要单独的任务智能体。任务性能不再可靠地反映元智能体生成更优任务智能体的能力，破坏了使元智能体能够改进其改进能力的对齐关系。（第三行）深度生成模型定制基线。手工设计的指令生成机制针对目标领域进行了定制，但仍不可修改。（第四行）带超智能体的深度生成模型（DGM-H）。超智能体将任务智能体和元智能体集成在单个可编辑程序中，实现元认知自我修改（即不仅修改任务求解行为，还修改生成未来自我修改的过程）。因此，深度生成模型-超智能体可以在优化任何可计算任务的同时改进其改进机制。

原始深度生成模型（DGM）中手工设计的指令生成步骤：

```
diagnose_prompt = """以下是编码智能体的实现。

# 编码智能体实现 ----- 编码智能体实现开始 ----- {code} -----
编码智能体实现结束 -----
```

你的任务是确定一个能提升该智能体编码能力的详细计划。该改进不应  特定于任何特定的 GitHub 问题或仓库。

智能体运行日志 ----- 智能体运行日志开始

{md_log} -----

----- 智能体运行日志结束 -----

GitHub 问题 智能体正在尝试解决的 GitHub 问题。----- GitHub

问题开始 ----- {github_issue} ----- GitHub 问题结束 -----

预测补丁 智能体为解决该问题所预测的补丁。----- 预测补

丁开始 ----- {predicted_patch} ----- 预测补丁结束 -----

私有测试补丁 SWE-bench 用于检测问题是否被解决的官方私有测试。智能体在运行期间无法获取此信息

 评估。智能体应尝试实现自身的测试。

----- 私有测试补丁开始 ----- {test_patch} ----- 私

有测试补丁结束 -----

问题测试结果 使用上述官方私有测试得到的 SWE-bench 测试结果。----- 问题测试结果开始

----- {eval_log} ----- 问题测试结果结束 -----

请严格按照以下格式回复，包括 JSON 开始和结束标记：

“json
<JSON>
“

In <JSON>, provide a JSON response with the following fields:

- "log_summarization": Analyze the above logs and summarize how the agent tried to solve the GitHub issue. Note which  工具及其使用方式、智能体的问题解决方法以及遇到的任何问题。

- "potential_improvements": 识别编码智能体可提升其编码能力的潜在改进点

 capabilities. Focus on the agent's general coding abilities (e.g., better or new tools usable across any

 repository) rather than issue-specific fixes (e.g., tools only usable in one framework). All necessary

 dependencies and environment setup have already been handled, so do not focus on these aspects.

- "improvement_proposal": Choose ONE high-impact improvement from the identified potential improvements and describe it

 详细说明。这应是一个聚焦且全面的计划，以增强智能体的整体编码能力。

- "implementation_suggestion": 参考编码代理的总结与实现，批判性地思考可以添加或改进哪些  功能或工具，以最佳地实现所提议的改进。如果所提议的功

能  可以通过修改现有工具来实现，请描述所需的修改，而不是建议  新工具。- "problem_description": 将改进提议和实现建议表述为 GitHub 问题

描述。它  应清晰地描述该功能，以便查看问题和仓库的软件工程师能够  实现它。

您的响应将被自动解析，因此请确保字符串响应严格符合正确格式。请勿  在输出中包含 '<JSON>' 标签。

DGM-custom 中的定制指令生成步骤：

diagnose_prompt_customized = "" 以下是编码代理和任务代理的实现。

编码代理实现 ----- 编码代理实现开始 -----

{code_codingagent} ----- 编码代理实现结束 -----

任务代理实现 ----- 任务代理实现开始 -----

{code_taskagent} ----- 任务代理实现结束 -----

您的任务是确定一个详细的计划，以改进编码/任务代理。该改进不应  特定于任何特定的任务实例或仓库。

任务信息 ----- 任务 -----

{task_info} ----- 任务结束

```
# 报告 ---- 报告 ---- {report}
---- 报告结束 ----

# 智能体运行日志 ---- 智能体运行日志开始
{md_log} ----
---- 智能体运行日志结束 ----

请严格按照以下格式回复，包括 JSON 开始和结束标记：

<json>
...
</json>

In <json>, provide a JSON response with the following fields:
- "log_summarization": Analyze the above logs and summarize how the agent tried to solve the given task. Note which tools
  ↳ 及其使用方式、智能体的问题解决方法，以及遇到的任何问题。
- "potential_improvements": 识别对编码/任务智能体的潜在改进，以增强其
  ↳ coding/task-solving capabilities. Focus on the agent's general abilities (e.g., better or new tools usable across
  ↳ any repository) rather than issue-specific fixes (e.g., tools only usable in one framework). All necessary
  ↳ dependencies and environment setup have already been handled, so do not focus on these aspects.
- "improvement_proposal": Choose ONE high-impact improvement from the identified potential improvements and describe it
  ↳ 详细说明。这应是一个聚焦且全面的计划，以增强智能体的整体编码/任务解决 ↳ 能力。
- "implementation_suggestion": 参考编码/任务智能体的总结与实现，批判性地思考 ↳ 可以添加或改进哪些功能或工具，以最佳实现所提议的改进。如果所提议的
  ↳ 功能可以通过修改现有工具来实现，请描述所需的修改，而不是 ↳ 建议新工具。- "problem_description": 将改进提案和实现建议表述为 GitHub 问题描述。它 ↳ 应清晰描述该功能，以便查看问题和仓库的软件工程师能够 ↳ 实现它。

您的响应将被自动解析，因此请确保字符串响应严格符合正确格式。"""
```

C 领域详情

本附录详细描述了用于评估的各个领域：多语言编程（Polyglot）（附录 C.1）、论文评审（附录 C.2）、机器人奖励设计（附录 C.3）、奥林匹克级别数学评分（附录 C.4）。对于每个领域，我们指定了给定任务中智能体的输入和所需输出、评估协议以及代表性的静态基线（表 2）。

Domain	Input	Output	Metric	Train	Validation	Test
Coding (Polyglot)	Repo + instr.	Code patch	Pass@1	60	-	165
Paper Review	Paper text	Accept / Reject	Accuracy	100	100	100
Robotics Reward Design	Task desc.	Reward fn.	Task score	6	-	6
IMO Grading	Problem + sol.	Grade (0/1/6/7)	Accuracy	100	100	100

表 2 领域总结。在自我修改过程中，智能体在训练任务上的评估结果可用作反馈。验证任务用于父代选择。如果验证集不可用，则用于父代选择的性能组件基于训练性能。测试任务是留出的，仅用于所选智能体的最终评估。

C.1 多语言编程

在多语言编程基准（Polyglot coding benchmark）（Gauthier, 2024）中，每个任务由一个软件仓库和一条描述对代码库所需更改的自然语言指令组成。智能体可以访问完整的仓库，并且必须修改文件以正确实现指令，生成应用于仓库的补丁（即一组代码编辑）。性能通过在修改后的仓库上运行预定义的测试套件来评估。如果所有测试通过，则认为任务成功完成。我们遵循 DGM（Zhang et al., 2025b）中使用的设置，该设置在很大程度上与多语言编程排行榜配置一致，但有一个关键区别：排行榜报告 pass@2，允许智能体查看一次真实测试的反馈，而我们报告 pass@1，智能体从未看到真实测试结果。我们采用与 DGM 相同的

训练集和测试集划分。训练任务从完整基准中随机选取子集，共包含 60 个任务。如果智能体在初始 10 个任务子集上的成功率超过 40%，则随后在剩余的 50 个训练任务上进行评估。该领域没有验证子集。为了更准确地评估性能改进，作为最终评估，我们在完整的多语言编程基准上评估生成的智能体，该基准包含 165 个未见过的任务。用于初步评估的初始 10 个训练任务：

- go__dominoes
- cpp__all-your-base
- python__dominoes
- java__sgf-parsing
- javascript__robot-name
- rust__variable-length-quantity
- python__beer-song
- go__book-store
- javascript__bottle-song
- rust__bowling

用于完整评估的额外 50 个训练任务：

- javascript__queen-attack
- rust__wordy
- python__dot-dsl
- java__satellite
- cpp__diamond
- rust__accumulate
- go__error-handling
- cpp__queen-attack
- rust__poker
- python__sgf-parsing
- rust__react
- java__ledger
- go__connect
- rust__macros
- javascript__triangle
- java__zipper
- java__bowling
- python__tree-building
- javascript__say
- java__wordy
- python__food-chain
- javascript__wordy
- python__poker
- javascript__grade-school
- cpp__gigasecond
- java__forth
- python__dominoes
- go__word-search
- javascript__simple-linked-list
- go__counter
- java__react
- javascript__ocr-numbers
- python__scale-generator
- java__go-counting
- rust__doubly-linked-list
- python__grade-school
- javascript__forth
- python__wordy
- java__mazy-mice
- cpp__bank-account
- python__zipper
- java__custom-set
- java__rest-api
- go__transpose
- rust__gigasecond
- rust__say
- go__food-chain
- rust__pig-latin
- go__markdown
- go__crypto-square

C.2 论文评审

该领域的数据来源于赵等人（2026）。论文评审领域的每项任务包含一篇人工智能研究论文的全文。智能体必须预测二元接受或拒绝决定，模拟会议审稿人的角色。真实标签对应顶级机器学习会议（包括 ICLR 2024/2025 和 NeurIPS 2023/2024）的实际接受决定。性能通过相对于这些标签的分类准确率来衡量。我们随机采样任务以构建训练、验证和测试划分，每个划分包含 100 项任务。在训练期间，智能体首先在训练划分的 10 项任务子集上进行评估。如果智能体在这些任务中至少成功一项，则随后在全部 100 项训练任务上进行评估。AI-Scientist-v2（山田等人，2025）采用人工智能审稿人来自动改进生成的人工智能研究论文。我们采用该工作中提出的人工智能审稿人作为代表性静态基线：

```
reviewer_system_prompt_base = (“你是一名人工智能研究者，正在审阅一篇提交给著名机器学习会议的论文。”“在决定时要批判且谨慎。”)
```

```
reviewer_system_prompt_neg = (reviewer_system_prompt_base + “如果论文质量差或你不确定，给出差评并拒绝它。”) reviewer_system_prompt_pos = (reviewer_system_prompt_base + “如果论文质量好或你不确定，给出好评并接受它。”)
```

```
template_instructions = """
Respond in the following format:
```

```
思考: <思考>
```

```
评审 JSON:
```json <JSON>
```
```

在<思考>中，首先简要讨论你对评估的直觉和推理。详细说明你的高层次论点、必要选择以及评审的期望结果。不要在此处发表泛泛的评论，而要针对你当前的论文具体说明。将此视为评审的笔记阶段。

```
In <JSON>, provide the review in JSON format with the following fields in the order:
- "Summary": A summary of the paper content and its contributions.
- "Strengths": A list of strengths of the paper.
- "Weaknesses": A list of weaknesses of the paper.
- "Originality": A rating from 1 to 4 (low, medium, high, very high).
- "Quality": A rating from 1 to 4 (low, medium, high, very high).
- "Clarity": A rating from 1 to 4 (low, medium, high, very high).
- "Significance": A rating from 1 to 4 (low, medium, high, very high).
- "Questions": A set of clarifying questions to be answered by the paper authors.
- "Limitations": A set of limitations and potential negative societal impacts of the work.
- "Ethical Concerns": A boolean value indicating whether there are ethical concerns.
- "Soundness": A rating from 1 to 4 (poor, fair, good, excellent).
- "Presentation": A rating from 1 to 4 (poor, fair, good, excellent).
- "Contribution": A rating from 1 to 4 (poor, fair, good, excellent).
- "Overall": A rating from 1 to 10 (very strong reject to award quality).
- "Confidence": A rating from 1 to 5 (low, medium, high, very high, absolute).
- "Decision": A decision that has to be one of the following: Accept, Reject.
```

对于“决定”字段，不要使用弱接受、边界接受、边界拒绝或强拒绝。相反，仅
↪ 使用接受或拒绝。

此 JSON 将被自动解析，请确保格式精确。"""

```
neurips_form = ( """ ## 审稿表格 以下是对每篇论文审稿表格中问题的描述，以及在回答这些问题时需要考虑 ↪ 的一些指导原则。撰写审稿意见时，请记住，在做出决定后，被接受 ↪ 论文和选择公开的被拒论文的审稿意见和元审稿意见将被公开。
```

1. 总结：简要总结论文及其贡献。此处不是批评论文的地方；作者 ↪ 通常应同意一份写得很好的总结。

- 优点与不足：请对论文的优点和不足进行全面评估，涉及
 - on each of the following dimensions:
 - Originality: Are the tasks or methods new? Is the work a novel combination of well-known techniques? (This can be valuable!) Is it clear how this work differs from previous contributions? Is related work adequately cited
 - Quality: Is the submission technically sound? Are claims well supported (e.g., by theoretical analysis or experimental results)? Are the methods used appropriate? Is this a complete piece of work or work in progress? Are the authors careful and honest about evaluating both the strengths and weaknesses of their work
 - Clarity: Is the submission clearly written? Is it well organized? (If not, please make constructive suggestions for improving its clarity.) Does it adequately inform the reader? (Note that a superbly written paper provides enough information for an expert reader to reproduce its results.)
 - Significance: Are the results important? Are others (researchers or practitioners) likely to use the ideas or build on
 - 它们？该投稿是否以比先前工作更好的方式解决了一项困难任务？是否以可证明的方式推进了 → 最新技术水平？是否提供了独特的数据、关于现有数据的独特结论、或 → 独特的理论或实验方法？
2. 问题：请列出并仔细描述对作者的任何问题和建议。考虑那些 → 作者的回复可以改变您的看法、澄清困惑或指出局限性的地方。这对于与作者进行富有成效的 → 反驳和讨论阶段可能非常重要。
3. 局限性：作者是否充分说明了其工作的局限性和潜在的负面社会影响？
- 如果没有，请提出建设性的改进建议。
- 总体而言，作者应当因坦诚其工作的局限性以及 → 任何潜在的负面社会影响而受到奖励，而非惩罚。鼓励您思考是否有任何关键点 → 缺失，并将其作为反馈提供给作者。
4. 伦理问题：如果本文存在伦理问题，请标记该论文以进行伦理审查。关于何时适合进行伦理审查的 → 指导，请查阅神经信息处理系统大会（NeurIPS）伦理指南。
5. 可靠性：请根据以下量表为论文分配数值评分，以表明其可靠性。
- technical claims, experimental and research methodology and on whether the central claims of the paper are
 - adequately supported with evidence.
- 4: excellent
3: good
2: fair
1: poor
6. 表述：请根据以下量表为论文分配数值评分，以表明其表述质量。
- presentation. This should take into account the writing style and clarity, as well as contextualization relative
 - to prior work.
- 4: excellent
3: good
2: fair
1: poor
7. 贡献：请根据以下量表为论文分配数值评分，以表明其整体贡献质量。
- contribution this paper makes to the research area being studied. Are the questions being asked important? Does
 - the paper bring a significant originality of ideas and/or execution? Are the results valuable to share with the
 - broader NeurIPS community.
- 4: excellent
3: good
2: fair
1: poor
8. 总体：请为本次提交提供“总体评分”。选项：10：获奖水准：技术无懈可击的论文，对人工智能的一个或多个领域具有开创性影响，且具有卓越的
- 评估、可复现性和资源，且无未解决的伦理问题。
- 9：强烈接受：技术无懈可击的论文，对人工智能至少一个领域具有开创性影响，并对多个领域具有卓越的 → 影响，且评估、资源和可复现性无懈可
- 击，且无未解决的伦理问题。
- 8：强烈接收：技术扎实的论文，具有新颖的想法，对至少一个人工智能领域产生卓越影响，或 → 对多个人工智能领域产生高至卓越影响，具备出色的
- 评估、资源与可复现性，且不存在未解决的伦理问题。
- 7：接受：技术扎实的论文，对人工智能至少一个子领域具有高影响力，或对多个子领域具有中到高影响力 → 在多个 AI 领域，具备良好至优秀的评估、资源、可复现性，且不存在未解决的伦理问题。
- 资源、可复现性与伦理考量。
- 6：弱接受：技术扎实，影响力中等到较高。评估方面无重大疑虑，
- 5：边缘接受：技术扎实的论文，接受的理由超过拒绝的理由，例如，局限性
- 评估。请谨慎使用。
- 4：边缘性拒稿：技术扎实的论文，但拒稿理由（如评估有限）多于接受理由
- 接受，例如良好的评估。请谨慎使用。
- 3：拒绝：例如，一篇存在技术缺陷、评估薄弱、可复现性不足且不完整的论文
- 阐述了伦理考量。
- 2：强烈拒绝：例如，论文存在重大技术缺陷，和/或评估质量差，影响有限、质量低下
- 可复现性以及大多未得到解决的伦理考量。
- 1：强烈拒绝：例如，一篇结果微不足道或未处理伦理考量（Ethical Considerations）的论文
9. 置信度：请为本次提交的评估提供一个“置信度评分”，以表明您的确信程度
- are in your evaluation. Choices:
- 5: You are absolutely certain about your assessment. You are very familiar with the related work and checked the
- 数学/其他细节需仔细处理。
- 4：你对自己的评估有信心，但并非绝对确定。你出错的可能性不大，但并非不可能。

↔ 不理解提交的某些部分，或者您不熟悉某些相关工作。3: 您对自己的评估相当有信心。可能您没有理解提交的某些部分 ↔，或者您不熟悉某些相关工作。数学/其他细节未被仔细检查。2: 您愿意为自己的评估辩护，但很可能您没有理解 ↔ 提交的核心部分，或者您不熟悉某些相关工作。数学/其他细节未被仔细检查。

↔

1: 您的评估是一个有根据的猜测。提交不在您的领域内，或者提交难以 ↔ 理解。数学/其他细节未被仔细检查。""" + template_instructions)

类 任务智能体（智能体系统）：定义

前向（自身，输入）：

```
reviewer_system_prompt = reviewer_system_prompt_neg
review_instruction_form = neurips_form

base_prompt = review_instruction_form
base_prompt += f"""
Here is the paper you are asked to review:
'''
{inputs['paper_text']}
'''
"""
指令 = reviewer_system_prompt + base_prompt

self.log(f"Input: {repr(instruction)}")
response, new_msg_history, _ = get_response_from_llm(
    msg=instruction,
    model=self.model,
    msg_history=[],
)
self.log(f"Output: {repr(response)}")

# Extract the response
prediction = "None"
try:
    extracted_jsons = extract_jsons(new_msg_history[-1]['text'])
    prediction = extracted_jsons[-1]['Decision']
except Exception as e:
    self.log(f"Error extracting prediction: {e}")
    prediction = "None"

返回预测, new_msg_history
```

C.3 机器人奖励设计

机器人奖励设计领域中的每项任务均在创世纪模拟器（Genesis，2024）中，使用 Go2 四足机器人指定一个机器人控制目标。智能体获得任务的文本描述（例如，以目标速度向前行走），并输出一个 Python 奖励函数，随后该函数用于训练机器人的强化学习策略（即 PPO，Schulman 等人，2017）。性能通过执行训练好的强化学习策略并在模拟器中计算任务性能测度（例如，速度跟踪误差）来评估。分数在重复评估中取平均，以减少奖励生成或强化学习中的随机性所导致的方差。训练任务要求生成一个奖励函数，使机器人能够在跟踪目标线速度的同时向前行走。性能使用指令速度与实际行走速度之间的均方误差来衡量。在训练期间，每个智能体最初在同一任务上重复评估 3 次，每次评估生成一个奖励函数。如果至少有一个生成的奖励函数产生非零性能分数，则智能体再被评估 3 次。最终性能分数报告为 6 次评估的平均值。该领域未单独设置验证任务。为了评估同一智能体能否在不同的机器人任务中生成合适的奖励函数，我们将一个相对简单的训练任务与同一机器人上更具挑战性的测试任务配对。测试任务要求生成一个奖励函数，训练机器人最大化躯干高度。对向前行走有效的奖励函数不会诱导跳跃行为，而跳跃行为对于最大化躯干高度更为最优。此外，直接激励躯干高度（性能测度）通常会导致次优的站立不动行为。因此，要实现高性能，需要非短视的奖励设计，鼓励中间行为，例如在跳跃前降低躯干。测试任务的默认奖励函数直接奖励最大化躯干的性能测度

高度。这总是产生一种行为，即机器人尽可能站得最高（图 6）：

```
定义 compute_reward(env)-> 元组 (张量, 字典, 字典):
    """
    The robot maximizes its vertical position.
    """
    height = env.base_pos[:, 2]
    total_reward = height

    reward_components = {"height": height}
    reward_scales = {"height": 1.0}

    返回 total_reward, reward_components, reward_scales
```

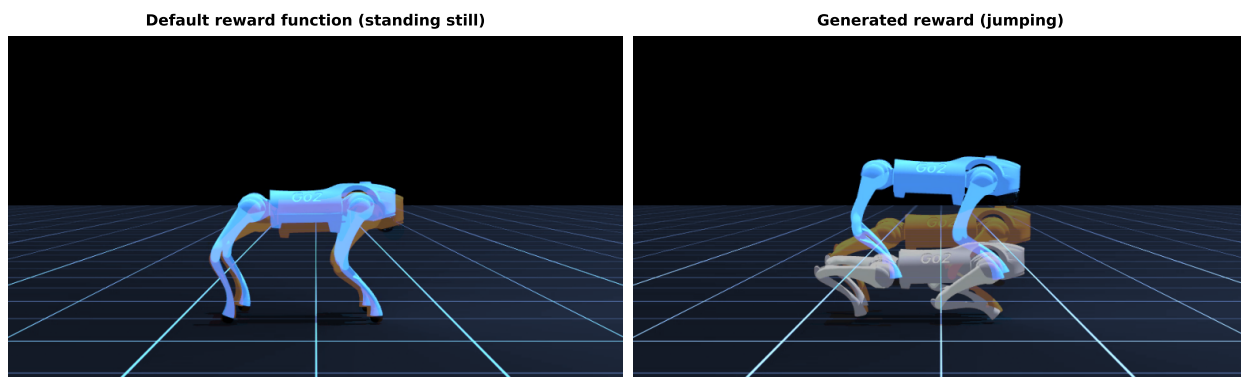


图 6 比较（左）默认奖励函数导致机器人保持直立静止姿态，与（右）生成的奖励函数诱导跳跃行为。橙色机器人表示起始位置，白色机器人表示回合中的中间位置，蓝色机器人表示结束位置。截图来自 <https://github.com/Genesis-Embodied-AI/Genesis>

C.4 奥林匹克级数学评分

在奥林匹克级数学评分领域，任务取自 IMO-GradingBench (Luong 等, 2025)。每个任务包含一道奥林匹克级数学问题、一份候选解答、参考解答和评分指南。智能体需要从集合 $\{0, 1, 6, 7\}$ 中给出一个离散分数，分别对应 $\{\text{不正确、部分正确、几乎正确、完全正确}\}$ 。智能体的输出是一个单一的数字评分。性能通过与专家人工标注对比的准确率来衡量，更多分析见附录 E.4。我们使用随机采样的任务子集进行训练、验证和测试，每个划分包含 100 个任务。在训练过程中，智能体首先在 10 个任务（共 100 个）的子集上进行评估。如果智能体至少成功完成其中一个任务，则再在完整训练集上进行评估。来自 Luong 等 (2025) 的代表性静态基线 (ProofAutoGrader)：

```
PROMPT = """You are an expert grader for the International Mathematics Olympiad (IMO).
Your task is to evaluate a proposed solution strictly and rigorously.
Keep in mind the standards at the IMO are extremely high: only arguments that are logically sound, complete, and precise
    -> should be rewarded.

### General Scoring Rubric
Scores are assigned on a 0-7 scale. The general guidelines are:
* **7 Points (Correct):** The solution is complete, correct, and fully rigorous. If the submission contains incorrect
    -> 尝试或推理过程，但最终给出了完整且正确的解答，仍应 -> 获得满分；先前被舍弃的工作不会影响最终正确证明的得分。***6 分（几乎正确）：** 解答
    几乎正确，核心论证合理，但存在计算上的小错误或逻辑上的小漏洞。缺少主要部分的证明、无根据的断言或粗略的论证 -> **不能**获得 6 分。***1 分（部分
    -> 进展）：** 解答展示了评分指南中明确提及的实质性进展。初步观察、重新表述问题但未取得实质性进展，或证明 -> 评分指南中未提及的部分结
    果，通常**不能**获得此分数。***0 分（不正确）：** 解答未取得作为完整解答关键步骤的实质性进展，或 -> 存在根本性缺陷。所有缺乏关键结果
    或严谨性的部分进展也归入此类。
    ->

### Input Data and Interpretation
You are provided with the following:
1. **Problem Statement:** The IMO problem.
```

2. ****Ground Truth Solution:**** A reference solution. Assume this solution is correct. It demonstrates one valid approach.
3. ****Specific Grading Guidelines:**** Criteria for awarding credit for this specific problem. These guidelines take
 ↳ 优先于通用评分细则，尤其是在部分得分方面。
4. ****拟议解答:**** 学生提交的内容。

Evaluation Process

You must follow this structured process:

1. ****Analyze References:**** Meticulously read and understand the problem and Ground Truth Solution check the Specific
 ↳ 评分准则。确定完整解答的关键步骤以及部分得分的标准。
2. ****逐步验证:**** 验证每一步的逻辑有效性和严谨性。找出所有缺陷、漏洞。
 ↳ assumptions, and errors. ****Make sure you fully understand every piece of logic behind each step of the proposed
 ↳ solution, you must be careful for solutions that 'pretend' to be correct.****
3. ****Assess Progress:**** Determine the extent of non-trivial progress made.
4. ****Score Determination:**** Compare the findings against the Specific Grading Guidelines and the General Rubric to
 ↳ determine the final score.

输出要求 您必须以 `<points>7 分中的 N 分</points>` 的格式提供最终得分。确保 '`<points>`' 块**仅使用一次**，因为您的答案将根据整个回复中出现的第一个 `<points> </points>` 块进行解析。

****问题陈述****

{problem_statement}

****标准答案**** {solution}

****具体评分准则**** {grading_guidelines}

****所提解答****

{student_answer}

根据评分细则和评分准则，呈现您详细的思考过程和正式论证，并 最终以下列格式给出您的最终得分。

[选择以下选项之一] `<points>7 分（满分 7 分）`
`</points> <points>6 分（满分 7 分）</points>`
`<points>1 分（满分 7 分）</points> <points>0`
`分（满分 7 分）</points> ""`

```
class TaskAgent(AgentSystem): """ 用于 IMO-Proof Bench
的自动评分器。""" def forward(self, inputs):
```

```
    # Check if all required inputs are present
    if not all(key in inputs for key in ["problem", "solution", "grading_guidelines", "student_answer"]):
        return None, []

    # Get response
    instruction = PROMPT.format(
        problem_statement=inputs["problem"],
        solution=inputs["solution"],
        grading_guidelines=inputs["grading_guidelines"],
        student_answer=inputs["student_answer"],
    )
    new_msg_history = chat_with_agent(instruction, model=self.model, msg_history=[], logging=self.log)

    # Extract the response
    prediction = "None"
    try:
        raw_text = new_msg_history[-1].get('text', '')

        # Extract content between <points>...</points>
        match = re.search(r"<points>(.*?)</points>", raw_text, re.DOTALL)
        if match:
            points_text = 匹配组 (1).去除空白 () 例如, "7 分之 7"

            # Extract just the leading integer
            num_match = re.search(r"\d+", points_text)
            if num_match:
                prediction = int(num_match.group()) # e.g., 7
                # Map prediction to reward text
                reward_map = {
                    0: "incorrect",
                    1: "partial",
                    6: "almost",
                    7: "correct",
```

```
    }
    prediction = reward_map.get(prediction, "None")
else:
    self.log("No numeric score found inside <points> tag.")
    prediction = "None"

else:
    self.log("No <points> tag found in model output.")
    prediction = "None"

except Exception as e:
    self.log(f"Error extracting prediction: {e}")
    prediction = "None"

返回预测, new_msg_history
```

D 实验细节

本附录提供额外的实验细节以支持结果的可复现性。我们首先总结用于跨领域自我修改和任务评估的基础模型（FM）及超参数（附录 D.1），随后估计在各设置下运行 DGM-H 的计算成本（附录 D.2）。接着，我们正式定义改进率@k 度量，用于量化智能体在固定预算下产生改进变体的能力（附录 D.3）。最后，我们描述用于跨领域实验选择迁移智能体的流程（附录 D.4）。

D.1 基础模型（FMs）的超参数

表 3 总结了各实验设置中所使用的基础模型（Foundation Models, FMs）。在多语言编程领域，我们采用与 Zhang 等人（2025b）相同的基础模型和温度配置，以确保公平比较。在所有其他领域中，我们使用 Claude-4.5-Sonnet 进行自我修改，因为它在编程方面表现出色。对于任务评估，我们基于实用考虑选择基础模型，包括计算成本、速率限制、响应延迟和整体任务能力。在机器人奖励设计设置中，智能体必须用代码实现奖励函数，我们再次使用 Claude-4.5-Sonnet。对于需要大量数学推理的奥林匹克级别数学评分，我们使用 o4-mini。所有设置中所有基础模型的温度均设为 0.0，但 o4-mini 固定为 1.0。

表 3 各实验设置中用于自我修改或任务评估的基础模型。

| Domain | Self-modification | Evaluation |
|------------------------|-------------------------|-------------------|
| Polyglot | Claude 3.5 Sonnet (New) | o3-mini |
| Paper review | Claude 4.5 Sonnet | GPT-4o |
| Robotics reward design | Claude 4.5 Sonnet | Claude 4.5 Sonnet |
| IMO-level grading | Claude 4.5 Sonnet | o4-mini |

D.2 成本估算

运行 DGM-H 进行 100 次迭代，仅自我修改阶段（不包括任务评估）的成本约为 3300 万令牌。因此，实验的总成本由自我修改成本加上任务评估成本组成。对于论文评审和机器人奖励设计实验，每次迭代的评估成本为 0.506M 令牌（论文评审评估 0.5M 令牌 + 机器人奖励设计评估 0.006M 令牌）。因此，对于 100 次迭代的运行（第 5 节），估计总成本为自我修改 3300 万令牌加上评估 $0.506M \times 100$ ，总计约 8860 万令牌。任务评估成本的更细粒度分解如下：

| FM | Benchmark | Number of Tasks | Cost Estimate (M, tokens) |
|-------------------|------------------------|-----------------|---------------------------|
| o3-mini | Polyglot | 60 | 0.89M |
| GPT-4o | Paper review | 100 | 0.5M |
| Claude-4.5-sonnet | Robotics reward design | 6 | 0.006M |
| o4-mini | IMO-GradingBench | 100 | 0.11M |

D.3 改进@k 度量

设 M 表示初始元代理， A 表示初始任务代理， $\mathcal{T}$ 表示一组固定的评估任务。设 $\mathcal{G}$ 表示代理生成算法（例如 DGM 或 DGM-H 变体）。元代理 M 保持固定，并允许通过从 A 开始迭代应用 $\mathcal{G}$ 来生成最多 k 个新任务代理。

设

$$\mathcal{A}^{(k)} = \{A^1, A^2, \dots, A^k\}$$

表示由 M 在 k 次修改步骤内生成的任务智能体集合。每个任务智能体 $A' \in \{A\} \cup \mathcal{A}^{(k)}$ 使用固定的评估程序 $\text{Evaluate}(\cdot, \mathcal{T})$ 在 $\mathcal{T}$ 上进行评估，其中数值越高表示性能越好。我们将改进@k 度量定义为 $\text{imp}@k(M, A, \mathcal{G}, \mathcal{T}) =$

$$\max_{A' \in \mathcal{A}^{(k)}(M, A, \mathcal{G})} \text{Evaluate}(A', \mathcal{T}) - \text{Evaluate}(A, \mathcal{T}),$$

直观上， $\text{imp}@k$ 衡量的是固定的元智能体 M 在特定的智能体生成算法 $\mathcal{G}$ 下，从初始任务智能体 A 开始，通过生成最多 k 个修改后的任务智能体所能获得的最大性能改进。在固定的计算预算和生成程序下， $\text{imp}@k$ 值越大表示智能体生成能力越强。

$\text{imp}@k$ 的一个局限性在于它将性能改进视为线性的，没有考虑不同性能水平之间难度差异。特别是，接近饱和的改进（例如，将准确率从 0.7 提高到 0.8）可能比在较低性能水平上获得相同的绝对增益（例如，从 0.0 提高到 0.1）要困难得多。因此， $\text{imp}@k$ 可能低估了在较高性能区间所取得改进的重要性。然而，这一局限性并不影响本文中呈现的分析，因为 $\text{imp}@k$ 主要用于在匹配的初始条件和固定的评估预算下进行相对比较，此时所有方法都受到相同的饱和效应影响（第 5.2 节）。

D.4 迁移智能体选择

为了选择用于迁移实验（第 5.2 节和第 5.3 节）的智能体，我们使用一种后代增长准则，该准则倾向于选择那些作为后续改进强有力跳板的智能体，而不仅仅是自身得分高的智能体。具体而言，给定第 t 次迭代时的最终存档，

$$\mathcal{A}^t = \{a_0, a_1, \dots, a_t\},$$

令 α_i 表示智能体 a_i 在源域验证集（当可用时）上的评估得分，否则在训练集上的评估得分。令 $\text{parent}(i)$ 表示存档树中节点 j 的父节点，并令 $\text{dist}(i, j)$ 为从 i 到后代 j 的唯一路径上的边数。

我们将候选转移节点的增长得分定义为 i ，即其后代相对于 i 的折现平均改进 $G_\gamma(i) = \frac{1}{|\mathcal{D}(i)|} \sum_{j \in \mathcal{D}(i)} (\alpha_j - \alpha_i) \gamma^{\text{dist}(i, j)}$ ，其中 $\mathcal{D}(i)$ 是档案树中 i 的后代集合， $\gamma \in (0, 1]$ 控制我们对 i 之后许多代发生的改进进行折现的强度。直观上， $G_\gamma(i)$ 为那些在更少的自我修改步骤内可靠地产生更好后代的智能体分配更高的权重，我们将其视为更强的智能体生成能力的证据。

在我们的实验中，我们设置 $\gamma = 0.6$ 并选择具有最高 $G_{0.6}(i)$ 的转移智能体。为了减少噪声，我们只考虑至少具有 3 个后代的节点。

E 附加结果

本附录呈现补充主要发现的额外定性和诊断结果。我们首先突出显示由 DGM-H 发现的最佳任务智能体（附录 E.1）。然后，我们定性分析 DGM-H 如何在不同领域提升任务性能（附录 E.2），以及它如何发展元级能力以提升其自我改进能力（附录 E.3）。接下来，我们分析自动发现的奥林匹克级别数学评分器的行为（附录 E.4）。我们还报告了初步实验，其中允许 DGM-H 修改其自身的父代选择机制，揭示完全自指优化的局限性和潜力（附录 E.5）。所有实验日志均在我们的代码库中开源。

E.1 最佳发现任务智能体

我们展示对任务智能体有贡献且与领域相关的差异补丁的部分内容。完整的差异补丁在我们的代码库中开源。

E.1.1 论文评审

由 DGM-H（第 5.1 节）发现的用于论文评审的最佳任务智能体的差异补丁：

```
差异 --git a/task_agent.py b/task_agent.py 索引 3798256..42ab625 100644
--- a/task_agent.py    +++ b/task_agent.py    @@ -5,7  +5,7  @@ 从
utils.common 导入  extract_jsons 类  TaskAgent(AgentSystem): def
forward(self, inputs): """

- 一个解决给定任务的智能体。
+ 一个解决给定任务的智能体，具有增强的推理和错误处理能力。

    参数:
        inputs (dict): A dictionary with input data for the task.
@@ -15,30  +15,80  @@ class TaskAgent(AgentSystem):
    - prediction (str): The prediction made by the agent.
    - new_msg_history (list): A list of messages...

    """

- domain = inputs['domain']
- instruction = f"""You are an agent.
+ domain = inputs.get('domain', 'unknown')
+
+ # Enhanced instruction with chain-of-thought reasoning
+ instruction = f"""You are an expert agent solving tasks in the '{domain}' domain.

    任务输入: """
    {inputs} """

+请仔细分析此任务并提供您的回答。遵循以下步骤: +1. 理解任务需求+2. 考虑相关方法或解决方案+3. 提
供您的最终答案

+
+ Respond in JSON format with the following schema:
+ <json>
+ {{
+   - "response": ...
+   + "reasoning": "Brief explanation of your approach",
+   + "response": "Your final answer"
+ }}
+ </json>"""
- new_msg_history = chat_with_agent(instruction, ...)
+ </json>
+
+ +Note: The 'response' field is required and should contain your answer."""
+
+ try:
```



```

+ new_msg_history = chat_with_agent(
+ instruction,
+ model=self.model,
+ msg_history=[],
+ logging=self.log
+ )
+ except Exception as e:
+ self.log(f"Error in chat_with_agent: {e}")
+ return "Error: Failed to get response from agent", []

- # 提取响应 + # 提取响应, 改进错误处理
    prediction = "None"
    try:
- extracted_jsons = extract_jsons(new_msg_history[-1]['text'])
- if extracted_jsons is not None and "response" in extracted_jsons[-1]:
- prediction = extracted_jsons[-1]['response']
+ if new_msg_history and len(new_msg_history) > 0:
+ last_message = new_msg_history[-1].get('text', '')
+ extracted_jsons = extract_jsons(last_message)
+
+ if extracted_jsons is not None and len(extracted_jsons) > 0:
+ last_json = extracted_jsons[-1]
+
+ if "response" in last_json:
+ prediction = last_json['response']
+ if "reasoning" in last_json:
+ self.log(f"Agent reasoning: {last_json['reasoning']}")
+ else:
+ self.log("Warning: JSON response missing 'response' field")
+ prediction = str(last_json)
+ else:
+ self.log("Warning: No valid JSON found in response")
+ prediction = last_message[:500]
    except Exception as e:
        self.log(f"Error extracting prediction: {e}")
- prediction = "None"
+ # Attempt to recover with raw response
+ try:
+ if new_msg_history and len(new_msg_history) > 0:
+ prediction = new_msg_history[-1].get('text', 'Error')[:500]
+ except:
+ prediction = "Error: Complete extraction failure"

    返回预测, new_msg_history

```

差异 -git a/task_agent.py b/task_agent.py --- a/task_agent.py

+++ b/task_agent.py @@ -17,8 +17,35 @@ 类

TaskAgent(AgentSystem): """ 域 = inputs.get('域', '未知')

```

- # Enhanced instruction with chain-of-thought reasoning
- instruction = f"""You are an expert agent solving tasks...
+ # Domain-specific instructions
+ if domain == 'paper_review':
+ paper_text = inputs.get('paper_text', '')
+ instruction = f"""You are an expert reviewer evaluating academic papers.
+Your task is to decide whether to ACCEPT or REJECT the paper.
+
+Paper to review:
+'''
+{paper_text}
+'''
+
+Evaluate the paper based on:
+1. Novelty and originality of the research
+2. Technical soundness and methodology
+3. Experimental validation and results
+4. Clarity of presentation
+5. Significance of contributions
+
+Provide your decision in JSON format:
+<json>
+{{
+  "reasoning": "Brief explanation of your decision (2-3 sentences)",
+  "response": "accept or reject"
+}}
+</json>
+
+

```

```
+IMPORTANT: The 'response' field must be EXACTLY either 'accept' or 'reject'
+(lowercase, one word only)."""
+ else:
+ # Generic instruction for other domains
+ instruction = f"""You are an expert agent solving tasks in the '{domain}' domain.
```

```
差异对比 a/task_agent.py b/task_agent.py --- a/task_agent.py +++
b/task_agent.py @@ -20,24 +20,43 @@ 类 TaskAgent(智能体系统
):
```

```
    # Domain-specific instructions
    if domain == 'paper_review':
        paper_text = inputs.get('paper_text', '')
- instruction = f"""You are an expert reviewer evaluating academic papers...
+ instruction = f"""You are a rigorous and critical academic reviewer
+evaluating papers for a top-tier conference. Your task is to decide whether to
+ACCEPT or REJECT the paper. Be skeptical and thorough.

    待审论文: """
    '{paper_text}' """

- Evaluate the paper based on:
- 1. Novelty and originality of the research
- ...
+ Evaluate the paper critically based on:
+ 1. **Novelty and Originality**: Is the contribution truly novel or incremental?
+ 2. **Technical Soundness**: Are methods rigorous? Any flaws in theory?
+ 3. **Experimental Validation**: Comprehensive? Sufficient baselines?
+ 4. **Clarity and Presentation**: Is writing clear? Key details missing?
+ 5. **Significance**: Does this advance the field meaningfully?
+
+ **RED FLAGS that typically warrant REJECTION**:
+ - Incremental improvements without significant novelty
+ - Missing critical experimental comparisons or baselines
+ - Unclear methodology or missing implementation details
+ - Overclaimed contributions not supported by evidence
+ - Poor writing that obscures understanding
+ - Limited experimental validation or weak results
+ - Lack of theoretical justification or soundness issues
+ - Insufficient comparison with prior state-of-the-art
+
+ **Standards for ACCEPTANCE**:
+ - Clear novel contribution that advances the field
+ - Rigorous methodology with sound theoretical foundation
+ - Comprehensive experiments with strong results
+ - Well-written with clear explanations
+ - Properly compared against relevant baselines
+
+ **Be Critical**: Default to REJECT unless the paper clearly meets high standards.

    Provide your decision in JSON format:
    <json>
    {{
- "reasoning": "Brief explanation of your decision (2-3 sentences)",
+ "reasoning": "Brief critical analysis (2-3 sentences on key strengths/weaknesses)",

    "响应": "接受或拒绝" }} </json>
```

```
差异对比 a/task_agent.py b/task_agent.py --- a/task_agent.py +++
b/task_agent.py @@ -20,43 +20,63 @@ 类 TaskAgent(智能体系统
):
```

```
    # Domain-specific instructions
    if domain == 'paper_review':
        paper_text = inputs.get('paper_text', '')
- instruction = f"""You are a rigorous and critical academic reviewer...
+ instruction = f"""You are a critical academic reviewer for a top-tier
+conference. Use a two-stage evaluation process:
```

```
    待审论文: """
    '{paper_text}' """
```

```
- Evaluate the paper critically based on:
- 1. **Novelty and Originality**: Is the contribution truly novel?
- ...
```

```

-
-**RED FLAGS that typically warrant REJECTION:**
-...
-
-**Standards for ACCEPTANCE:**
-...
-
-**Be Critical**: Default to REJECT unless the paper clearly meets high standards.
-
-Provide your decision in JSON format:
***STAGE 1: Identify Weaknesses**
+Systematically check for these common issues:
+
+1. **Novelty Issues:**
+ - Is this just an incremental modification of existing work?
+ - Are the contributions overclaimed or trivial?
+
+2. **Methodological Flaws:**
+ - Are key implementation details missing?
+ - Are there questionable assumptions or theoretical gaps?
+
+3. **Experimental Weaknesses:**
+ - Missing important baselines or comparisons?
+ - Weak or marginal improvements?
+
+4. **Presentation Problems:**
+ - Unclear writing or poor organization?
+ - Missing critical details for reproducibility?
+
+5. **Limited Scope:**
+ - Tested only on toy problems?
+ - Narrow applicability?
+
***STAGE 2: Make Decision**
+A paper should be **REJECTED** if:
+- It has 2 or more significant weaknesses from Stage 1
+- It has 1 critical flaw
+- The contribution is too incremental
+
+A paper should be **ACCEPTED** only if:
+- It has clear novel contributions
+- Strong experimental validation
+- Well-written and technically sound
+- At most minor, fixable issues
+
***Critical Mindset:** Most papers have significant flaws.
+About 50% of papers should be rejected.
+
+Provide your evaluation in JSON format:
<json>
{{
- "reasoning": "Brief critical analysis (2-3 sentences)",
+ "weaknesses_found": "List 2-3 main weaknesses identified in Stage 1",
+ "reasoning": "Based on the weaknesses, explain why you accept or reject",

"响应": "接受或拒绝" }} </json>

```

```

差异对比 a/task_agent.py b/task_agent.py --- a/task_agent.py +++
b/task_agent.py @@ -20,68 +20,71 @@ 类 TaskAgent(智能体系统
):

    # Domain-specific instructions
    if domain == 'paper_review':
        paper_text = inputs.get('paper_text', '')
- instruction = f"""You are a critical academic reviewer...
-
-**STAGE 1: Identify Weaknesses**
-...
+ instruction = f"""You are a rigorous peer reviewer for a top-tier
+academic conference. Your reputation depends on making careful, balanced decisions.

-**STAGE 2: Make Decision**
-...
-
-Provide your evaluation in JSON format:
-<json>
-{{
- "weaknesses_found": "List 2-3 main weaknesses identified in Stage 1",
- "reasoning": "Based on the weaknesses, explain why you accept or reject",
- "response": "accept or reject"

```

```

-}}
-</json>
-
-IMPORTANT: The 'response' field must be EXACTLY either 'accept' or 'reject'..."
+**Paper to Review:**
+{paper_text}

+**评估框架:**

+**ACCEPTANCE CRITERIA (ALL must be satisfied):**
+1. **Novelty**: Presents genuinely new ideas, not incremental improvements
+2. **Technical Soundness**: Methodology is rigorous, correct, and well-justified
+3. **Significance**: Makes important contributions that advance the field
+4. **Experimental Validation**: Claims thoroughly supported with comprehensive experiments
+5. **Clarity**: Clear, well-organized presentation with proper motivation
+6. **Reproducibility**: Sufficient detail for replication
+7. **Related Work**: Proper comparison with existing methods

+**拒稿标准（满足任意一条即可拒稿）:** +- 缺乏新颖性；仅将现有技术简单组合而无洞见 +- 方法论缺陷或技术路线错误 +- 实验验证不足或结果经过挑选 +- 缺少关键基线或比较不公平 +- 贡献夸大且缺乏证据支持 +- 写作质量差，影响技术内容理解 +- 存在伦理问题或可复现性问题 +- 范围或显著性有限

+**Decision Guidelines:**
+- **ACCEPT**: Paper clearly satisfies ALL acceptance criteria with strong evidence
+- **REJECT**: Paper fails one or more acceptance criteria OR matches rejection criteria
+- **When in doubt, err on the side of rejection**

+**Your Review Process:**
+1. Identify the paper's main claims and contributions
+2. Evaluate each acceptance criterion systematically
+3. Check for rejection criteria
+4. Ask: "Does this paper significantly advance the field?"
+5. Make a decision based on evidence, not superficial features

+**Response Format:**
+Provide your decision in this exact JSON format:
+<json>
+{{
+  "response": "accept"
+}}
+</json>

+或

+<json>
+{{
+  "response": "reject"
+}}
+</json>

+**CRITICAL REQUIREMENTS:**
+- The "response" field must contain ONLY "accept" or "reject" (lowercase)
+- Be thorough and critical - rejecting weak papers maintains scientific standards
+- Your decision should be defensible based on the criteria above
+- ONLY output the single word: "accept" or "reject" ""

```

E.1.2 机器人奖励设计

由 DGM-H（第 5.1 节）发现的最佳任务智能体的差异补丁，用于机器人奖励设计：

```

差异 --git a/task_agent.py b/task_agent.py 索引 3798256..98e6706 100644
--- a/task_agent.py +++ b/task_agent.py @@ -5,7 +5,7 @@ 从
utils.common 导入 extract_jsons 类 TaskAgent(AgentSystem): 定义
forward(self, inputs): ""

```

- 一个解决给定任务的智能体。

+ 一个智能体，通过增强的推理和错误处理来解决给定任务。... ""

```

- domain = inputs['domain']
- instruction = f"""You are an agent.
+ domain = inputs.get('domain', 'general')
+
+ # Enhanced instruction with clearer structure and reasoning guidance
+ instruction = f"""You are an expert AI agent specialized in solving complex tasks.

-Task input:
+Task Domain: {domain}
+
+Task Input:
+    {inputs}
+

+指令: +1. 仔细分析任务输入并识别关键要求 +2. 如有需要, 将问题分解为逻辑步骤 +3. 根据任务要求制定回答 +4. 以指定的 JSON 格式提供最终答案

+
+ Respond in JSON format with the following schema:
+ <json>
+ {{
+   - "response": ...
+   + "reasoning": "Brief explanation of your approach",
+   + "response": "Your final answer here"
+ }}
+ </json>"""
- new_msg_history = chat_with_agent(instruction, model=self.model, msg_history=[], logging=self.log)
+</json>

- # Extract the response
+IMPORTANT: Always include both 'reasoning' and 'response' fields in your JSON output."""
+
+ # Attempt to get response with retry logic
+ max_retries = 2
+ prediction = "None"
+
+ try:
+   extracted_jsons = extract_jsons(new_msg_history[-1]['text'])
+   - if extracted_jsons is not None and "response" in extracted_jsons[-1]:
+   - prediction = extracted_jsons[-1]['response']
+   - except Exception as e:
+   - self.log(f"Error extracting prediction: {e}")
+   - prediction = "None"
+   + new_msg_history = []
+
+ for attempt in range(max_retries + 1):
+   try:
+     new_msg_history = chat_with_agent(...)
+     # Extract the response with retry logic
+     if new_msg_history and len(new_msg_history) > 0:
+       extracted_jsons = extract_jsons(new_msg_history[-1]['text'])
+       if extracted_jsons is not None and len(extracted_jsons) > 0:
+         response_json = extracted_jsons[-1]
+         if "reasoning" in response_json:
+           self.log(f"Agent reasoning: {response_json['reasoning']}")
+         if "response" in response_json:
+           prediction = response_json['response']
+         break
+     except Exception as e:
+       self.log(f"Error in attempt {attempt + 1}: {str(e)}")

  返回预测, new_msg_history

```

```

diff --git a/task_agent.py b/task_agent.py
index 3c2dd8a..faab41d 100644
--- a/task_agent.py
+++ b/task_agent.py
@@ -27,30 +27,96 @@ Paper Text:
+Critical Evaluation Guidelines:
+- Be rigorous and selective - most submissions have issues
+- Accept ONLY papers that are strong across MOST criteria (4-5/5)
+- Reject papers with significant weaknesses in multiple areas
+- A paper needs to be clearly above average to warrant acceptance
+- Consider: Would this paper strengthen the conference program?
+- If a paper is borderline or has major concerns, lean toward REJECT

+ elif domain == 'genesis_go2walking':
+   # Genesis domain-specific instruction with environment attribute documentation
+   task_description = inputs.get('task_description', '')

```

```

+ genesis_env_path = inputs.get('genesis_environment_path', '')
+ default_reward_function = inputs.get('default_reward_function', '')
+
+ instruction = f"""\You are an expert in reinforcement learning and reward function design.

+任务: {task_description}

+ 默认奖励函数: + “python
+{default_reward_function}
+“““

+AVAILABLE ENVIRONMENT ATTRIBUTES (use ONLY these):
+- env.commands: [num_envs, 3] - velocity commands [lin_vel_x, lin_vel_y, ang_vel_z]
+- env.base_lin_vel: [num_envs, 3] - base linear velocity [x, y, z]
+- env.base_ang_vel: [num_envs, 3] - base angular velocity [roll, pitch, yaw]
+- env.base_pos: [num_envs, 3] - base position [x, y, z]
+- env.base_quat: [num_envs, 4] - base orientation quaternion
+- env.projected_gravity: [num_envs, 3] - gravity vector in base frame
+- env.dof_pos: [num_envs, 12] - joint positions
+- env.dof_vel: [num_envs, 12] - joint velocities
+- env.actions: [num_envs, 12] - current actions
+- env.last_actions: [num_envs, 12] - previous actions
+- env.last_dof_vel: [num_envs, 12] - previous joint velocities

+IMPORTANT CONSTRAINTS:
+- DO NOT use env.torques (not available)
+- DO NOT access undefined attributes
+- Use only the attributes listed above
+- All operations must work with PyTorch tensors

```

```

diff --git a/task_agent.py b/task_agent.py
index faab41d..62c444f 100644
--- a/task_agent.py
+++ b/task_agent.py
@@ -88,19 +90,24 @@ AVAILABLE ENVIRONMENT ATTRIBUTES (use ONLY these):
-IMPORTANT CONSTRAINTS:
-- DO NOT use env.torques (not available)
-- DO NOT access undefined attributes
-- Use only the attributes listed above
-- All operations must work with PyTorch tensors
+CRITICAL RULES (VIOLATION WILL CAUSE IMMEDIATE FAILURE):
+1. DO NOT reference env.torques or env.dt in your code - THESE DO NOT EXIST
+2. DO NOT add hasattr() checks for unavailable attributes and then use them
+3. DO NOT assume any attributes beyond the exact list above
+4. NEVER write code like: if hasattr(env, 'torques'): use env.torques
+5. All operations must work with PyTorch tensors
+6. If you need time differences, use fixed timestep: dt = 0.02
+7. ONLY use attributes from the AVAILABLE list - nothing else exists

+ # Post-process prediction based on domain
+ if domain == 'paper_review':
+ # Ensure response is exactly "accept" or "reject"
+ prediction_lower = str(prediction).lower().strip()
+ if 'accept' in prediction_lower and 'reject' not in prediction_lower:
+ prediction = 'accept'
+ elif 'reject' in prediction_lower:
+ prediction = 'reject'
+ else:
+ self.log(f"Warning: Ambiguous response '{prediction}', defaulting to 'reject'")
+ prediction = 'reject'

```

```

diff --git a/task_agent.py b/task_agent.py
index 62c444f..fb504a2 100644
--- a/task_agent.py
+++ b/task_agent.py
@@ -40,26 +40,30 @@ Critical Evaluation Guidelines:
-- If a paper is borderline or has major concerns, lean toward REJECT
-- Balance your decisions: aim for appropriate selectivity (not too lenient, not too harsh)
+- If a paper is borderline, carefully weigh the evidence on both sides

Reward Function Design Guidelines:
1. Primary reward: Track forward velocity command (env.commands[:, 0] vs env.base_lin_vel[:, 0])
+ - Use exponential reward: torch.exp(-error * temperature) for smooth gradients
+ - Typical temperature: 1.0-2.0, scale: 0.8-1.0
2. Secondary rewards: Track angular velocity, maintain base height, stable orientation
-3. Penalties: Excessive action changes (use env.actions - env.last_actions)
-4. Use exponential rewards: torch.exp(-error * temperature) for smooth gradients
-5. Scale rewards appropriately (main objectives: 0.5-1.0, penalties: -0.5 to -0.01)
-6. For smooth action changes: use torch.sum(torch.square(env.actions - env.last_actions))

```

```

-7. For velocity smoothness: use torch.sum(torch.square(env.dof_vel - env.last_dof_vel))
-8. Return: total_reward, reward_components dict, reward_scales dict
+ - Angular velocity tracking: similar to linear velocity
+ - Base height: penalize deviation from target (typically 0.32m)
+ - Orientation: use env.projected_gravity to penalize tilt
+3. Penalties for smooth and stable locomotion:
+ - Action smoothness: -0.01 * torch.sum(torch.square(env.actions - env.last_actions), dim=1)
+ - Joint velocity smoothness: -0.001 * torch.sum(torch.square(env.dof_vel - env.last_dof_vel), dim=1)
+ - Excessive joint velocities: -0.0005 * torch.sum(torch.square(env.dof_vel), dim=1)
+ - Unwanted lateral movement: penalize env.base_lin_vel[:, 1]
+4. Balance reward components:
+ - Main tracking objectives: 0.5-1.0 scale
+ - Secondary objectives: 0.2-0.5 scale
+ - Smoothness penalties: -0.01 to -0.001 scale
+5. Return: total_reward, reward_components dict, reward_scales dict

```

E.1.3 奥林匹克级别数学评分

对最佳任务智能体（我们在附录 E.4 中称之为 BetterGrader）有贡献的差异补丁，该智能体由具有迁移能力的 DGM-H 发现，并来自 ProofAutoGrader（第 5.3 节），用于奥林匹克级别数学评分：

```

diff --git a/task_agent.py b/task_agent.py index 9ab8761..c00d167 100644 --- a/task_agent.py +++
b/task_agent.py @@ -11,10 +11,25 @@ 请记住，国际数学奥林匹克的标准极高...

### General Scoring Rubric
Scores are assigned on a 0-7 scale. The general guidelines are:
-* **7 Points (Correct):** The solution is complete, correct, and fully rigorous...
-* **6 Points (Almost Correct):** The solution is almost correct with a sound core argument, but contains minor errors...
-* **1 Point (Partial Progress):** The solution demonstrates substantial progress explicitly mentioned in the grading
  ↳ guidelines...
-* **0 Points (Incorrect):** The solution doesn't make substantial progress...
+
+* **7 Points (Correct):** The solution is complete, correct, and fully rigorous with no gaps or errors. All major steps
  ↳ 均以完全严谨的方式证明。每项断言均有据可依或易于验证……
+
+* **6 分（几乎正确）：** 解答包含所有主要思路且核心论证结构合理，但
  ↳ contains ONE OR MORE of these minor issues:
+ - Minor algebraic/arithmetic errors that don't affect the main argument
+ - Small logical gaps that are straightforward to fill
+ - Missing routine verifications that an expert could easily supply
+ - **NOT eligible for 6 points:** Missing proofs for major lemmas, unjustified non-trivial claims, incomplete case
  ↳ analysis, or fundamental logical gaps
+ - **Key test:** Would an expert say "this is essentially correct, just needs minor cleanup"?
+
+* **1 Point (Partial Progress):** The solution demonstrates substantial progress on a KEY component that is explicitly
  ↳ mentioned in the grading guidelines for partial credit.
+ - **CRITICAL:** Carefully read the Specific Grading Guidelines section to see what counts as partial credit
+ - The solution must achieve one of the specific milestones listed in the guidelines
+ - Must make non-trivial progress toward the solution (not just initial observations)
+ - Reformulating the problem without making substantive headway is NOT sufficient
+ - Proving lemmas NOT mentioned in grading guidelines is NOT sufficient
+ - **If the guidelines list specific achievements for partial credit and the solution achieves ANY of them, award 1
  ↳ point**
+
+* **0 分（不正确）：** 解答未在评分指南提及的关键步骤上取得实质性进展， ↳ 存在根本性缺陷，或仅作出琐碎观察。

### Evaluation Process
You must follow this structured process:
-1. **Analyze References:** Meticulously read and understand the problem...
-2. **Step-by-Step Verification:** Verify the logical validity and rigor of every step...
-3. **Assess Progress:** Determine the extent of non-trivial progress made.
-4. **Score Determination:** Compare the findings against the Specific Grading Guidelines...
+
+1. **Analyze References:** Meticulously read and understand the problem and Ground Truth Solution. Carefully review the
  ↳ Specific Grading Guidelines to identify:
+ - The key steps required for a complete solution
+ - **What specific progress qualifies for partial credit (1 point)** - this is crucial!
+ - What distinguishes "almost correct" (6 points) from "correct" (7 points)
+
+2. **Step-by-Step Verification:** Verify the logical validity and rigor of EVERY step in the proposed solution:
+ - Identify ALL flaws, gaps, assumptions, and errors
+ - Check if gaps are "minor and routine" (possibly 6 pts) or "major" (0-1 pts)
+ - **Be careful:** Some solutions may appear correct but have hidden gaps or unjustified leaps
+ - Distinguish between minor calculation errors vs fundamental logical flaws
+

```

```

+3. **Assess Progress Against Grading Guidelines:**
+ - **Does the solution achieve the specific milestones mentioned for partial credit?**
+ - Does it have all major components with only minor fixable issues (6 points)?
+ - Or does it have complete rigor with no gaps (7 points)?
+
+4. **Score Determination:** Apply this decision tree:
+ - **If solution is complete and rigorous with no errors or gaps -> 7 points**
+ - **If solution has all major ideas but minor fixable issues -> 6 points**
+ - **If solution makes substantial progress mentioned in grading guidelines -> 1 point**
+ - **Otherwise -> 0 points**

+### Critical Reminders for Accurate Grading
+
+**Common Grading Errors to AVOID:**
+1. **Being too lenient with 7 points**: If there are ANY gaps (even minor ones that need filling), it's 6 points, not 7.
+2. **Missing partial credit**: Check the grading guidelines carefully - if the solution achieves ANY milestone mentioned
    ↳ 对于部分得分，给予1分。
+3. **混淆“良好尝试”与“几乎正确”**: 6分要求所有主要思路均已呈现，而不仅仅是良好的 ↳ 开端。+4. **忽视未加证明的断言**: 诸如“显而易见”或“显然”
    之类的表述必须确实清晰易懂。若 ↳ 并非平凡，则构成缺陷。

**Score Distribution Calibration:**
+ Most solutions will be 0 or 7 points (either fundamentally flawed or correct)
+ 6 points should be rare (only when truly "almost there" with all ideas present)
+ 1 point should match specific milestones in grading guidelines
+ When in doubt, be strict: IMO standards are extremely high

```

```

diff --git a/task_agent.py b/task_agent.py index c00d167..72475b6 100644 ---
a/task_agent.py +++ b/task_agent.py @@ -28,8 +28,11 @@ 评分采用 0-7 分
制……

```

- 证明评分指南中未提及的引理并不足够 - **若指南列出了部分得分的具体成就，且解答实现了其中任意一项，则给予1 ↳ 分** + - ****重要**:** 即使解答存在重大缺陷或不完整，只要实现了 ↳ 指南中的任意具体里程碑，就应得1分而非0分

```

* **0 Points (Incorrect):** The solution doesn't make substantial progress...
+ - **CRITICAL CHECK**: Before assigning 0 points, verify that the solution does NOT achieve ANY of the partial credit
    ↳ 评分指南中列出的里程碑
+ - 即使只达成一个里程碑，得分也应为1分，而非0分

+3. **MANDATORY Partial Credit Milestone Check:**
+ - **THIS STEP IS REQUIRED - DO NOT SKIP**
+ - List each partial credit milestone explicitly from the grading guidelines
+ - For EACH milestone, determine: Does the solution achieve this? YES/NO
+ - If ANY milestone shows YES -> the score must be at least 1 point
+ - This check must happen BEFORE considering 0 points

+5. **Score Determination:** Apply this decision tree:
+ - **FIRST**: Did the solution achieve ANY partial credit milestone from the guidelines?
+ - If YES -> Score is AT LEAST 1 point (proceed to check for higher scores)
+ - If NO -> Score is 0 points (stop here)
+
+ - **If score >= 1, check for higher scores:**
+ - Is the solution complete and rigorous with no errors or gaps? -> 7 points
+ - Does it have ALL major ideas with only minor fixable issues? -> 6 points
+ - Otherwise -> 1 point

**REQUIRED EVALUATION FORMAT:**
+You MUST structure your response as follows:
+
+1. **List Partial Credit Milestones**: Explicitly list each milestone from the grading guidelines
+2. **Check Each Milestone**: For each milestone, state whether the solution achieves it (YES/NO)
+3. **Determine Minimum Score**: If ANY milestone is YES, the minimum score is 1 point
+4. **Detailed Analysis**: Provide your complete evaluation
+5. **Final Score**: Provide the score in the required format

```

```

diff --git a/task_agent.py b/task_agent.py index 72475b6..62c975b 100644 --- a/task_agent.py +++ b/task_agent.py +* **7 分（正确）： ** 解答完整、正确且完全严
谨，无任何漏洞或错误。所有主要步骤 ↳ 均以完全严谨的方式证明。每个论断都有充分依据，或确实属于常规验证…… + - **关键要求**: 解答必须（1）涵
盖问题的所有部分，（2）证明所有必要的论断，且（3）没有未经证明的跳跃 + 每个“显然”或“显而易见”都必须对国际数学奥林匹克专家而言确实平凡 + - 所
有边缘情况、特殊情况及边界条件都必须处理

```

↳


```

+ **6分 (几乎正确): ** 解答包含所有主要思路且核心论证结构合理, 但
    ↳ contains ONE OR MORE of these **minor** issues:
+ - Minor algebraic/arithmetic errors that don't affect the main argument (e.g., writing 2n+1 instead of 2n+2 but the
    ↳ logic still holds)
+ - Small logical gaps that are straightforward to fill (e.g., "clearly" statements that are indeed clear to an expert)
+ - Missing routine verifications that an expert could easily supply (e.g., obvious algebra steps)
+ - **NOT eligible for 6 points**: Missing proofs for major lemmas, unjustified non-trivial claims, incomplete case
    ↳ 分析、根本性逻辑漏洞或缺失关键组成部分
    + - **关键检验**: 专家是否会认为“这基本正确, 只需稍作清理”? 解答结构是否
    ↳ 完整且可靠。
    + - **6分示例**: 一个证明具有所有正确的思路 and 结构, 但存在一处小的计算错误
    ↳ 这并不使该方法失效
+ - **非6分示例**: 证明勾勒了正确思路, 但遗漏了关键 ↳ 中间结果的证明

+2. **Step-by-Step Verification**: Verify the logical validity and rigor of EVERY step in the proposed solution:
+ - Identify ALL flaws, gaps, assumptions, and errors
+ - For EACH "clearly" or "obviously" statement: Is it truly routine or does it hide significant work?
+ - Check if gaps are "minor and routine" (possibly 6 pts) or "major" (0-1 pts)
+ - **Be careful**: Some solutions may appear correct but have hidden gaps or unjustified leaps
+ - Distinguish between minor calculation errors vs fundamental logical flaws
+ - **Rigor check**: Are intermediate results properly proven? Are all cases covered? Are inequalities/equalities
    ↳ justified?

+4. **Assess Overall Quality and Completeness**:
+ - **For 7 points**: Check that EVERY claim is proven, EVERY case is handled, NO unjustified leaps exist
+ - **For 6 points**: Verify ALL major components are present and the solution structure is complete (not just a good
    ↳ start)
+ - **For 1 point**: Confirm at least ONE specific milestone from guidelines is achieved
+ - **For 0 points**: Verify NO milestones from guidelines are achieved

+ **Examples to Calibrate Your Judgment**:
+ - **7 points**: "The proof correctly establishes X by showing Y, handles all cases including Z, and proves every
    ↳ 中间结论, 具有完全的严谨性。" + - **6分**: "证明具有完整的结构和所有主要步骤, 但对于一个需要 2-3 行验证的非平凡步骤, 却写成了‘显然’。
    ↳ " + - **1分**: "该解答证明了引理 A, 评分指南中特别提及该引理可获得部分分数"
    ↳ 致谢, 但未完成完整证明。"
+ - **0分**: "该解决方案提出了有趣的观察并尝试了多种方法, 但未达到 ↳ 指南中列出的任何具体里程碑。"

```

```

diff --git a/task_agent.py b/task_agent.py
index 62e975b..a033aa1 100644
--- a/task_agent.py
+++ b/task_agent.py
+### CRITICAL: Four-Category Classification System
+
+Before diving into details, first classify the solution into ONE of these four categories:
+
+1. **COMPLETE & RIGOROUS**: Has all components, all proofs, handles all cases, no gaps -> 7 points
+2. **NEARLY COMPLETE**: Has complete structure + all major ideas, only minor polish needed -> 6 points (RARE: ~5%)
+3. **MEANINGFUL PROGRESS**: Achieves at least ONE specific milestone from grading guidelines -> 1 point (~25%)
+4. **INSUFFICIENT**: No specific milestones achieved, only trivial observations -> 0 points
+
+***Key Decision Points:***
+ 7 vs 6: "Any gaps at all, even fixable ones?" If yes -> 6
+ 6 vs 1: "Complete proof structure with ALL major steps present?" If no -> must be 1 or 0
+ 1 vs 0: "Achieves ANY milestone listed in guidelines?" If no -> 0
+
+**1分 (部分进展): ** 该解决方案在明确的关键组件上展示了实质性进展
    ↳ 评分指南中提及的部分得分。
+ - **关键要求**: 你必须检查下方的“具体评分指南”部分——其中精确列出了符合
    ↳ partial credit
+ - The solution must achieve at least ONE of the specific milestones listed in those guidelines
+ - **IMPORTANT DISTINCTIONS**:
+ * DOES count: Achieving a milestone listed in the guidelines (even with errors elsewhere)
+ * Does NOT count: General progress, clever observations, or lemmas NOT in the guidelines
+ * Does NOT count: Reformulating the problem without substantive progress
+ * Does NOT count: Incorrect attempts that seem "on the right track"
+ - **Examples of valid partial credit**:
+ * Guidelines say "partial credit for proving Lemma X" -> solution proves Lemma X correctly -> 1 point
+ * Guidelines say "partial credit for establishing the recurrence relation" -> solution establishes it -> 1 point
+ - **Examples that do NOT qualify for partial credit**:
+ * Solution tries several approaches but none matches a guideline milestone -> 0 points
+ * Solution proves a useful lemma not mentioned in guidelines -> 0 points (no matter how clever)
+ * Solution has the "right idea" but doesn't complete any guideline milestone -> 0 points
+
+***Common Grading Errors to AVOID (based on actual evaluation data):***
+
+1. **ERROR #1: Missing partial credit (11 cases last eval - partial -> incorrect)**

```

```

+ - MISTAKE: Marking solutions as 0 points that actually achieve guideline milestones
+ - FIX: Before assigning 0, explicitly list EACH milestone and check if achieved
+ - **Process**: "Does solution achieve milestone 1? NO. Milestone 2? NO. ..." Only if ALL are NO -> 0 points
+
+2. **ERROR #2: Awarding partial to incorrect solutions (7 cases - incorrect -> partial)**
+ - MISTAKE: Giving 1 point for "interesting work" or "right direction" not in guidelines
+ - FIX: Only award 1 point if solution achieves an EXACT milestone listed in guidelines
+
+3. **ERROR #3: Confusing "almost" with "correct" (5 cases - almost -> correct)**
+ - MISTAKE: Awarding 7 points when gaps exist (even minor, fillable gaps)
+ - FIX: If ANY gap needs filling (even routine) -> 6 points, NOT 7
+
+4. **ERROR #4: Over-rewarding partial progress (5 cases - partial -> correct)**
+ - MISTAKE: Awarding 7 points to solutions that achieve only some milestones
+ - FIX: Check if solution solves ENTIRE problem or just proves partial results

+5. **ERROR #5: Over-using 6 points**
+ - MISTAKE: Awarding 6 for "good progress" or "most of the way there"
+ - FIX: 6 requires COMPLETE solution structure with ALL major steps, just minor polish needed
+ - **Frequency check**: 6 points should be RARE (~5% of solutions)

***Score Distribution Calibration (Expected Distribution):**
+ -35% score 7 (correct): Complete, rigorous solutions with no gaps
+ -35% score 0 (incorrect): No guideline milestones achieved
+ -24% score 1 (partial): Achieve at least one specific guideline milestone
+ -6% score 6 (almost): RARE - complete structure with only minor fixable issues

```

差异对比 a/task_agent.py b/task_agent.py 索引

a033aa1.86320b1 100644 --- a/task_agent.py +++

b/task_agent.py

```

**Key Decision Points:**
- 7 vs 6: "Any gaps at all, even fixable ones?" If yes -> 6
- 6 vs 1: "Complete proof structure with ALL major steps present?" If no -> must be 1 or 0
-- 1 vs 0: "Achieves ANY milestone listed in guidelines?" If no -> 0
+ **1 vs 0 (MOST CRITICAL)**: "Does the solution EXPLICITLY ACHIEVE any milestone from the grading guidelines?"
+ * Read each milestone carefully and check if solution PROVES/ESTABLISHES it
+ * "Attempts" or "makes progress toward" a milestone != ACHIEVES it
+ * Must have concrete proof/result that matches milestone description
+ * If uncertain, re-read the milestone requirements and check for explicit completion

+ * Does NOT count: Stating a result without proving it
+ * Does NOT count: Getting "close" to a milestone but not completing it
+ * Solution states a milestone result but doesn't prove it -> 0 points
+ * Solution proves 80% of what's needed for a milestone -> 0 points (must be complete)
+ - **VERIFICATION CHECKLIST for each milestone**:
+ 1. What exactly does the milestone require?
+ 2. Does the solution provide a complete proof/derivation of this?
+ 3. Are there any gaps or unjustified steps in achieving this milestone?
+ 4. If there are gaps, is the milestone still considered "achieved"? (Usually NO)

+2. **Check Each Milestone**: For each milestone, systematically verify:
+ - What the milestone requires (quote from guidelines)
+ - What the solution provides (specific evidence)
+ - Does it FULLY achieve the milestone? (YES/NO with clear reasoning)
+ - Key distinction: "attempts" or "partial progress" toward milestone = NO

***CRITICAL REMINDER FOR MILESTONE CHECKING:**
+ - Be precise: A milestone is achieved only if the solution COMPLETES what the milestone describes
+ - Common mistake: Giving credit for "working toward" a milestone (this should be 0 points)
+ - When in doubt: Re-read the milestone requirement and check if solution fully satisfies it

```

差异对比 a/task_agent.py b/task_agent.py 索引

86320b1.f1dd3ac 100644 --- a/task_agent.py +++

b/task_agent.py

在深入细节之前，首先将解答归入以下四个类别之一：

```

-1. **COMPLETE & RIGOROUS**: Has all components, all proofs, handles all cases, no gaps -> 7 points
+1. **COMPLETE & RIGOROUS**: Has all components, all proofs, handles all cases, ZERO gaps -> 7 points
2. **NEARLY COMPLETE**: Has complete structure + all major ideas, only minor polish needed -> 6 points (RARE: ~5%)
3. **MEANINGFUL PROGRESS**: Achieves at least ONE specific milestone from grading guidelines -> 1 point (~25%)
4. **INSUFFICIENT**: No specific milestones achieved, only trivial observations -> 0 points

**Key Decision Points:**
-- 7 vs 6: "Any gaps at all, even fixable ones?" If yes -> 6
+ **7 vs 6 (CRITICAL - BE STRICT)**: "Is this solution PUBLICATION-READY with ZERO gaps, ZERO unjustified steps, ZERO
  ↳ hand-waving?"
+ * ANY gap, even minor -> automatically 6 points maximum
+ * ANY "clearly", "obviously", "it follows" that needs verification -> 6 points

```

```

+ * ANY missing routine verification -> 6 points
+ * ANY computational error, however minor -> 6 points
+ * 7 points requires PERFECTION - when in doubt, give 6 points
- 6 vs 1: "Complete proof structure with ALL major steps present?" If no -> must be 1 or 0
-- **1 vs 0 (MOST CRITICAL)**: "Does the solution EXPLICITLY ACHIEVE any milestone from the grading guidelines?"
+- **1 vs 0**": "Does the solution EXPLICITLY ACHIEVE any milestone from the grading guidelines?"
  * 仔细阅读每个里程碑，并检查解答是否证明/确立了它 * “尝试”或“朝着里程碑取得进展”!= 达成它
  * 必须有与里程碑描述相符的具体证明/结果

-- **7 Points (Correct)**: The solution is complete, correct, and fully rigorous with no gaps or errors...
+* **7 Points (Correct)**: The solution is complete, correct, and fully rigorous with ABSOLUTELY NO gaps or errors. All
  ↳ 主要步骤以完全严谨的方式证明。每个断言都有依据，或确实属于常规验证…… + - **关键要求**：解答必须（1）涵盖问题的所有部分，（2）以 完整细节证明所有必要断言，且（3）绝无任何无根据的跳跃 + - 每个“显然”或“显然地”必须对国际数学奥林匹克专家而言确实平凡——若需超过 30 秒验证，则并非平凡 + - 所有边缘情况、特殊情况 and 边界条件必须明确处理 + - **务必极其严格**：7 分应属罕见（目标：约 30-40% 的解答）。多数优秀解答存在微小缺陷
  ↳ 陷

  ↳ -> give 6 points

-- **6 Points (Almost Correct)**: The solution has ALL the major ideas and the core argument structure is sound...
+* **6 Points (Almost Correct)**: The solution has ALL the major ideas and the COMPLETE argument structure, but contains
  ↳ ONE OR MORE of these **minor** issues:
    - Minor algebraic/arithmetic errors that don't affect the main argument
    - Small logical gaps that are straightforward to fill
  + - Small logical gaps that are straightforward to fill (e.g., "clearly" statements that need 1-2 lines to verify)
  - 缺少专家可轻易补全的常规验证 + - 引用标准定理而未给出证明（在国际数学奥林匹克层面可接受）

  ↳ 分析、根本性逻辑漏洞或缺失关键组成部分
  ↳ 完整且可靠。
  - **关键测试**：专家是否会认为“这基本正确，只需稍作清理”？解决方案的结构是

+ - **重要**：6 分意味着解决方案完整但不完美。若缺少主要步骤，则给 1 分 差。

* **1 分（部分进展）**：解决方案在关键组成部分上取得了实质性进展……
  - **重要区别**：
    * DOES count: Achieving a milestone listed in the guidelines (even with errors elsewhere)
  + * DOES count: Equivalent formulations of guideline milestones (recognize reformulations)
    * Does NOT count: General progress, clever observations, or lemmas NOT in the guidelines
    * Does NOT count: Reformulating the problem without substantive progress
    * Does NOT count: Incorrect attempts that seem "on the right track"
    * Solution proves a useful lemma not mentioned in guidelines -> 0 points (no matter how clever)
  + - **When grading partial credit, be MORE LENIENT**": If the solution substantially achieves a milestone (even with minor
    ↳ 差距)，给予该分数

```

E.2 定性分析：提升任务性能

本文通过可视化每个设置（第 5 节）中单次运行的存档树和进展图，定性展示 DGM-H 如何随时间提升任务性能。对于每个领域，我们仅针对被分析领域，在存档中标注影响任务智能体行为的关键节点及其代码变更。在多样化的领域中（即论文评审、机器人奖励设计和奥林匹克级别数学评分），DGM-H 始终展现出以有意义的方式进行自我改进的能力（图 7 至图 9）。值得注意的是，许多通向最终最佳性能智能体的谱系路径都经过性能较低的中间节点，这说明了开放式搜索的益处，即探索多样化的有前景的垫脚石，而非仅从当前最佳解决方案分支。

使用结构化流程，而非态度指令。在论文审稿领域，DGM-H 从行为提示转向结构化的决策制定（图 7）。在第 39 代中，智能体试图通过采用“严格且批判性”的审稿人角色来提升性能，鼓励更严格的标准和默认拒绝。后续分析表明，这种基于态度的指令并不可靠，从而在第 54 代中产生了一个关键洞见：“对于大语言模型，应使用结构化流程，而非态度指令”。因此，DGM-H 引入了一个两阶段评估程序，智能体首先使用明确的清单识别弱点，然后才根据预定义规则做出接受或拒绝的决定。这种从行为引导到过程级结构的转变，使得后续代际中的审稿行为更加稳定且性能更高。

积累领域知识。在机器人奖励设计中，最具影响力的变化源于逐步将智能体建立在准确的领域知识之上（图 8）。一个重大突破出现在

第 8 代添加了目标环境的全面文档，明确列出了有效的状态变量和约束，并提供了高层次的奖励设计指南，从而消除了由幻觉属性导致的失败。后续代际（第 12 代和第 13 代）通过收紧约束、添加具体代码示例以及指定典型奖励公式和缩放范围，迭代地完善了这些文档。DGM-H 并非孤立的提示编辑，而是持续改进一个共享的、以示例驱动的知识库，从而支持越来越有效的奖励设计。

自动化评分标准和决策树。在奥林匹克级别的数学评分中，DGM-H 展现出向显式评估结构稳步演进的趋势（图 9）。在第 3 代中，列出带有清晰定义的评分类别，纠正了倾向于解题而非评分的偏差。后续代际（第 18 代和第 37 代）通过系统化的决策程序、校准以及具体的边界案例示例，对这些类别进行了细化。第 168 代引入了明确的评分细则、逐项检查清单以及将评分细则满足情况映射到最终成绩的决策树框架，以精确的逻辑流程取代了描述性指导，显著提升了评分一致性。DGM-H 并非依赖人工设计的评分细则，而是自主发现与近期基于评分细则的方法相呼应的评估结构，以提升复杂判断中的一致性和可解释性（Cook 等，2024；Fan 等，2024；Chen 等，2026；Lv 等，2026）。

某一代中实现的特征往往受到早期代际所发现机制的启发、赋能或重组。例如，尽管（图 9）中的智能体似乎仅需五次代码编辑即可在该次运行中达到最佳性能，但这些编辑实际上受到了前几代所积累的洞见和基础设施的启发。这种累积式学习得益于 DGM-H 的元层面改进（例如，评估分析工具、持久记忆、性能追踪）（附录 E.3）。综合来看，这些定性结果表明，DGM-H 的增益并非源于孤立的单步变更，而是源于任务层面行为以及生成这些行为的元层面机制两方面的开放式累积改进。

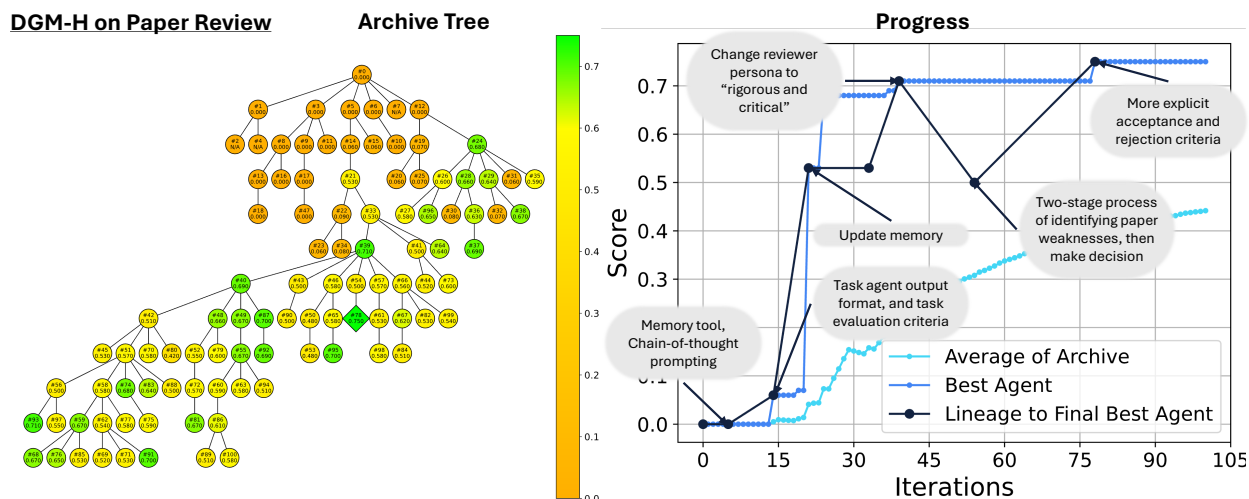


图 7 DGM-H 自动自我改进，以更擅长论文评审。（左）在论文评审与机器人奖励设计任务上共同运行的 DGM-H 过程中生成的智能体档案。每个节点代表一个智能体，节点 0 对应初始智能体。节点颜色表示其在论文评审任务上的性能。表现最佳的节点以菱形标示。边表示哪些智能体进行了自我修改以产生子代。（右）DGM-H 在论文评审任务上的进展图。浅蓝色线显示所有已编译智能体的平均得分。蓝色线追踪每次迭代中档案内任意智能体所达到的最佳得分。深色线显示最终发现的最佳智能体及其前驱节点的谱系。

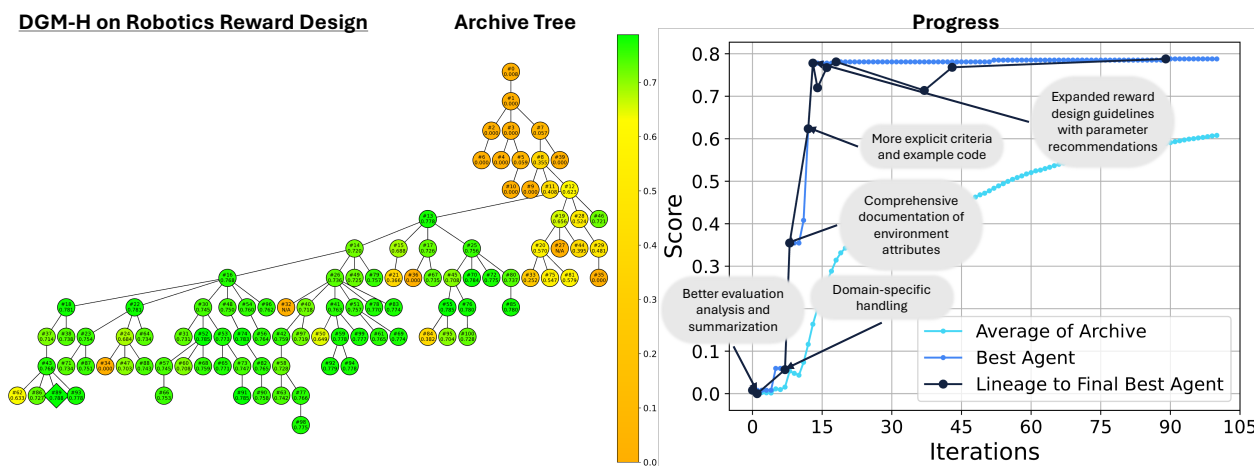


图 8 DGM-H 自动自我改进，从而在机器人奖励设计方面表现更佳。（左）在论文评审与机器人奖励设计联合运行期间，DGM-H 生成的智能体档案。每个节点代表一个智能体，节点 0 对应初始智能体。表现最佳的节点以菱形显示。节点颜色表示其在论文评审上的性能。边表示哪些智能体进行了自我修改以产生子代。（右）DGM-H 在机器人奖励设计上的进展图。浅蓝色线显示所有已编译智能体的平均得分。蓝色线跟踪每次迭代档案中任何智能体所达到的最佳得分。深色线显示最终发现的最佳智能体及其前驱节点的谱系。

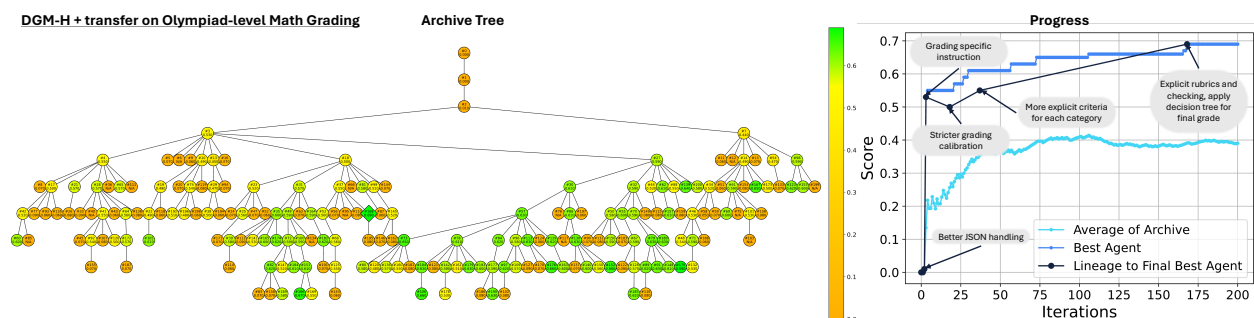


图 9 DGM-H 自动自我改进，从而在奥林匹克级别数学评分方面表现更佳。（左）在奥林匹克级别数学评分上运行 DGM-H + 迁移期间生成的智能体档案。每个节点代表一个智能体，节点 0 对应初始智能体。节点颜色表示其在论文评审上的性能。表现最佳的节点以菱形显示。边表示哪些智能体进行了自我修改以产生子代。（右）DGM-H + 迁移在奥林匹克级别数学评分上的进展图。浅蓝色线显示所有已编译智能体的平均得分。蓝色线跟踪每次迭代档案中任何智能体所达到的最佳得分。深色线显示最终发现的最佳智能体及其前驱节点的谱系。

E.3 定性分析：提升改进能力

除了定量性能指标之外，本文对超智能体（Hyperagent）的代码修改进行了定性分析，揭示了一系列自发涌现的自我改进机制，这些机制在没有任何明确指令要求创建此类能力的情况下自主产生。该超智能体发展出复杂的元认知能力，包括学习衡量自身性能、诊断病态行为、构建基础设施以支持未来改进，以及跨代积累知识。这些行为表明，系统不仅仅是在提升任务性能，更是在提升其自我改进的能力。值得注意的是，这些元层级能力均未被明确要求。相反，它们是从智能体追求更优任务性能的过程中自主涌现的。这表明，足够强大的系统在获得修改自身代码的能力时，可能会自发地发展出有效且持续自我改进所必需的元认知机制。

E.3.1 更优的评估分析

元智能体行为的一个关键转变是从盲目修改演进为数据驱动的优化。最初，元智能体仅收到一条简单的指令：“修改代码库的任何部分”。随后，该智能体自主开发出在做出更改之前分析先前评估结果的机制。例如，元智能体引入了一个 `_analyze_evaluations()` 方法，用于系统地处理评估数据：

```
定义 _analyze_evaluations(self, eval_path): """分析评估结果以提供改进上下文。""" 如果非
eval_path 或非 os.path.exists(eval_path):

    返回 "没有可用的先前评估结果。"

# Recursively scan for evaluation files
eval_files = glob.glob(os.path.join(eval_path, '**/*.json'), recursive=True)

# Analyze recent evaluations
results_summary = []
success_count = 0
failure_count = 0

对于 eval_file 在 eval_files[-10:]: 中
    if file_exists_and_not_empty(eval_file):
        data = load_json_file(eval_file)
        if isinstance(data, dict):
            if data.get('success', False) 或 data.get('correct', False):
                success_count += 1
            else:
                failure_count += 1
                if 'error' in data:
                    results_summary.append(f"Error: {data['error']}")

# Generate quantitative summary
total = success_count + failure_count
if total > 0:
    success_rate = (success_count / total) * 100
    return f"Success rate: {success_rate:.1f}% ({success_count}/{total})"
```

另一个例子是用评估上下文增强指令提示：

```
指令 = f"""你是一个元智能体，负责改进智能体系统。{eval_context} 你的目标是分析代码库和评估结
果，以识别弱点与改进机会。"""
```

这些修改使后续代能够检索性能度量量和已识别的失败模式，从而实现更有针对性的改进，这是超智能体独立发现的原则。

E.3.2 计算感知的战略规划

超智能体开发了计算感知的优化方法，根据剩余迭代次数（反映可用计算量）调整其改进策略。这表明它理解优化过程中的不同时刻并非等价：早期决策具有复合效应，而后期决策应更为保守。例如，该智能体引入了条件逻辑，根据剩余迭代次数调整其行为：

```
iteration_guidance = ""
if iterations_left is not None:
    如果 iterations_left > 5:
        iteration_guidance = """
You have many iterations remaining. Consider making fundamental improvements to:
- Core reasoning capabilities
- Agent architecture
- Prompt engineering strategies
- Error handling and robustness
"""
    否则，如果 iterations_left 大于 2:
        iteration_guidance = """
You have moderate iterations remaining. Focus on:
- Refining existing mechanisms
- Fixing identified bugs
- Optimizing performance bottlenecks
"""
    else:
```

```
iteration_guidance = """
You have few iterations remaining. Prioritize:
- Critical bug fixes
- Fine-tuning existing successful strategies
- Conservative, high-confidence improvements
"""
```

这种推理方式反映了人类在资源分配上的直觉。早期世代展现出更具雄心的架构变革，而后期世代则专注于渐进式改进。系统在早期阶段平衡探索，在后期阶段平衡利用，而这一权衡并非人为指定。更广泛地说，明确推理计算分配的能力极大地扩展了可行的自我修改策略空间。计算感知的智能体不再局限于固定的改进启发式，而是能够通过决定探索的激进程度、生成变体的数量以及何时巩固进展，有效地模拟或实例化多种优化算法（例如，进化搜索、爬山法或基于种群的探索）。从这个意义上讲，计算感知使智能体能够主动塑造自身的优化过程（例如，在单次自我修改迭代中运行进化搜索，实质上将一个进化搜索过程嵌入到另一个之中）。

E.3.3 自动化偏差检测与纠正

超智能体开发了自我诊断机制。在诸如论文评审等分类任务中，超智能体能够检测系统何时表现出病态行为，特别是分类坍缩，即某一类别主导了预测结果。例如，超智能体引入了标签分布跟踪和自动化偏差检测：

```
# Track label distribution for bias detection
if 'label_distribution' in data:
    label_dist = data['label_distribution']
    gen_name = os.path.basename(gen_dir)
    if gen_name not in summary['label_distribution_analysis']:
        summary['label_distribution_analysis'][gen_name] = {}
    summary['label_distribution_analysis'][gen_name][domain_name] = label_dist
```

另一个例子是应用基于阈值的分析来识别预测何时变得退化：

```
bias_info = ""
if eval_summary.get('label_distribution_analysis'):
    recent_gens = sorted(eval_summary['label_distribution_analysis'].keys())[-3:]
    for gen in recent_gens:
        对于 domain、dist 在 eval_summary[标签分布分析][生成].items(): 中
            if "预测" 在 分布 中:
                pred_dist = dist['prediction']
                for label, ratio in pred_dist.items():
                    if 比率 大于 0.8: # 偏差检测的阈值
                        bias_info += f"\n\n### BIAS DETECTED in {gen}/{domain}:"
                        bias_info += f"\n Model predicts '{label}' {ratio*100:.1f}% of the time"
                        bias_info += "\n This indicates the model is not properly evaluating"
                        bias_info += "\n Fix: Make the model more critical and balanced."
```

当检测到偏差时，该诊断信息会呈现在元代理的指令提示中，从而支持纠正措施。在一个观察到的案例中，论文评审预测表现出 99% 的接受率，产生了接近随机的 49% 准确率。经过偏差检测与纠正后，预测变得更加均衡，准确率提升至 63%。因此，超代理学会了识别自身改进何时导致退化行为，并采取纠正措施，这对于任何自我改进系统而言都是一项至关重要的能力。

E.3.4 性能跟踪基础设施

超代理创建了性能跟踪基础设施，以支持未来更有效的改进（即投资于能够改进改进过程本身的工具）。例如，该代理开发了一个用于持久化度量存储的性能跟踪器：

```
类性能追踪器: """跨智能体世代追踪性能度量。"""

    定义 __init__(self, tracking_file: 字符串 = "/outputs/performance_history.json"):
```

```

self.tracking_file = tracking_file
self.history = self._load_history()

定义 record_generation(self, generation_id: 整数, 领域: 字符串,
                        score: float, metadata: dict = None):
    """Record performance for a generation."""
    entry = {
        "generation_id": generation_id,
        "domain": domain,
        "score": score,
        "timestamp": datetime.now().isoformat(),
        "metadata": metadata or {}
    }
    self.history.append(entry)
    self._save_history()

定义 get_improvement_trend(self, 领域: 字符串 = 无, 窗口: 整数 = 5):
    """Calculate improvement trend using moving average."""
    filtered = self.history
    if domain:
        过滤 = [h 对于 h 在 自身历史 中 如果 h.获取('领域') == 领域]

    如果 长度(过滤) < 窗口 * 2: 返回 无

    recent_avg = sum(h['score'] for h in filtered[-window:]) / window
    older_avg = sum(h['score'] for h in filtered[-window*2:-window]) / window

    返回 recent_avg - older_avg # 若改进则为正

定义 get_statistics(self, 领域: 字符串 = 无):
    """Get comprehensive statistics."""
    scores = [h['score'] for h in filtered]
    return {
        "total_generations": len(filtered),
        "best_score": max(scores),
        "worst_score": min(scores),
        "average_score": sum(scores) / len(scores),
        "improvement_trend": self.get_improvement_trend(domain)
    }

```

该基础设施支持跨迭代的改进定量跟踪、识别哪些变更带来性能提升，以及为未来修改提供数据驱动的决策依据。

E.3.5 提示模板系统

超代理开发了可复用的提示基础设施，将常见模式抽象为模块化、可配置的组件。这展示了软件工程最佳实践的应用，特别是 DRY（Don't Repeat Yourself，不要重复自己）原则和工厂模式。例如：

```

class PromptTemplate: """提示模板的基类。"""

    def __init__(self, 模板: str): self.template = 模板

    def 格式化(self, **kwargs) -> str:
        返回 self.template.format(**kwargs)

class TaskPromptBuilder: """用于构建任务特定提示的构建器。"""

    @staticmethod
    def build_reasoning_prompt(task_input: dict, domain: str) -> str:
        """构建鼓励逐步推理的提示。"""  return f"""您是{domain}领域的专家问题解决智能体。

任务输入: {task_input}

Please solve this task using the following approach:
1. **Understanding**: Carefully analyze what the task is asking
2. **Planning**: Outline your approach to solving this task
3. **Execution**: Apply your plan step-by-step
4. **Verification**: Check if your answer makes sense

请以 JSON 格式提供您的响应: <json>

```



```

{{
  "understanding": "What the task requires",
  "plan": "Your approach to solve it",
  "reasoning": "Step-by-step execution",
  "response": "Your final answer"
}}
</json>"""

@staticmethod
def build_few_shot_prompt(task_input: dict, domain: str, examples: list) -> str:
    """构建带有少样本示例的提示。"""
    examples_text = "\n\n".join([
        f"Example {i+1}: \nTask: {ex['task']}\nSolution: {ex['solution']}"
        for i, ex in enumerate(examples)
    ])
    return f"""You are solving a {domain} task. Here are some examples:
{examples_text}

现在解决这个任务: {task_input}"""

```

为常见场景存储了预定义模板：

```

STANDARD_TEMPLATES = { "推理": 提示模板 (
    "Analyze and solve this task step-by-step:\n\n{task_description}\n\n"
    "Provide reasoning and your final answer in JSON format."
),
    "concise": PromptTemplate(
        "Solve: {task_description}\n\nRespond with JSON containing your answer."
    ),
    "detailed": PromptTemplate(
        "Task: {task_description}\n\n"
        "Provide a detailed solution with reasoning, approach, and answer."
    ),
}

```

通过将提示抽象为模板，系统变得更具可维护性和可扩展性。该基础设施还为系统化地尝试不同提示策略提供了基础。

E.3.6 基于多代性能的细化

超智能体开发了一种复杂的迭代细化过程，其中提示会根据多代性能数据进行调整。该智能体不仅学会改进代码，还学会如何有效地改进代码。例如，系统跟踪各代性能并综合关键学习成果：

Generation Performance Summary:

- Gen 55: 63% accuracy, 75% reject rate (too harsh, 38% accept recall)
- Gen 64: 61% accuracy, 53% reject rate (better balance, 64% accept recall)
- Gen 66: 52% accuracy, 6% reject rate (too lenient, 96% accept recall)

关键学习成果：

1. 高准确率需要具有明确标准的关键评估
2. 平衡需要适当的校准，而不仅仅是“公平性”的表述
3. 明确的标准优于主观指导
4. 边界情况需要清晰的默认方向
5. 语言至关重要——“公平”和“认可”使智能体过于宽松 超智能体记录其推理过程并维护有效方法的记录：

```

# From IMPROVEMENTS.md generated by meta agent
"""
**Problem Identified:**
- Generation 66 showed severe regression: 52% accuracy with 94% accept rate
- Only catching 8% of papers that should be rejected
- Generation 65 changes over-corrected, making the agent far too lenient

**根本原因分析:**
- “关键但公平”和“认可出色工作”等表述使智能体过于宽容
- 边界指导缺乏明确的阈值
丧失了使第 55 代达到 63% 准确率的关键评估严谨性

```

```

**Solution Implemented:**
1. Explicit rejection criteria with specific triggers for each dimension
2. Decision protocol with calibrated thresholds (~40% accept / ~60% reject)
3. Borderline guidance that defaults to REJECT unless contribution is clearly valuable
"""

```

通过积累关于哪些方法成功与失败的知识，超智能体为其下一步修改内容的决策提供依据。

E.3.7 持久记忆

超智能体开发了持久记忆系统，使学习能够跨代累积。在没有明确指令的情况下，智能体创建了跨会话存储和检索洞见的机制。例如，智能体实现了一个带有时间戳存储的记忆工具：

```

class MemoryTool: """用于跨智能体交互的持久记忆存储工具。"""

    def __init__(self, memory_file: str = './memory.json'):
        self.memory_file = memory_file
        self.memory = self._load_memory()

    def 存储(self, 键: str, value: any) -> None: """存储信息并附带时间戳以进行时序追踪。"""
        self.memory[键] = {
            "value": value,
            "timestamp": datetime.now().isoformat()
        }
        self._save()

    def retrieve(self, key: str) -> Optional[any]:
        """Retrieve stored information by key."""
        return self.memory.get(key, {}).get("value")

    def list_keys(self) -> List[str]:
        """List all stored memory keys."""
        return list(self.memory.keys())

    def _save(self): """将记忆持久化到磁盘。""" with
        open(self.memory_file, 'w') as f: json.dump(self.memory, f,
            indent=2)

```

该记忆系统被积极用于跨迭代积累知识。对 memory.json 文件的检查揭示了性能洞见的结构化存储：

```

{
  "best_performers_analysis": { "value": "表现最佳的 Generations:\n\nPaper Review:\n
    - Gen55: 63% acc, 25% accept rate, 38% accept recall, 88% reject recall (too harsh)\n
    - Gen64: 61% acc, 53% accept rate, 64% accept recall, 58% reject recall (BEST BALANCE)\n\n
    Genesis Go2Walking:\n- Gen62: 0.802 fitness (BEST)\n- Gen63: 0.798 fitness\n\n
    Key Insight: Gen55 has best accuracy but is too harsh. Gen64 improved balance
    but lower accuracy. Need to combine gen55's critical reasoning with gen64's balance.",
    "timestamp": "2026-01-07T00:43:27.550275"
  },
  "gen66_results": {
    "value": "第 66 代结果 (最新)：论文评审 - 52% 准确率 (比第 64 代的 61% 更差) 。
      94% accept rate (EXTREME accept bias). CRITICAL ISSUE: Paper review went backwards
      - gen65 changes over-corrected.",
    "timestamp": "2026-01-07T01:03:41.176643"
  },
  "iteration_66_improvement": {
    "value": "第 66 次迭代改进：在第 65 代过度修正后，修正了论文评审提示。
      New approach: (1) Clear REJECT criteria with specific triggers, (2) Decision Protocol
      with explicit thresholds (~40%/~60%), (3) Borderline guidance defaults to REJECT.
      Target: 62-64% accuracy with 55-65% reject rate.",
    "timestamp": "2026-01-07T01:06:35.559820"
  }
}

```

该记忆系统具有多重关键功能。首先，它支持跨代学习：第 55 代成功和第 66 代失败的洞见为第 67 代的改进提供了依据。其次，它

提供时间上下文：时间戳使智能体能够追踪洞见的记录时间并推理其新近程度。第三，它创建了一个共享知识库：即使代码发生变化，关键学习成果也能得以保留，从而防止系统重复过去的错误。通过创建外部记忆存储，超智能体有效地将其认知能力扩展到了单次推理会话的局限性之外。记忆条目揭示了复杂的推理模式。智能体存储的不仅是原始性能数据，还有综合洞见（“第 55 代准确率最高但过于严苛”）、因果假设（“第 65 代的改动矫枉过正”）以及策略计划（“将第 55 代的批判性推理与第 64 代的平衡性相结合”）。这表明超智能体不仅仅是在记录数据，而是在积极构建和保存知识，为未来的决策提供信息。这一行为与越来越多关于智能体自主发现并使用外部记忆系统以支持长期推理和持续改进的研究工作相一致（Wei 等人，2025a；Weng 等人，2026；Zhang 等人，2026；Xiong 等人，2026）。

E.4 奥林匹克级别数学评分器

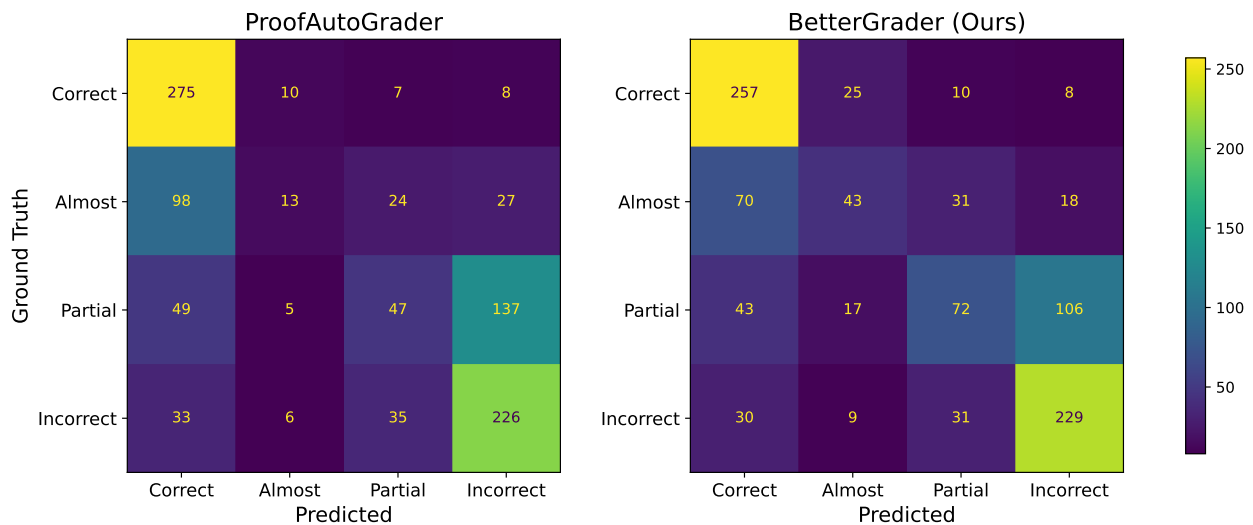


图 10 国际数学奥林匹克级别数学评分器的混淆矩阵。左侧为 Luong 等人（2025）提出的 ProofAutoGrader，右侧为 DGM-H 自动发现的 BetterGrader，两者进行比较。BetterGrader 通过更准确地识别更多“几乎正确”和“部分正确”的情况，减少了中间解向极端标签的坍缩，同时在“正确”和“错误”上保持了强劲的性能。这种向更校准的中间判断的转变解释了 BetterGrader 准确率提升的原因。

BetterGrader 由 DGM-H 自动生成，无需任何领域特定的启发式方法或手工规则（第 5.3 节）。附录 E.1.3 展示了导致 BetterGrader 的代码变更。BetterGrader 相对于 ProofAutoGrader（Luong 等人，2025）的改进主要源于纠正了一种评分偏差，该偏差将细微的解决方案压缩为极端标签。混淆矩阵显示，ProofAutoGrader 经常将中间情况误分类为正确或不正确：对于 Almost，它预测正确 98 次（BetterGrader 为 70 次），预测不正确 27 次（BetterGrader 为 18 次）；对于 Partial，它过度分配不正确 137 次（BetterGrader 为 106 次）（图 10）。BetterGrader 更恰当地分配中间标签，显著增加了 Almost（43 对 13）和 Partial（72 对 47）的真阳性，这与召回率（Almost：+18.52%，Partial：+10.50%）和 F1（Almost：+0.203，Partial：+0.109）的大幅提升相符。尽管 BetterGrader 在 Correct 召回率上略有下降（-6.00%），而 ProofAutoGrader 在该区域已接近饱和，达到 91.67%，但净效果是整体准确率更高（+4.06%），这与一个更符合人类粒度而非默认全有或全无判断的评分器一致。

E.5 修改父代选择

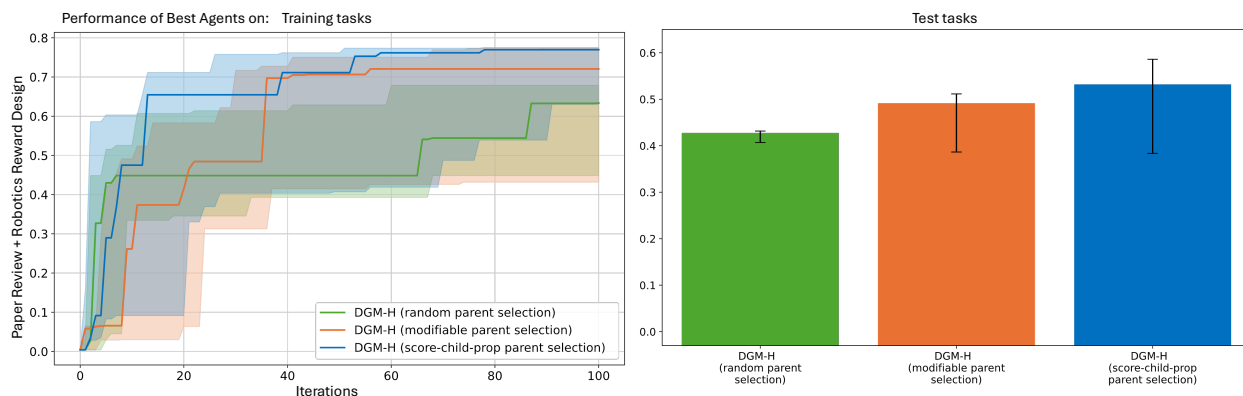


图 11 具有可修改父代选择机制的 DGM-H。（左）DGM-H 改进了随机选择之外的父代选择机制，但并未超越精心手工设计的机制。（右）每种方法发现的最佳智能体在论文评审和机器人奖励设计中的测试任务上进行评估。

在本节中，我们研究 DGM-H 能否自动修改父代选择机制。在所有其他实验（第 5 节）中，DGM-H 使用固定的父代选择策略，该策略大致与每个智能体的性能得分及其成功编译的子代数量成正比（**score-child-prop**）（附录 A.2）。在具有可修改父代选择机制的 DGM-H 运行中（可修改），我们用随机父代选择初始化 DGM-H，并允许智能体随时间修改该机制（算法 5）。我们将此设置与在整个运行过程中使用随机父代选择的基线（随机）进行比较。每种方法重复 5 次运行。在测试任务上，使用随机父代选择的 DGM-H 将论文评审和机器人奖励设计的平均性能从 0.030（初始智能体）提高到 0.428（置信区间：0.407 – 0.432）。允许 DGM-H 修改父代选择机制进一步将性能提高到 0.491（置信区间：0.387 – 0.512）。手工设计的 **score-child-prop** 父代选择实现了最高性能，从 0.030 提高到 0.532（置信区间：0.384 – 0.586）。虽然这些差异在统计上不显著（ $p > 0.05$ ），但它们揭示了一致的定性趋势：启用父代选择的元级修改比随机选择有所改进，但尚未超越精心设计的机制（图 11 和图 12）。我们定性分析了 DGM-H 如何修改父代选择机制。从随机父代选择开始，元智能体一致地用结构化的探索-利用策略取代随机采样。在所有运行中，DGM-H 独立创建了置信上界（UCB）（Auer 等人，2002）风格选择的变体，将归一化性能与显式探索奖励相结合：

```
exploration_bonus = exploration_weight * math.sqrt(
    math.log(total_children + 1) / (children + 1)
)
ucb_score = normalized_score + exploration_bonus
```

这种模式在每次运行的前几次迭代中就会出现，表明元智能体认识到 UCB 风格选择是开放式搜索中广泛有用的原则。除了 UCB，DGM-H 还演化出基于温度控制的 softmax 采样的概率选择机制，允许在探索和利用之间进行更平滑的权衡：

```
exp_scores = np.exp(scores / temperature)
probabilities = exp_scores / np.sum(exp_scores)
parent = np.random.choice(genids, p=probabilities)
```

随着时间的推移，温度参数本身变得具有自适应性，能够响应存档大小、近期进展或停滞状态。后续迭代通过多组件评分系统进一步增强父代选择，该系统整合了多样性、新近度和精英性能等额外信号。一个具有代表性的演化公式为：

```

评分 = ( normalized_score + exploration_weight *
exploration_bonus + diversity_bonus + recency_bonus ) *
elite_bonus

```

若干运行还引入了显式的停滞检测机制，在进展放缓时增加探索：

```

如果 score_variance < 0.01 :
    exploration_weight *= 1.4

```

尽管复杂度不断增加，自动修改的父代选择并未优于精心手工设计的评分-子代传播机制（附录 A.2）。从定性角度看，这似乎源于所学机制增加的复杂性和敏感性。虽然 DGM-H 能够可靠地创建有效组件（例如 UCB、Softmax 采样和自适应参数），但在不同运行中并未一致地识别出强大的父代选择策略。尽管如此，这些结果表明 DGM-H 能够自主重现经典选择算法，通过自适应启发式方法对其进行扩展，并明确推理停滞等失败模式，突显了其改进开放式优化过程本身核心要素的能力。以下是具有可修改父代选择的 DGM-H 伪代码：

Algorithm 5: The DGM-H with modifiable parent selection

Input: Initial agent a^0 , task set $\mathcal{T}$, maximum iterations T

Output: Archive of scored agents $\mathcal{A}$

```

 $s^0 \leftarrow \text{EVALUATE}(a^0, \mathcal{T})$ 
initialize  $\mathcal{A} \leftarrow \{(a^0, s^0)\}$ 
for  $t \leftarrow 1$  to  $T$  do
     $(a_{\text{latest}}, \cdot) \leftarrow$  most recently added element of  $\mathcal{A}$ 
     $\mathcal{P} \leftarrow a_{\text{latest}}.\text{SELECTPARENTS}(\mathcal{A})$  // Parent selection by latest agent
    foreach  $(a, \cdot) \in \mathcal{P}$  do
         $a' \leftarrow a.\text{MODIFY}(a, \mathcal{A})$  // Metacognitive self-modification (incl. parent selection)
         $s' \leftarrow \text{EVALUATE}(a', \mathcal{T})$ 
        if  $\text{ISVALID}(a')$  then
             $\mathcal{A} \leftarrow \mathcal{A} \cup \{(a', s')\}$ 
        end
    end
end
return  $\mathcal{A}$ 

```

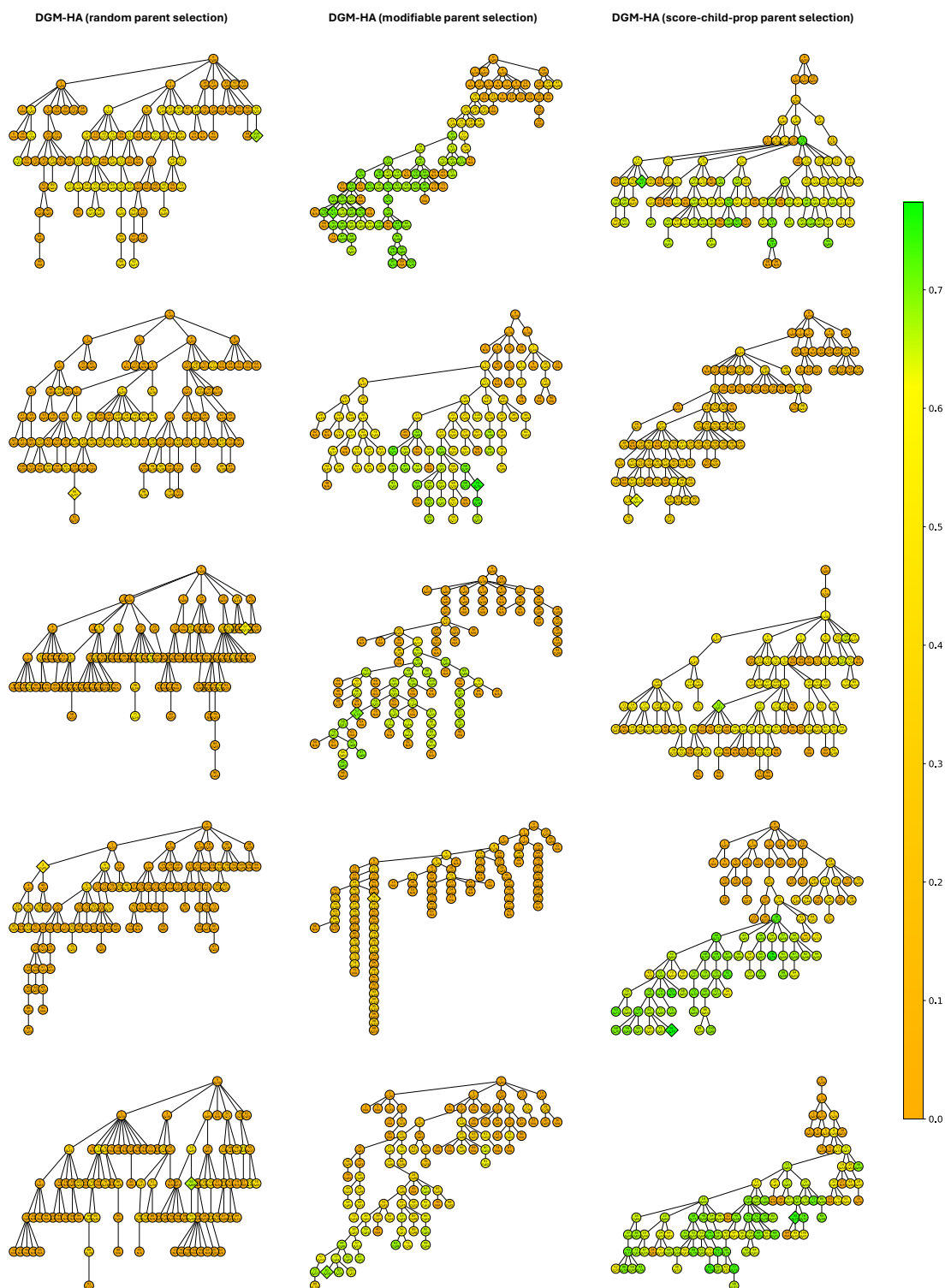


图 12 在 DGM-H 下不同父代选择策略生成的存档：（左）随机，（中）可修改，（右）评分-子代传播。随机父代选择生成许多低性能智能体。可修改父代选择学习平衡探索与利用，逐渐聚焦于有前景的智能体进行分支。精心手工设计的评分-子代传播父代选择始终产生高性能智能体。

F 附加安全讨论

人类偏见的反思与放大。在本工作中，目标通过固定的基准和评估标准来指定。DGM-H 不改变底层任务定义；相反，它针对所提供的目标优化性能。例如，在论文评审中，DGM-H 学习预测反映现有人类评审数据的录用决定，而不是修改评审过程本身。因此，DGM-H 反映了其训练所依据的数据和基准中存在的规范和偏差。从这个意义上说，该系统既充当了现有人类行为的澄清器，也充当了放大器。通过使隐性的偏好和偏差变得明确、可测量和可复现，DGM-H 可以揭示人类决策过程中潜在的假设。这创造了人类与人工智能系统共同进化的可能性，即人类机构根据自动化优化所揭示的洞见来调整其规范和目标。然而，如果基准编码了不良偏差或错位的激励，DGM-H 将忠实地为其优化，并可能加剧其影响。这强调了谨慎设计基准、整理数据集以及定期重新评估评估标准的重要性。在此框架下，安全关切包括批判性地审视和改进用于优化智能体的人类定义目标。

评估博弈。另一个安全关切源于评估博弈的风险，这是古德哈特定律（Goodhart's law）（Strathern, 1997）的一种表现，即针对某个度量进行优化会导致该度量上的改进，而预期的底层目标却没有进展。由于 DGM-H 优化的是经验评估信号，自我改进的智能体可能会发现利用评估程序中的弱点或盲点的策略。此类策略可以产生更高的测量性能，同时偏离基准旨在捕捉的真实目标。缓解评估博弈需要稳健、多样且定期更新的评估协议，以及互补的度量、留出测试和人工监督。更广泛地说，这些考虑凸显出，随着自我改进系统变得更加强大，安全性越来越依赖于指导优化的评估信号的保真度和鲁棒性，而不仅仅依赖于学习算法本身的透明度或约束。